

# 基于删失需求修正、Copula 需求网络与两阶段选择型报童模型的蔬菜自动定价及补货决策

## 摘 要

针对蔬菜保鲜期短、凌晨订货时成本尚未完全揭示且陈列空间有限的经营特征，本文以 878503 条销售流水、55982 条批发价记录和 251 个单品为基础，依次完成删失需求修正、需求关联分析、概率预测、随机定价补货和单品组合优化。文中关键数值均由程序计算，并通过回测和约束检查复核。

针对问题一，分离正向销售与退货，构造销量加权售价、固定篮子价格指数和损耗有效成本；以 **Hurdle-Gamma** 模型分别描述需求是否发生及正需求大小，并将疑似售罄日销量视为潜在需求下界。剔除趋势与日历成分后，采用经验 **Copula** 秩变换与 **Graphical Lasso** 识别条件关联，再用工具变量岭回归估计价格响应。六类日销量均呈右偏；最强条件相关边为花叶类—辣椒类 (**0.397**)，决策用自身价格弹性为 **0.200–0.825**。该关联仅反映剩余波动的同步性，不能据此判定顾客存在互补购买行为。

针对问题二，以动态岭回归、Fourier 日历回归和四周同星期均值组成误差倒数加权预测，**84** 日滚动回测的六品类销量 **WAPE** 为 **17.4%–32.1%**。随后整组抽取 1600 个需求—批发成本联合残差情景，建立“先定补货与加成率、成本实现后再更新售价”的两阶段损耗型随机报童模型。2023 年 7 月 1–7 日总补货量为 **2,645.0 kg**，情景期望利润为 **10,491.61 元**；模型内反事实回测利润较基准策略提高 126.1%，但该结果仍需真实库存数据和在线试验验证，不能直接视为门店会计增益。

针对问题三，从近期可售的 49 个候选单品中建立含外部选项的品类内 **MNL** 份额模型，用销量网络重复度保持组合多样性，并以 **Hurdle-Lognormal** 情景估计单品利润。针对 27–33 种、每种不少于 2.5 kg 且六品类均需覆盖的约束，采用二分搜索与混合整数线性规划进行字典序求解：先保障最弱品类，再最大化利润。最终选择 **33** 个单品，总补货 **331.9 kg**，预计销量满足率为 **43.6%–55.6%**，情景期望利润为 **248.43 元**。水生根茎类的短期亏损进一步揭示了品类保障与利润之间的权衡。

针对问题四，以预期样本信息价值与采集成本的比值评价新增数据。结果表明，应优先采集实时库存与售罄时刻、实际报损与折价销售、供应商可供数量，分别用于修正删失需求、更新损耗残值和设置补货上限。滚动回测、压力测试、求解可行性检查和单品数敏感性分析用于比较需求满足率、损耗和尾部利润风险；由于缺少在线对照试验，该方案仅作为试运行起点。

关键词：删失需求；Copula 网络；成本响应定价；随机报童；字典序组合优化

# 一、问题重述与总体分析

## 1.1 问题重述

蔬菜商品具有保鲜期短、损耗快、采购价日内不确定和需求季节性显著等特点。题目给出 2020 年 7 月 1 日至 2023 年 6 月 30 日的单品信息、销售流水、批发价格及近期损耗率，要求依次完成四项任务：其一，分析六个蔬菜品类及单品销量的分布规律和相互关系；其二，在仅能按品类制订计划时，给出 2023 年 7 月 1 日至 7 日的每日补货总量与定价策略；其三，在可售单品数限定为 27–33 种、每个单品陈列量不少于 2.5 kg 时，制订 7 月 1 日的单品组合、补货量与售价；其四，说明还应收集哪些数据以及这些数据如何改进决策。

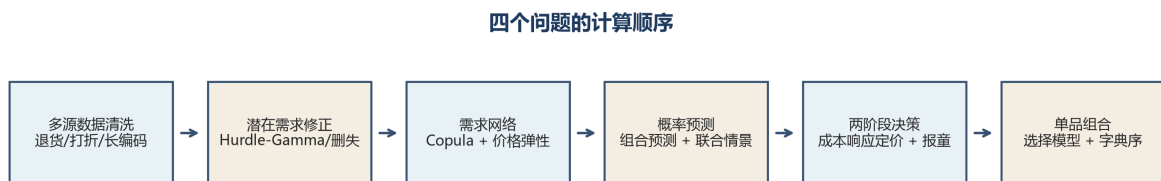
四问按品类分析、补货定价、单品选择和数据补充依次衔接。问题一得到的边际分布、条件需求网络 and 价格弹性用于问题二的联合需求情景；问题二给出的品类需求构成问题三单品决策的总量边界；问题四针对库存截断、动态损耗和供给不确定性补充所需数据。

## 1.2 关键难点

- **销量不等于潜在需求。**无销量可能源于零需求、未上架或提前售罄，直接补零会系统性低估需求；
- **相关不等于替代或互补。**共同周末客流、趋势与促销会放大原始相关，需要先去除可预测共同成分；
- **决策存在先后顺序。**凌晨需先订货，实际批发成本随后实现，售价可以据此调整，故应采用两阶段策略；
- **品种集合会反向改变需求。**未上架单品的顾客可能转向同品类商品，组合选择、定价和补货不能完全分开；
- **同时考虑利润和品类需求。**问题三设置最弱品类服务约束，再在可行范围内提高利润，防止服务目标被利润目标掩盖。

## 1.3 总体技术路线

本文先审计销售、批发价和损耗数据，再依次估计潜在需求、品类关联、价格弹性、未来需求和成本，最后求解补货定价与单品组合。各问的计算顺序见图1。



问题1估计需求特征 → 问题2计算品类方案 → 问题3计算单品方案 → 问题4确定新增数据

图 1 四个问题的建模流程

问题二给出品类层的需求、价格和风险情景；问题三依据可售集合和选择概率分配品类需求总量，并保持单品补货量与品类预测总量一致。

## 二、模型假设与符号说明

### 2.1 基本假设

在不改变附件事实的前提下作如下假设，并在第八节讨论其风险：

- (1) 同一名称后的数字编号代表不同供应来源，单品编码是唯一标识，建模时不合并；
- (2) 需求建模使用正向销售量，退货记录单独用于收入和数据质量核算；
- (3) 题目没有给出隔日残值，主模型令未售商品残值为 0，打折占比只作为清货能力的代理变量；
- (4) 附件 4 中的损耗率在预测周内保持稳定，缺失的单品损耗率由所属品类近期均值补齐；
- (5) 历史周内与年度季节结构在预测周内延续，需求和成本预测误差通过同日联合抽样保留相关性；
- (6) 交叉价格弹性受促销与内生性限制，仅用于经营情景而不宣称完全因果；由于无购物篮编号，不使用 Apriori 规则代表顾客互补关系。

### 2.2 主要符号

表 1 主要符号及含义

| 符号                               | 含义                  | 单位或范围      |
|----------------------------------|---------------------|------------|
| $i, k, t, \omega$                | 单品、品类、日期与随机情景索引     | —          |
| $Y_{it}^+, Y_{it}^-$             | 正向销售量与退货量           | kg         |
| $D_{kt}(\omega)$                 | 情景 $\omega$ 下品类潜在需求 | kg         |
| $C_{kt}(\omega), P_{kt}(\omega)$ | 批发成本与售价             | 元/kg       |
| $L_k, l_i$                       | 品类与单品损耗率            | $[0, 1]$   |
| $q_{kt}, q_i$                    | 品类与单品订购量            | kg         |
| $m_{kt}, m_i$                    | 损耗调整后的有效加成率         | $\geq 0$   |
| $z_i$                            | 单品是否入选的 0-1 变量      | $\{0, 1\}$ |
| $s_i(z, P)$                      | 给定集合与价格时的单品需求份额     | $[0, 1]$   |
| $\Pi(\omega)$                    | 情景利润                | 元          |

### 2.3 模型评价指标

预测精度采用加权绝对百分比误差

$$\text{WAPE} = \frac{\sum_t |y_t - \hat{y}_t|}{\sum_t |y_t|}, \quad (2.1)$$

既避免单日真实值接近 0 造成的 MAPE 爆炸，又能按销量规模衡量总体偏差。经营方案同时报告期望利润、需求满足率、缺货概率和最差 10% 情景平均利润；问题三另报告最差品类需求捕获率，以防平均指标掩盖弱势品类。

## 三、数据预处理与口径统一

### 3.1 数据读取与质量审计

单品编码按字符串读取，避免 15 位编码转成科学计数法后连接失败。附件 2 先与附件 1 做多对一映射，再聚合为“单品一日”和“品类一日”；销售时间转换为距零点秒数，用于构造疑似提前售罄标记。附件 3 按“日期—单品”连接批发价，附件 4 同时

保留品类和单品损耗率。复制入案例目录前后，题目 PDF 及四个附件的 SHA-256 逐一相同，原始附件未被改名或回写。

表 2 数据质量审计结果

| 审计指标    | 结果         |
|---------|------------|
| 销售流水数   | 878503     |
| 正向销售流水数 | 878042     |
| 退货流水数   | 461        |
| 打折销售流水数 | 47366      |
| 单品数     | 251        |
| 品类数     | 6          |
| 批发价记录数  | 55982      |
| 销售起始日   | 2020-07-01 |
| 销售结束日   | 2023-06-30 |

注：来源为附件 1-4，数值由程序读取原始附件后计算。

审计发现销售流水中含有退货和打折记录，部分单品日期缺少批发价。程序保留这些缺失状态，不把无销售记录统一视为零需求。品类日总量分析需要完整日历，因此仅在确认营业期间将品类缺口补为 0；这一处理在模型局限中单独说明。

### 3.2 销售、退货与加权售价

对单品  $i$  在日期  $t$  的流水集合  $R_{it}$ ，销售与退货分别汇总为

$$Y_{it}^+ = \sum_{r \in R_{it}} q_r \mathbf{1}(r \text{ 为销售}), \quad Y_{it}^- = \sum_{r \in R_{it}} |q_r| \mathbf{1}(r \text{ 为退货}). \quad (3.1)$$

需求建模使用  $Y_{it}^+$ ，净收入核算时退货再冲减收入。正向销量加权售价和打折占比分别为

$$P_{it} = \frac{\sum_{r: \text{sale}} q_r p_r}{\sum_{r: \text{sale}} q_r}, \quad H_{it} = \frac{\sum_{r: \text{discount}} q_r}{\sum_{r: \text{sale}} q_r}. \quad (3.2)$$

这能避免一笔 0.1 kg 高价交易与一笔 10 kg 低价交易获得相同权重，也避免将退货量当成负需求参与分布拟合。

### 3.3 损耗有效成本与固定篮子指数

若订购  $q_i$  kg、损耗率为  $l_i$ ，则可售库存与每千克可售商品的有效成本为

$$S_i = (1 - l_i)q_i, \quad c_i^{\text{eff}} = \frac{c_i}{1 - l_i}. \quad (3.3)$$

若直接用当日销售结构加权品类价格，高价单品份额上升会被误判为整体涨价。本文在每个品类内选择累计销量覆盖 98% 且销售天数不少于 20 天的核心单品，以全样本销量份额  $w_{ik}^0$  作固定权重；缺失价格仅在 14 日内前向或后向填补，构造固定篮子售价与成本

$$P_{kt}^L = \frac{\sum_{i \in I_k} w_{ik}^0 P_{it} a_{it}}{\sum_{i \in I_k} w_{ik}^0 a_{it}}, \quad C_{kt}^L = \frac{\sum_{i \in I_k} w_{ik}^0 C_{it} a_{it}}{\sum_{i \in I_k} w_{ik}^0 a_{it}}, \quad (3.4)$$

其中  $a_{it}$  表示该日价格可得。于是损耗调整后的有效加成率为

$$m_{kt}^{\text{eff}} = \frac{P_{kt}^L (1 - L_k)}{C_{kt}^L} - 1. \quad (3.5)$$

## 四、问题一：销量分布规律与关联关系

问题一先识别零销量和疑似售罄，去除共同季节性，再分析销量分布、条件关联和价格响应。具体计算步骤见图2。

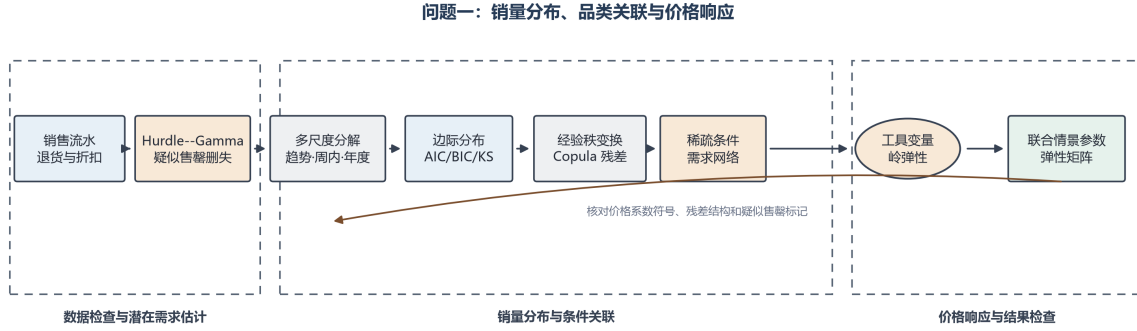


图 2 问题一建模流程

图2中，Hurdle-Gamma 与疑似售罄规则用于修正观测需求，多尺度分解和 Copula 网络估计品类间的剩余联动，工具变量岭回归估计价格响应。问题二生成联合需求情景时同时使用上述结果。

### 4.1 多时间尺度分解

先把六品类日销量映射到完整日历，再用趋势、星期虚拟变量、月份虚拟变量和三阶年度 Fourier 项解释可预测部分，同时控制活跃单品数与打折占比：

$$\log(1 + Y_{kt}) = \alpha_k + \beta_{k1}t + \beta_{k2}t^2 + W_{k,d(t)} \quad (4.1)$$

$$+ \sum_{h=1}^3 \left[ a_{kh} \sin \frac{2\pi ht}{365.25} + b_{kh} \cos \frac{2\pi ht}{365.25} \right] + \gamma_k^\top x_{kt} + e_{kt}. \quad (4.2)$$

残差  $e_{kt}$  用于关联网，避免把共同周末效应误认为直接互补。图3同时展示时间演化与边际分布；表3给出可复核统计量。

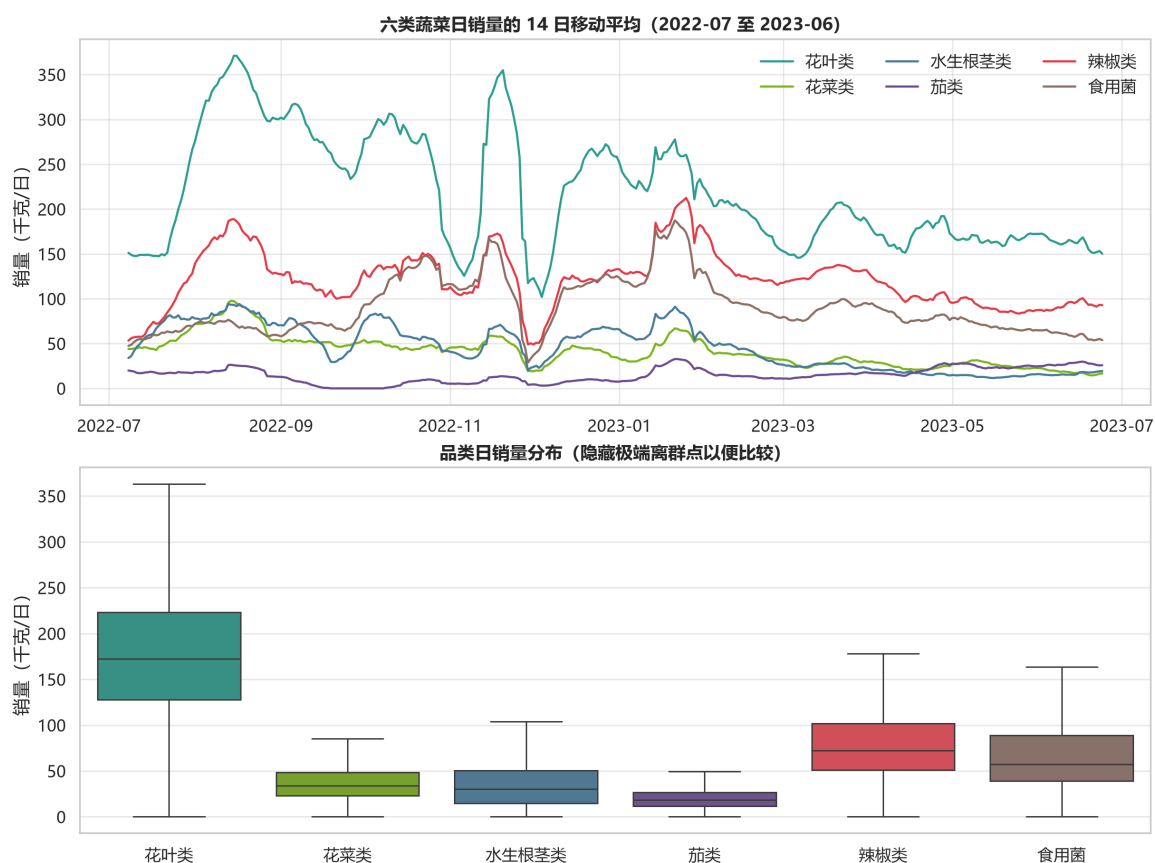


图 3 六品类日销量的时间演化与分布

表 3 六品类日销量分布统计

| 品类    | 均值/kg  | 中位数/kg | CV    | 偏度    | 活跃数   | 损耗率   |
|-------|--------|--------|-------|-------|-------|-------|
| 水生根茎类 | 37.08  | 30.05  | 0.847 | 2.485 | 2.93  | 13.7% |
| 花叶类   | 181.42 | 172.11 | 0.483 | 2.713 | 16.52 | 12.8% |
| 花菜类   | 38.16  | 33.86  | 0.600 | 1.473 | 1.71  | 15.5% |
| 茄类    | 20.50  | 18.27  | 0.662 | 1.556 | 2.93  | 6.7%  |
| 辣椒类   | 83.69  | 72.54  | 0.643 | 3.061 | 9.82  | 9.2%  |
| 食用菌   | 69.53  | 57.28  | 0.701 | 2.938 | 8.64  | 9.4%  |

花叶类规模最大，日均销量为 181.42 kg；茄类最小，日均为 20.50 kg。六类偏度为 1.47–3.06，均明显右偏，故正态需求假设不合适。图4显示各品类均呈“周中低、周末高”，周六相对指数约为 1.25–1.34，因此 7 月 1 日和 2 日不能直接沿用最近工作日均值。

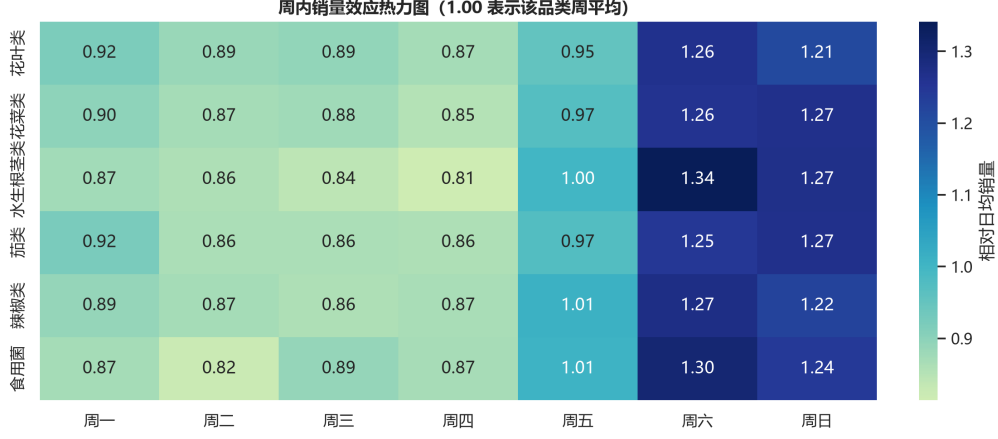


图 4 六品类周内销量相对指数

#### 4.2 Hurdle-Gamma 与疑似售罄右删失

单品日销量包含大量零值且正值部分右偏。对可售状态能够识别的单品日，先估计需求是否发生，再对正销量拟合 **Gamma** 分布：

$$\Pr(Y_{it} > 0 \mid A_{it} = 1) = \text{logit}^{-1}(\alpha_i + \beta_i^\top x_t), \quad (4.3)$$

$$Y_{it} \mid (Y_{it} > 0, A_{it} = 1) \sim \text{Gamma}(a_i, b_i). \quad (4.4)$$

题目没有提供真实库存和售罄时刻。本文将“最后一笔销售早于 18:00 且当日销量不低于该单品历史 75% 分位数”标记为疑似售罄，此时观察销量只是潜在需求下界。相应删失似然为

$$Y_{it}^{\text{obs}} = \min(D_{it}, I_{it}), \quad \ell_{it} = \begin{cases} \log f_D(Y_{it}^{\text{obs}}), & s_{it} = 0, \\ \log[1 - F_D(Y_{it}^{\text{obs}})], & s_{it} = 1. \end{cases} \quad (4.5)$$

该规则仅限定“需求不低于销量”，不估计缺少观测依据的库存轨迹。实际应用时应记录上架量、补货量与售罄时刻，并用库存截断似然替换该代理规则；这一处理符合删失需求文献给出的识别条件 [1]。

20 个核心单品若逐行列出参数会形成大篇幅长表，因此将需求发生概率、**Gamma** 正需求均值和疑似售罄日数整合为图5。颜色区分所属品类，圆点大小与标注共同反映疑似售罄日数。

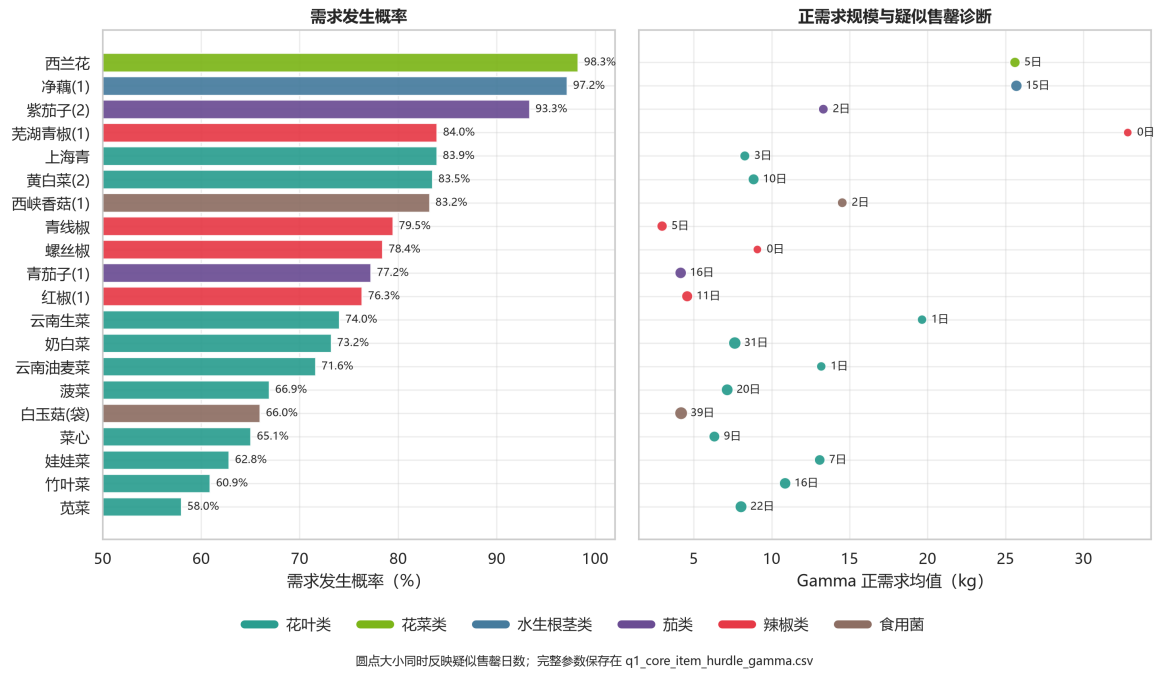


图 5 核心单品的 Hurdle-Gamma 参数与疑似售罄诊断

图5左侧表明西兰花、净藕 (1) 和紫茄子 (2) 的需求发生概率最高；右侧中白玉菇 (袋)、奶白菜和苋菜的疑似售罄标记较多，应优先补采库存轨迹。Gamma 均值只描述正需求规模，疑似售罄日数仅为基于 18:00 阈值的诊断量，不能视为题目给出的真实缺货次数；其余核心单品均按相同口径完成参数估计与诊断。

### 4.3 边际分布拟合

对品类正销量分别拟合 Gamma、对数正态、Weibull 与正态分布，以 AIC 作主排序，同时报告 BIC 与 KS 统计量。结果见表4：花叶、花菜、水生根茎和茄类由 Gamma 取得最低 AIC，辣椒与食用菌更接近对数正态。KS 统计量约为 0.024–0.030；考虑到大样本对轻微偏离非常敏感，后续情景不强行锁定单一参数分布，而以预测残差经验分布为主。

表 4 AIC 最优的品类正销量边际分布

| 品类    | 分布        | AIC       | BIC       | KS    | p 值   |
|-------|-----------|-----------|-----------|-------|-------|
| 水生根茎类 | Gamma     | 9,896.72  | 9,906.70  | 0.025 | 0.490 |
| 花叶类   | Gamma     | 12,442.59 | 12,452.57 | 0.030 | 0.271 |
| 花菜类   | Gamma     | 9,547.06  | 9,557.03  | 0.029 | 0.334 |
| 茄类    | Gamma     | 8,058.82  | 8,068.74  | 0.024 | 0.549 |
| 辣椒类   | Lognormal | 11,036.19 | 11,046.17 | 0.025 | 0.508 |
| 食用菌   | Lognormal | 10,840.05 | 10,850.03 | 0.025 | 0.524 |

### 4.4 非正态 Copula 条件需求网络

为同时保留非正态边际与品类间联合波动，对去季节残差作经验秩变换

$$Z_{kt} = \Phi^{-1} \left( \frac{\text{rank}(e_{kt}) - 0.5}{T} \right), \quad (4.6)$$



再用 Graphical Lasso 估计稀疏精度矩阵 [2, 3]:

$$\hat{\Theta} = \arg \min_{\Theta \succ 0} \{ \text{tr}(S\Theta) - \log |\Theta| + \lambda \|\Theta\|_{1,\text{off}} \}, \quad (4.7)$$

其中惩罚参数由交叉验证选择。控制其余品类后的条件相关为

$$\rho_{k\ell|\{-k,\ell\}} = -\frac{\Theta_{k\ell}}{\sqrt{\Theta_{kk}\Theta_{\ell\ell}}}. \quad (4.8)$$

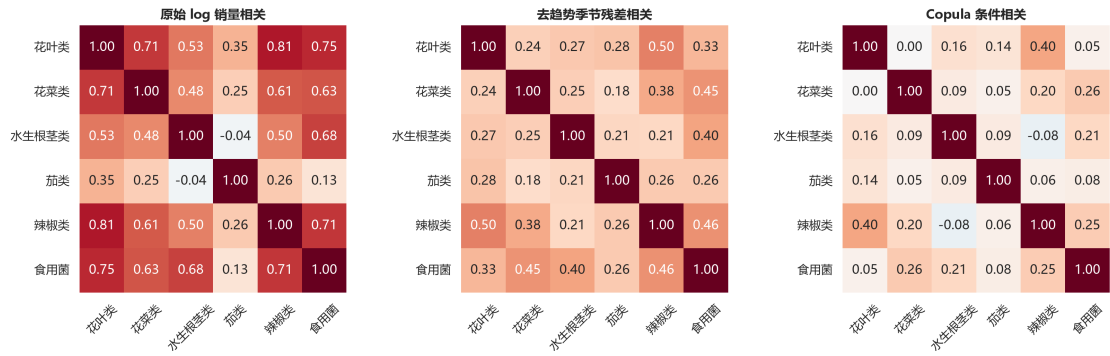


图 6 原始相关、去季节残差相关与 Copula 条件相关对比

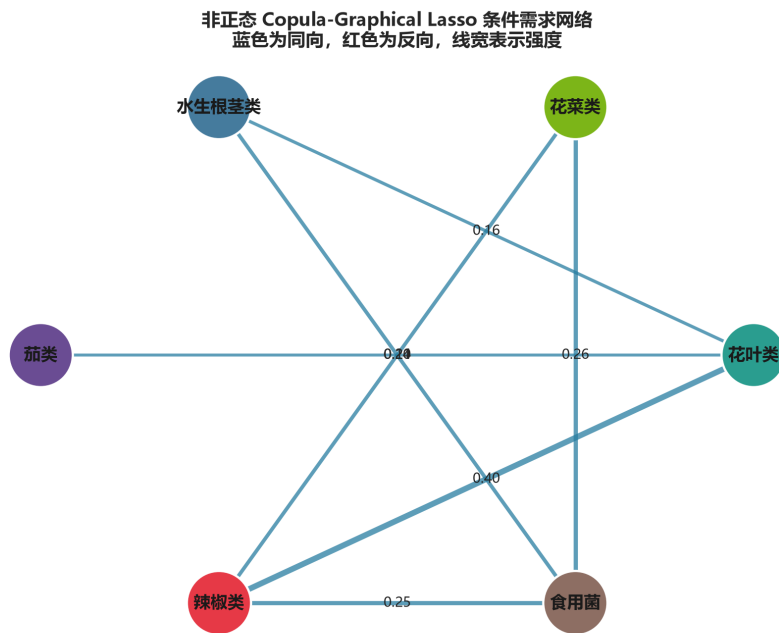


图 7 六品类 Copula 条件需求网络

表 5 Copula 条件需求网络中的主要边

| 品类 1  | 品类 2  | 条件相关  | 关系   |
|-------|-------|-------|------|
| 花叶类   | 辣椒类   | 0.397 | 同向联动 |
| 花菜类   | 食用菌   | 0.259 | 同向联动 |
| 辣椒类   | 食用菌   | 0.253 | 同向联动 |
| 水生根茎类 | 食用菌   | 0.207 | 同向联动 |
| 花菜类   | 辣椒类   | 0.204 | 同向联动 |
| 花叶类   | 水生根茎类 | 0.159 | 同向联动 |
| 花叶类   | 茄类    | 0.138 | 同向联动 |

花叶—辣椒的原始对数销量相关约为 0.81，剔除共同趋势与季节并控制其他品类后，条件相关降至 0.397。这说明大部分表面联动来自共同客流和日历冲击。表5中的七条边用于问题二的联合情景生成，但正条件相关只表示同向剩余波动，不能单凭符号断言商品互补。

#### 4.5 工具变量岭回归价格弹性

门店可能在需求高涨时主动提价，售价与需求存在内生性。以固定篮子批发成本作工具变量，第一阶段用六品类成本、日历、活跃单品数和打折占比预测售价；第二阶段以预测售价估计需求。为缓解价格共线性，对第二阶段加入岭惩罚：

$$\log P_{kt}^L = a_k + \sum_{\ell=1}^6 b_{k\ell} \log C_{\ell t}^L + \gamma_k^\top x_t + v_{kt}, \quad (4.9)$$

$$\log(1 + D_{kt}) = \alpha_k + \sum_{\ell=1}^6 \varepsilon_{k\ell} \widehat{\log P_{\ell t}^L} + \delta_k^\top x_t + u_{kt}. \quad (4.10)$$

表 6 工具变量第一阶段与自身价格弹性诊断

| 品类    | 第一阶段 $R^2$ | 岭参数    | 原始系数   | 决策弹性  |
|-------|------------|--------|--------|-------|
| 花叶类   | 0.886      | 0.728  | -0.432 | 0.432 |
| 花菜类   | 0.850      | 11.721 | -0.459 | 0.459 |
| 水生根茎类 | 0.929      | 2.212  | -0.039 | 0.200 |
| 茄类    | 0.872      | 1.269  | -0.259 | 0.259 |
| 辣椒类   | 0.955      | 0.418  | -0.497 | 0.497 |
| 食用菌   | 0.806      | 0.728  | -0.825 | 0.825 |

为检查交叉价格反应的方向和量级，表7再给出六品类弹性矩阵；它只承担数值核验，主要模式已由图13左侧热力图直观展示。

表 7 六品类价格弹性矩阵（行为需求品类，列为价格变化品类）

| 需求品类  | 花叶     | 花菜     | 水生根茎   | 茄      | 辣椒     | 食用菌    |
|-------|--------|--------|--------|--------|--------|--------|
| 花叶类   | -0.432 | -0.098 | 0.252  | 0.124  | -0.081 | 0.199  |
| 花菜类   | -0.236 | -0.459 | 0.154  | 0.308  | -0.064 | -0.586 |
| 水生根茎类 | 0.463  | -0.207 | -0.200 | -0.060 | 0.265  | -0.743 |
| 茄类    | 0.190  | 0.376  | 0.104  | -0.259 | -0.274 | 0.330  |
| 辣椒类   | -0.554 | -0.073 | 0.385  | 0.623  | -0.497 | 0.097  |
| 食用菌   | -0.079 | 0.149  | 0.255  | 0.507  | -0.122 | -0.825 |

注：对角线为自身价格系数；水生根茎类按单调需求约束截断为-0.200。

第一阶段  $R^2$  为 0.806–0.955，批发成本对售价具有较强解释力。六类原始自身价格系数均为负；水生根茎类绝对值仅 0.039，为防止决策出现近似无弹性的极端价格，按单调需求约束截断为  $-0.20$ 。最终决策用自身弹性范围为 0.200–0.825。交叉弹性只参与联合价格情景，结论仍以符号稳定性与压力测试为界。

## 五、问题二：品类补货与成本响应定价

问题二使用问题一得到的价格弹性和条件需求网络，并将需求、成本预测误差带入随机决策。计算顺序见图8。

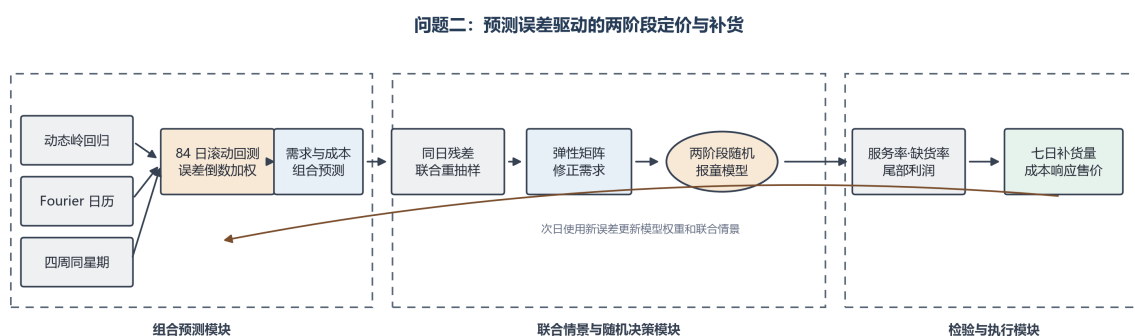


图 8 问题二建模流程

三个候选模型先通过 84 日滚动回测确定权重，同日需求和成本残差用于生成联合情景。每天先确定补货量，批发成本确定后再按加成率更新售价；当日服务率、缺货率和尾部利润用于次日参数更新。

### 5.1 需求与成本组合预测

对每个品类分别预测日销量与固定篮子批发成本。候选模型包括：含 1, 2, 7, 14, 21, 28 阶滞后、7 日及 28 日滚动均值和日历变量的动态岭回归；只含趋势、星期、月份及 Fourier 项的日历回归；最近四周同星期均值。设第  $j$  个模型在 84 日滚动回测中的 WAPE 为  $E_j$ ，组合权重取

$$w_j = \frac{(E_j + 10^{-6})^{-1}}{\sum_h (E_h + 10^{-6})^{-1}}, \quad \hat{D}_{kt} = \sum_j w_j \hat{D}_{kt}^{(j)}. \quad (5.1)$$

各模型的权重由近期预测误差确定，误差越小，权重越大。组合预测还能降低单一时间序列结构失效的风险 [4]。

表8列出六品类组合预测的 WAPE，用于比较各品类的预测误差。

表 8 六品类销量组合预测的 84 日回测精度

| 品类    | WAPE  | 组合权重和 |
|-------|-------|-------|
| 花叶类   | 18.6% | 1.0   |
| 花菜类   | 32.1% | 1.0   |
| 水生根茎类 | 26.1% | 1.0   |
| 茄类    | 25.2% | 1.0   |
| 辣椒类   | 19.1% | 1.0   |
| 食用菌   | 17.4% | 1.0   |

#### 5.1.1 候选预测模型的回测明细

销量与成本的候选模型共形成 42 条回测记录，直接排成长表会中断论证，因此图9把 WAPE 和误差倒数权重编码在同一热力矩阵中。颜色表示误差高低，单元格内同

时保留精确 WAPE 与权重。

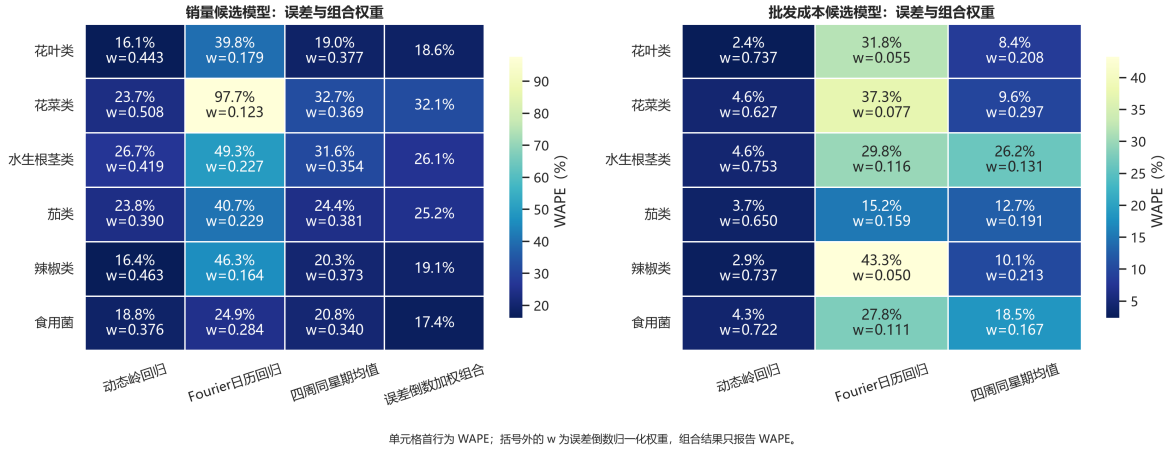


图 9 销量与批发成本候选模型的 84 日回测对比

花叶类和食用菌组合 WAPE 分别为 18.6% 和 17.4%，花菜类为 32.1%，说明高损耗、小规模品类仍有较大不确定性。图10给出未来一周点预测及不确定区间，周末峰值与问题一的周内效应相一致。

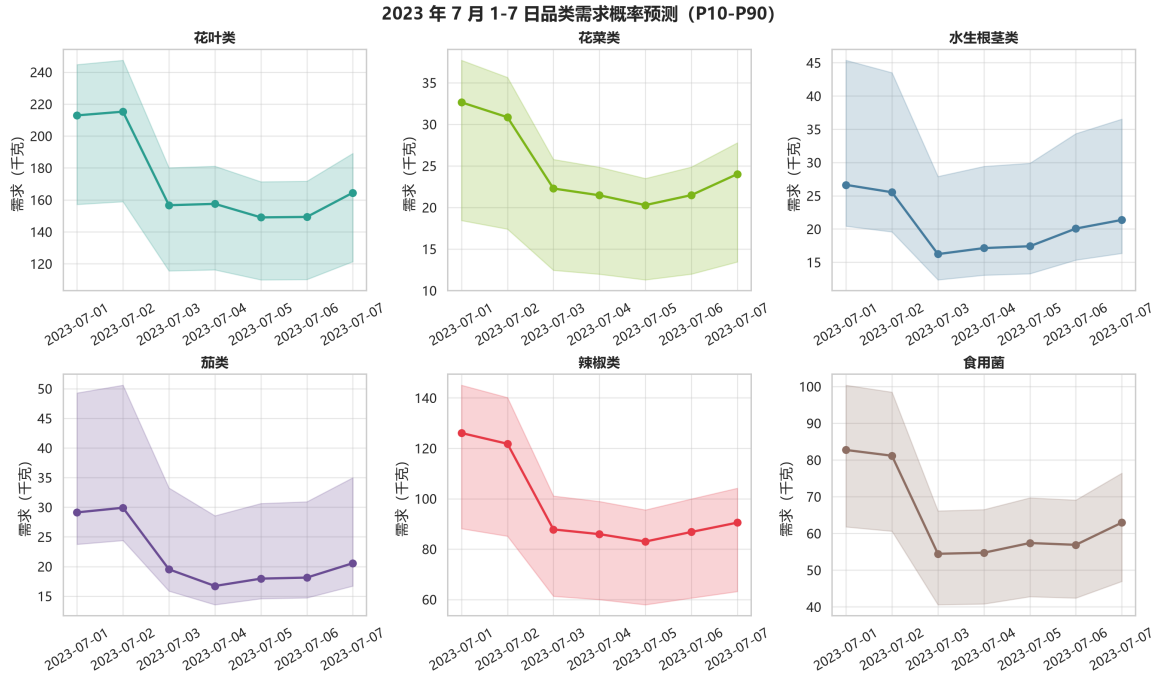


图 10 六品类需求与批发成本的组合预测

## 5.2 经验 Copula 联合情景

若分别独立抽取六个品类需求和成本，会破坏花叶—辣椒等条件联动。本文在相同历史日期上保存 12 维预测残差向量

$$\mathbf{r}_t = (r_{1t}^D, \dots, r_{6t}^D, r_{1t}^C, \dots, r_{6t}^C), \quad (5.2)$$

每个未来日从历史日期索引中联合重抽 1 600 次，并叠加到点预测上：

$$D_{kt}^{(\omega)} = \exp \left[ \log(1 + \hat{D}_{kt}) + r_{kt}^{D,(\omega)} \right] - 1, \quad C_{kt}^{(\omega)} = \hat{C}_{kt} \exp(r_{kt}^{C,(\omega)}). \quad (5.3)$$

这一经验 Copula 方式在不假设多元正态的同时，保留同日需求—成本及跨品类误差结构。

### 5.3 两阶段成本响应型定价

经营时序为“先订货、后看到实际成本、再挂牌”。因此第一阶段决定补货量  $q_{kt}$  和有效加成率  $m_{kt}$ ；成本情景实现后，售价按

$$P_{kt}^{(\omega)} = \frac{C_{kt}^{(\omega)}}{1 - L_k} (1 + m_{kt}) \quad (5.4)$$

自动更新。若改为预先固定绝对售价，成本上涨会侵蚀利润，且不符合题目所述成本加成定价。加成率候选区间取近一年历史有效加成率的 5%–95% 分位并限制于合理边界。

在参考价格  $P_{ellt}^0$  附近，价格情景通过弹性矩阵调整需求：

$$\tilde{D}_{kt}^{(\omega)} = D_{kt}^{(\omega)} \exp \left\{ \sum_{\ell=1}^6 \varepsilon_{k\ell} \log \frac{P_{\ell t}^{(\omega)}}{P_{ellt}^0} \right\}. \quad (5.5)$$

价格比限制在  $[0.35, 3.00]$ ，需求上界限制为点预测的 8 倍，以控制样本区间外的预测幅度。压力测试进一步检验这些边界对方案稳定性的影响。

### 5.4 损耗型随机报童模型

情景销量和利润分别为

$$S_{kt}^{(\omega)} = \min \{ \tilde{D}_{kt}^{(\omega)}, (1 - L_k) q_{kt} \}, \quad (5.6)$$

$$\Pi_t^{(\omega)} = \sum_{k=1}^6 \left( P_{kt}^{(\omega)} S_{kt}^{(\omega)} - C_{kt}^{(\omega)} q_{kt} \right). \quad (5.7)$$

易腐库存与价格型报童研究表明损耗、售价和订货量必须联合考虑 [5, 6]；本文进一步按随机规划思想 [7] 将成本实现后的价格规则纳入两阶段情景。对每个品类的加成率搜索 17 个网格，对调整后需求搜索 12 个分位点作为订货候选，并进行 4 轮坐标下降，目标为最大化样本平均利润

$$\max_{q, m} \frac{1}{1600} \sum_{\omega=1}^{1600} \Pi_t^{(\omega)}. \quad (5.8)$$

模型同时输出需求满足率、缺货概率和最差 10% 情景平均利润，用于识别均值较高但尾部风险过大的方案。

### 5.5 未来一周方案与解释

先用图11展示一周总量和品类构成，再用短表汇总每日总补货与利润，避免读者在进入本节时突然面对密集数字。

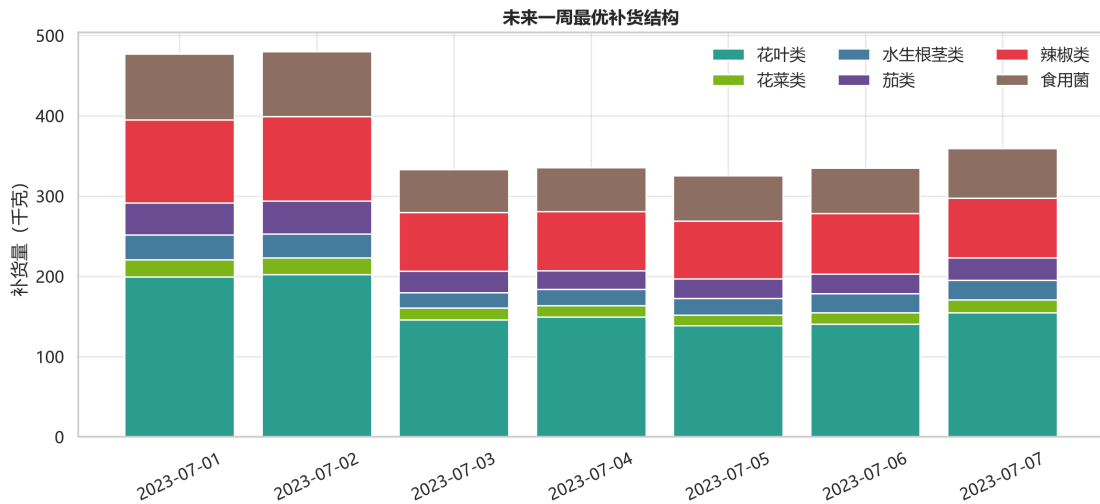


图 11 2023 年 7 月 1-7 日各品类补货结构

表9只保留七天汇总量，用于核验周末峰值与一周期望利润。

表 9 2023 年 7 月 1-7 日品类方案按日汇总

| 日期    | 总补货/kg | 期望销量/kg | 期望利润/元   |
|-------|--------|---------|----------|
| 07-01 | 477.3  | 390.5   | 1,887.56 |
| 07-02 | 480.0  | 391.4   | 1,896.61 |
| 07-03 | 333.0  | 273.1   | 1,337.49 |
| 07-04 | 335.3  | 273.7   | 1,339.90 |
| 07-05 | 325.2  | 266.0   | 1,294.40 |
| 07-06 | 334.9  | 273.1   | 1,316.12 |
| 07-07 | 359.3  | 293.8   | 1,419.51 |

7 月 1 日和 2 日为周末，总补货分别为 477.3 kg 和 480.0 kg，明显高于 7 月 3-6 日的约 325-335 kg；一周累计补货 2,645.0 kg，情景期望利润 10,491.61 元。各日均以花叶类补货量最大；花菜类损耗率最高且成本较高，订货更保守。

表 10 2023 年 7 月 1 日各品类定价补货方案

| 品类    | 成本 P50 | 售价   | 补货/kg | 期望销量  | 满足率   | 期望利润   |
|-------|--------|------|-------|-------|-------|--------|
| 花叶类   | 3.80   | 9.8  | 199.4 | 160.8 | 91.1% | 835.77 |
| 花菜类   | 7.95   | 15.9 | 21.5  | 16.4  | 83.9% | 95.52  |
| 水生根茎类 | 9.02   | 15.6 | 31.0  | 24.5  | 83.4% | 105.02 |
| 茄类    | 3.88   | 8.5  | 39.7  | 33.4  | 85.8% | 131.48 |
| 辣椒类   | 4.32   | 9.9  | 103.6 | 87.7  | 88.7% | 426.88 |
| 食用菌   | 4.46   | 9.9  | 82.1  | 67.7  | 90.3% | 292.90 |

图12分别展示七天六品类的补货量、参考售价、需求满足率和情景期望利润。

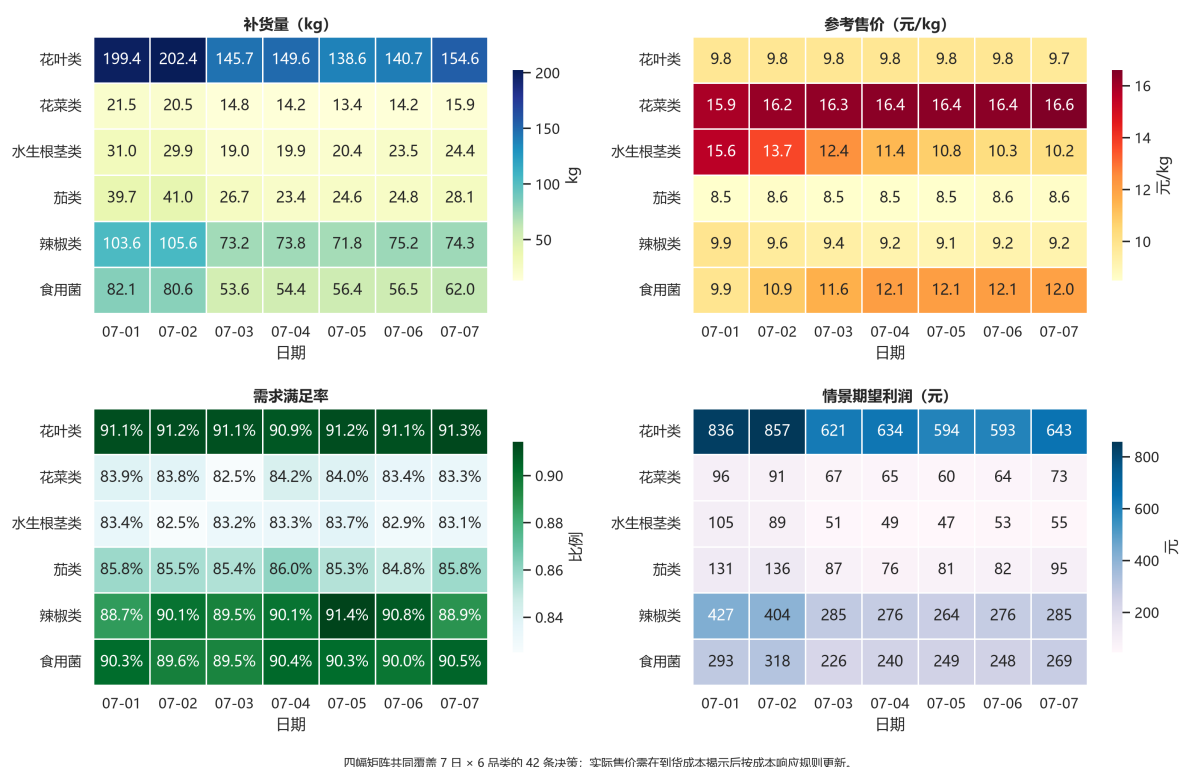


图 12 七天六品类的补货、定价、满足率与利润矩阵

图12覆盖全部 42 条决策，既能横向比较日期，也能纵向比较品类。花叶类和辣椒类承担主要补货量与利润，水生根茎类参考售价随预测成本下降而逐日回落；执行时应根据每日实际批发成本更新售价，并同步复核需求满足率与尾部利润。

表10中的售价由预测中位成本计算，仅用于展示，不作为七天的固定执行价。每日执行时，应把实际成本代入式(5.4)。7月1日花叶类补货 199.4 kg、参考售价 9.8 元/kg，情景期望利润约 835.77 元；花菜类和水生根茎类在左尾情景下可能亏损，需要结合表11监控风险。

表 11 未来一周每日利润分布与尾部风险

| 日期    | 期望利润     | P10      | P90      | 最差 10% 均值 |
|-------|----------|----------|----------|-----------|
| 07-01 | 1,887.56 | 1,489.09 | 2,261.92 | 1,091.47  |
| 07-02 | 1,896.61 | 1,526.04 | 2,283.18 | 1,047.57  |
| 07-03 | 1,337.49 | 1,089.84 | 1,610.04 | 819.89    |
| 07-04 | 1,339.90 | 1,071.18 | 1,618.24 | 809.22    |
| 07-05 | 1,294.40 | 1,048.78 | 1,570.32 | 760.22    |
| 07-06 | 1,316.12 | 1,066.41 | 1,590.62 | 797.76    |
| 07-07 | 1,419.51 | 1,113.98 | 1,724.56 | 803.47    |

## 5.6 滚动回测

最近 84 日依次作为待预测日。基准策略使用最近 7 日平均需求补货和历史中位加成率；本文策略使用组合预测、不确定性缓冲与优化加成率。实际销量依据自身价格弹性换算为各策略价格下的反事实需求。回测利润用于比较两种策略，不作为门店真实会计利润。

表12以六个品类为单位给出本文策略与基准策略的可比利润，随后用图形强调差异。

表 12 84 日滚动回测的品类结果

| 品类    | WAPE  | 本文利润      | 基准利润      | 提升     | 本文服务  | 基准服务  |
|-------|-------|-----------|-----------|--------|-------|-------|
| 花叶类   | 18.6% | 36,706.19 | 16,452.90 | 123.1% | 79.9% | 87.1% |
| 花菜类   | 32.1% | 1,042.77  | 1,597.76  | -34.7% | 87.0% | 86.0% |
| 水生根茎类 | 26.1% | 2,385.87  | 407.23    | 485.9% | 86.5% | 83.3% |
| 茄类    | 25.2% | 7,308.04  | 2,290.20  | 219.1% | 82.6% | 84.5% |
| 辣椒类   | 19.1% | 18,704.85 | 8,278.08  | 126.0% | 82.5% | 87.3% |
| 食用菌   | 17.4% | 2,541.63  | 1,358.93  | 87.0%  | 62.8% | 76.0% |

注：利润为同一弹性假设下的反事实模拟利润，不等同于门店会计利润。

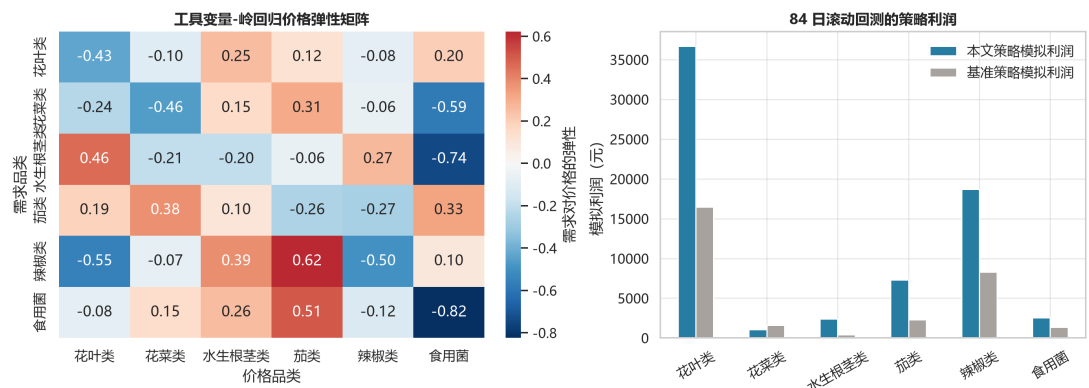


图 13 价格弹性矩阵与 84 日滚动回测利润

六品类合计模拟利润从 30,385.10 元提高至 68,689.34 元，模型内提升 126.1%。其中花菜类模拟利润下降 34.7%，表明高波动和高损耗会降低该策略的收益。该策略对不同品类的效果存在差异。

六、问题三：单品组合、定价与补货

问题三在问题二给出的品类总量边界下建立单品选择需求模型，并采用字典序方法求解：先提高最低品类满足率，再在该服务水平下最大化利润。具体计算步骤见图14。

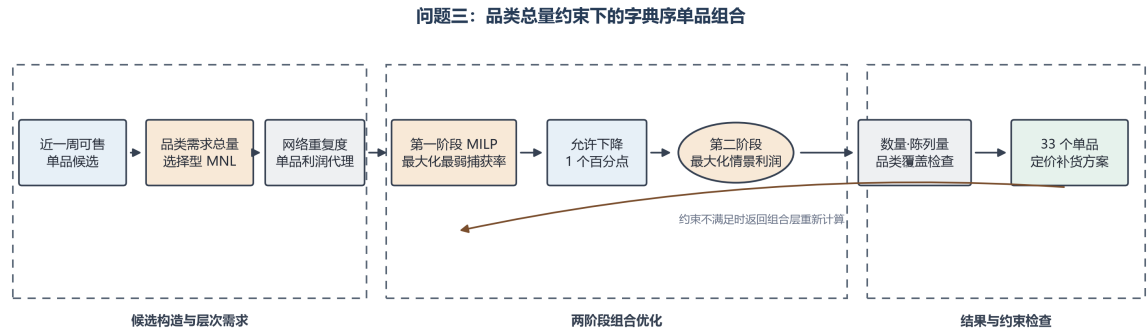


图 14 问题三建模流程

图14将候选生成、MNL 层次需求、网络重复度、两阶段 MILP 和约束检查分为三个模块。数量、最低陈列或弱品类覆盖检查未通过时，求解器返回组合层重新计算，禁止在输出端手工删改单品。



## 6.1 候选集合与层次一致性

候选集合限定为 2023 年 6 月 24–30 日至少出现一笔正向销售的单品：

$$\mathcal{C} = \{i : Y_{it}^+ > 0 \text{ for some } t \in [\text{Jun.24}, \text{Jun.30}]\}. \quad (6.1)$$

程序得到 49 个候选。单品需求首先受问题二的 7 月 1 日品类需求总量约束，再在品类内部通过选择型份额分配，避免 49 个单品独立预测之和与六品类预测冲突。

由于附件没有购物篮编号，无法可靠估计完整 Nested Logit。本文采用可识别的品类内 MNL 近似 [8–10]：以近 28 日销量和经验贝叶斯先验构成基础效用

$$u_i = \frac{Q_{i,28} + \tau_k}{\sum_{j \in \mathcal{C}_k} (Q_{j,28} + \tau_k)}, \quad V_i = \log u_i - \eta_k \log \frac{P_i}{P_{\text{pref}}}, \quad (6.2)$$

并引入外部选项质量  $U_{0k}$  表示未购买或转向其他渠道。给定选择集合  $z$  和价格  $P$ ，需求份额为

$$s_i(z, P) = \frac{z_i \exp(V_i)}{U_{0k} + \sum_{j \in \mathcal{C}_k} z_j \exp(V_j)}, \quad D_i(z, P) = D_k^0 s_i(z, P). \quad (6.3)$$

外部选项份额由活跃单品数对品类销量的岭回归系数校准，并限制在 6%–30%。删除单品后，需求通过外部流失和同类转移重新分配，未采用简单平均分摊。

## 6.2 网络多样性与单品利润代理

对最近 180 日的单品对数销量建立同品类相关网络。单品  $i$  与同品类其他商品前三大正相关的均值记为网络重复度  $r_i$ ；若多个候选都高度重复，其同时入选会降低品种多样性。第一阶段仍以品类需求捕获为主，第二阶段利润代理加入 8%  $r_i$  折减，鼓励在相近利润下保留不同需求角色。

入选后，对每个单品用 90 日销售记录估计 Hurdle–Lognormal 情景。需求发生概率由有销量日比例加经验修正得到，正需求对数标准差由历史变异系数换算；每个单品抽取 2200 个需求一成本情景，在 15 个加成率网格和 11 个需求分位订货点上搜索，且始终满足最低陈列量。

## 6.3 字典序选择模型

设  $z_i$  表示单品是否上架， $q_i$  为补货量。题目约束为

$$27 \leq \sum_{i \in \mathcal{C}} z_i \leq 33, \quad q_i \geq 2.5 z_i, \quad \sum_{i \in \mathcal{C}_k} z_i \geq 1. \quad (6.4)$$

品类  $k$  在基础效用层面的需求捕获率定义为

$$\rho_k(z) = \frac{\sum_{i \in \mathcal{C}_k} z_i u_i}{U_{0k} + \sum_{i \in \mathcal{C}_k} z_i u_i}, \quad \rho(z) = \min_k \rho_k(z). \quad (6.5)$$

对给定目标  $\rho$ ，约束可线性化为

$$\sum_{i \in \mathcal{C}_k} z_i u_i \geq \frac{\rho}{1 - \rho} U_{0k}, \quad k = 1, \dots, 6. \quad (6.6)$$

因此第一阶段用二分搜索与混合整数线性规划求

$$\rho^* = \max_z \min_k \rho_k(z). \quad (6.7)$$

第二阶段允许最差捕获率比  $\rho^*$  降低 1 个百分点，在此前提下最大化情景期望利润并惩罚网络重复度：

$$\max_{z,q,m} \mathbb{E}[\Pi(z, q, m)] - \kappa \sum_i r_i z_i, \quad \text{s.t. } \rho_k(z) \geq \rho^* - 0.01. \quad (6.8)$$

这种字典序结构先兑现“尽量满足各品类需求”，再在可接受服务集合中追求利润，比把服务率和利润用任意权重相加更具可解释性。对 27–33 种的七个固定数量模型和两阶段模型求解，均得到满足数量、品类覆盖和最低陈列约束的可行解。

#### 6.4 单品数敏感性与最终组合

表13仅保留 27–33 种七个候选规模的服务率结果，随后用折线图展示边际收益变化。

表 13 固定单品数量下的最大最低品类捕获率

| 单品数 | 最差品类捕获率 | 相对 33 种损失 |
|-----|---------|-----------|
| 27  | 62.91%  | 2.90%     |
| 28  | 64.39%  | 1.42%     |
| 29  | 64.57%  | 1.24%     |
| 30  | 64.69%  | 1.11%     |
| 31  | 65.53%  | 0.27%     |
| 32  | 65.67%  | 0.14%     |
| 33  | 65.81%  | 0.00%     |

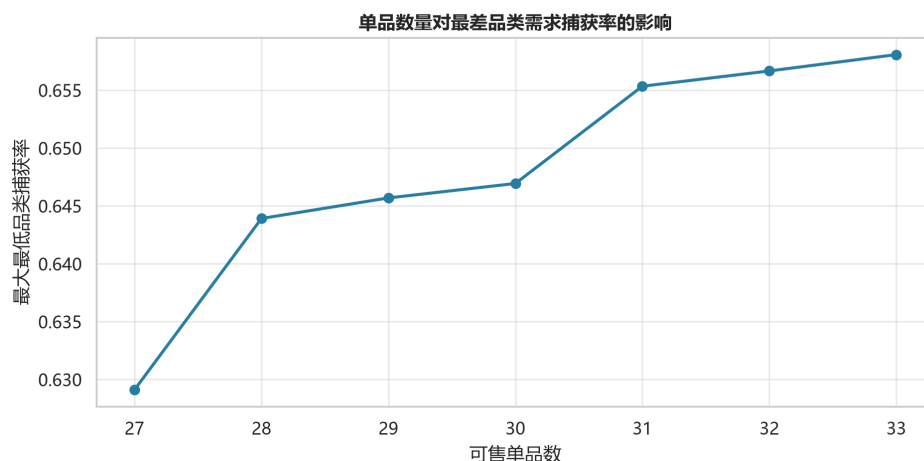


图 15 单品数量对最差品类需求捕获率的影响

候选数从 27 增至 28 时最差品类捕获率明显提高，31 种以后边际收益减弱；允许 27–33 种时第一阶段在 33 种达到 65.81%，第二阶段采用 64.81% 的最低捕获率约束。因此最终选择 33 个单品。

表14只汇总六品类的单品数、满足率、补货量和利润，用于检验组合层面的约束表现。

表 14 单品组合方案的品类汇总

| 品类    | 单品数 | 基准需求   | 预计销量   | 满足率   | 补货量   | 期望利润   |
|-------|-----|--------|--------|-------|-------|--------|
| 花叶类   | 11  | 212.89 | 102.52 | 48.2% | 138.2 | 77.52  |
| 花菜类   | 2   | 32.64  | 18.13  | 55.6% | 23.1  | 73.41  |
| 水生根茎类 | 5   | 26.64  | 11.91  | 44.7% | 19.6  | -36.03 |
| 茄类    | 3   | 29.12  | 12.70  | 43.6% | 18.2  | 20.23  |
| 辣椒类   | 7   | 126.06 | 65.10  | 51.6% | 84.0  | 60.17  |
| 食用菌   | 5   | 82.72  | 40.72  | 49.2% | 48.8  | 53.13  |

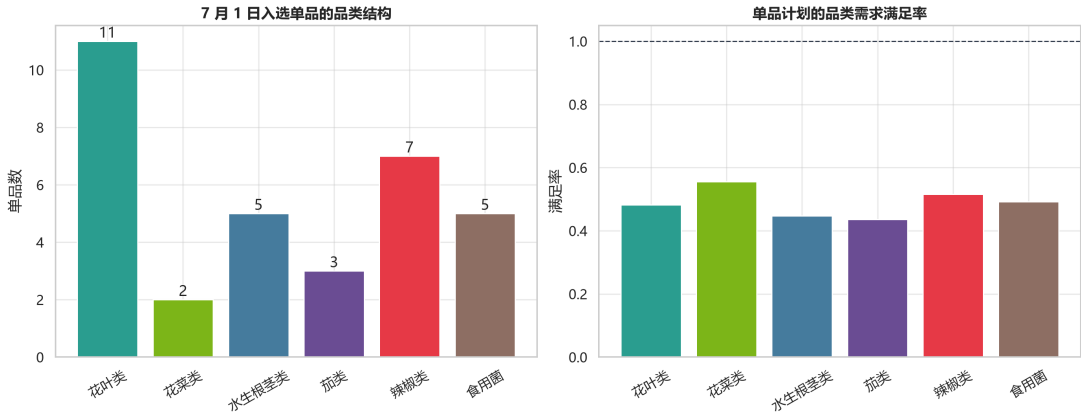


图 16 入选单品结构与品类预计销量满足率

最终组合包含花叶类 11 种、花菜类 2 种、水生根茎类 5 种、茄类 3 种、辣椒类 7 种和食用菌 5 种；总补货 331.9 kg，情景期望利润 248.43 元。预计销量满足率为 43.6%–55.6%，低于基础效用层面的捕获率，是因为成本、价格反应、最低陈列量与随机缺货共同限制了可实现销量。水生根茎类期望利润为 -36.03 元，反映了“先保障最弱品类、再最大化利润”的真实代价；若经营者只追求会计利润，应明确放松服务约束，而不能删去这一负结果。

33 个单品若逐项排成表格需要跨页，图17改用六个品类面板展示执行方案：横条长度表示补货量，条末标注建议售价和期望利润，负利润单品使用暖色提示。

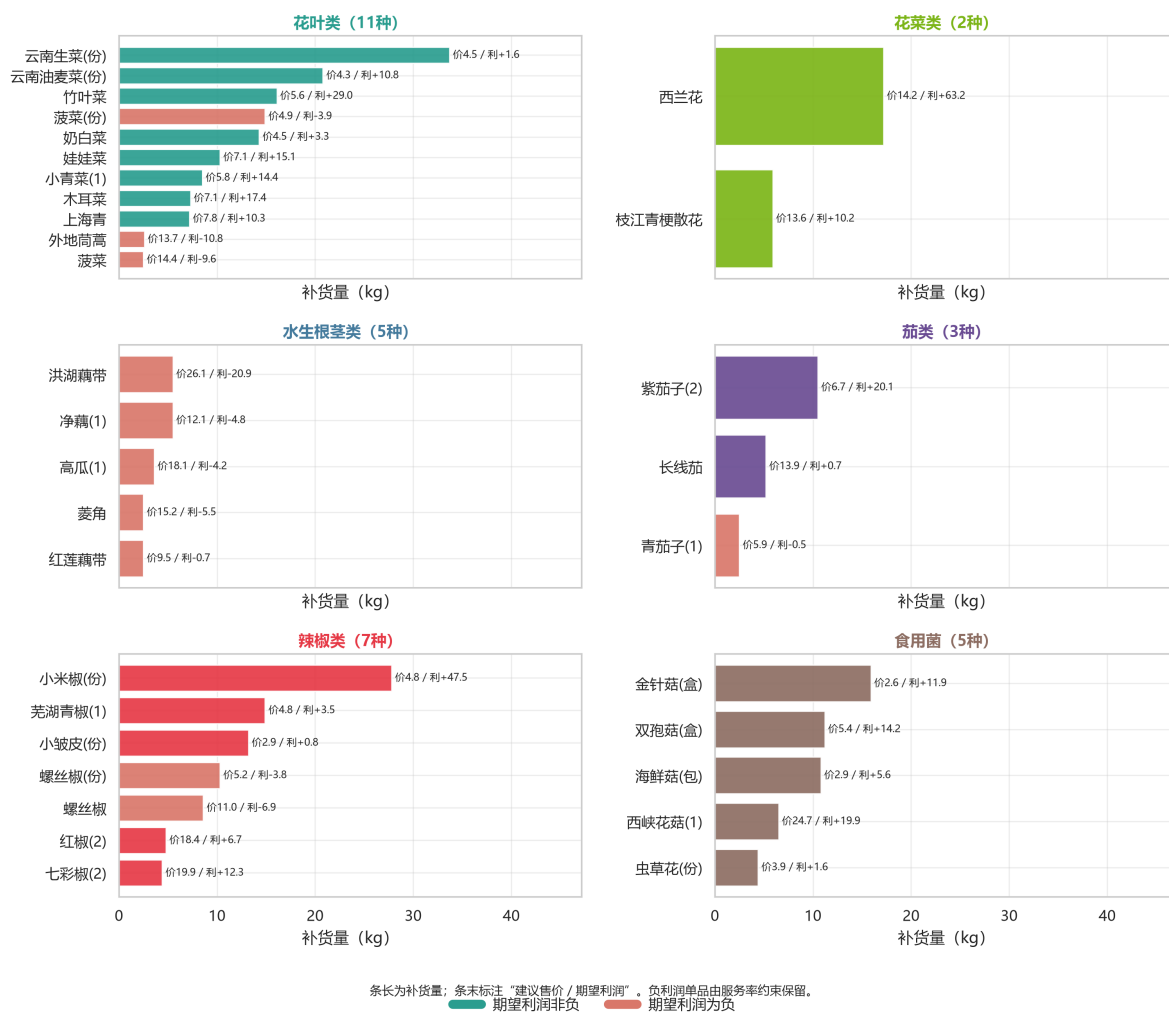


图 17 2023 年 7 月 1 日 33 个入选单品的图形化定价补货方案

图17给出可以直接执行的 33 个单品方案；价格按 0.1 元取整，补货量按 0.1 kg 取整。图中水生根茎类的五个单品均出现负利润，直观呈现服务率约束的代价。若供应商临时缺货，应在同品类备选中优先选择基础效用高、网络重复度低且不破坏最低捕获率的单品，而不能跨品类随意替换。

## 七、问题四：新增数据及其信息价值

问题四根据前三问的数据缺口确定候选采集项，再比较信息价值和采集成本，并说明各字段用于更新哪些参数、约束或验证指标。具体步骤见图18。

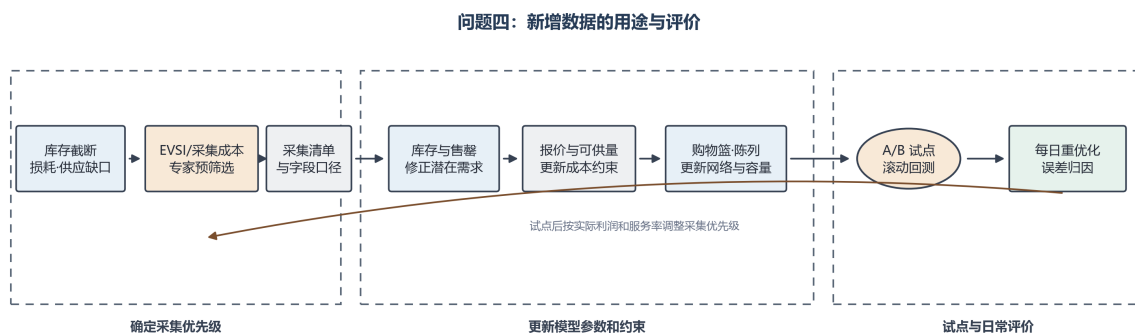


图 18 问题四数据评价流程

图18中，库存与售罄数据用于修正潜在需求，供应报价用于更新成本约束，购物篮与陈列数据用于更新关联网络和容量。完成 A/B 试点和滚动回测后，根据实际利润与服务率的变化重新计算采集优先级。

## 7.1 评价原则

新增数据的优先级由预期样本信息价值和采集成本共同确定。设  $X$  为拟采集信息， $a$  为补货定价动作， $\xi$  为需求和成本不确定性，预期样本信息价值定义为

$$EVSI(X) = \mathbb{E}_X \left[ \max_a \mathbb{E}(\Pi(a, \xi) | X) \right] - \max_a \mathbb{E}[\Pi(a, \xi)]. \quad (7.1)$$

若采集成本为  $K_X$ ，则优先比较  $EVSI(X)/K_X$ 。附件无法直接提供未来数据的真实 EVSI，故表15先以 1–5 分专家评分形成预筛选；部署后应通过 A/B 试点和滚动重优化把评分替换为实际利润增益。

## 7.2 建议采集清单

表 15 建议新增数据及其信息价值优先级

| 拟采集数据                  | 决策作用                           | 对应问题   | 优先级 | 价值/成本 |
|------------------------|--------------------------------|--------|-----|-------|
| 实时库存、订货量、到货量、售罄时刻      | 区分零需求、未上架与缺货；将销量右删失修正为潜在需求     | 问题 1-3 | 最高  | 2.50  |
| 实际报损量、报损原因、打折时刻与折后销量   | 动态估计损耗率与残值，优化日内折价和订购量          | 问题 2-3 | 最高  | 1.67  |
| 温度、降雨、湿度、节假日和社区活动      | 改善短期需求概率预测并解释异常波动              | 问题 1-2 | 中   | 1.50  |
| 货架面积、陈列位置、面数及补架记录      | 识别陈列效应并补充空间容量约束                | 问题 1、3 | 高   | 1.33  |
| 匿名订单号或购物篮编号            | 可靠识别共同购买、互补和替代，校准 Nested Logit | 问题 1、3 | 高   | 1.33  |
| 促销、优惠券、满减与广告曝光         | 解除价格与促销混淆，提高价格弹性因果解释           | 问题 1-2 | 高   | 1.33  |
| 凌晨供应商报价、可供数量、等级、产地、履约率 | 刻画成本与供应联合不确定性，校准两阶段定价          | 问题 2-3 | 最高  | 1.25  |
| 冷链温度、在架时长与质量图像         | 建立品质衰减曲线和动态折价模型                | 问题 2-3 | 中高  | 1.00  |
| 客流量、会员结构与客群画像          | 将销量变化拆分为客流和购买率变化               | 问题 1-2 | 中高  | 1.00  |
| 竞争门店价格与促销              | 估计渠道替代和竞争价格效应                  | 问题 1-2 | 中高  | 1.00  |

实时库存、订货量、到货量和售罄时刻位居首位，因为它们可以区分零需求、未上架和库存截断，直接修正问题一至三的需求下界。实际报损量、报损原因、折价时刻与折后销量可将固定损耗率升级为随温度和在架时长变化的品质衰减模型。供应商报价、可供数量、等级、产地和履约率则可把经验成本残差升级为供给侧结构模型。

购物篮编号应优先于“时间戳相近”的交易拼接：只有同一订单内的商品共现才支持真实互补关系分析。货架面积与陈列面数能补充问题三缺失的空间容量；气象、节假日和社区活动则有助于解释预测异常。数据采集应遵循匿名化、最少必要与用途限定原则，顾客画像只保留完成需求分层所需的聚合字段。

7.3 每日数据更新

每日按以下顺序更新模型：闭店后用实际销量、剩余库存和报损更新潜在需求；凌晨获取供应报价和可供量后更新成本情景并求解补货量与加成率；营业中记录补架和售罄；次日根据预测误差、服务率和利润评估前一日策略。每项新增数据均对应具体的参数、约束或验证指标。

图19按时间顺序列出一天内的数据输入、定价补货动作和次日更新。

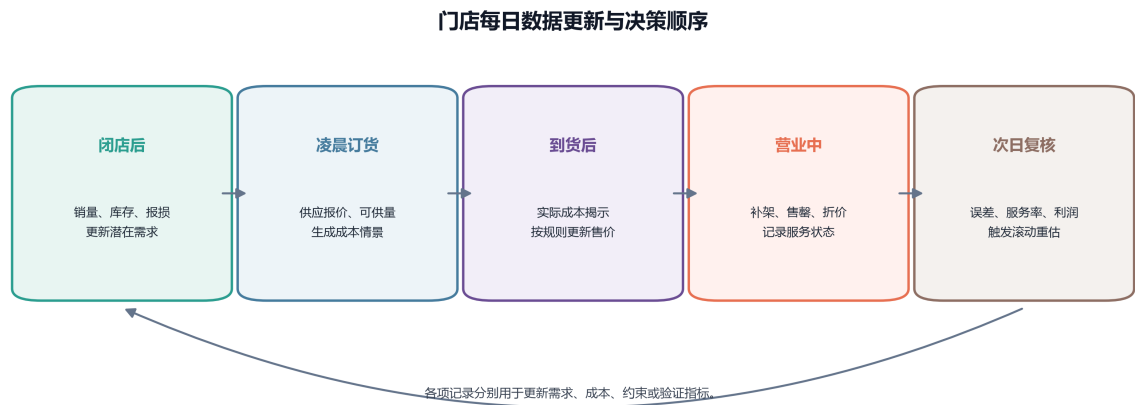


图 19 蔬菜定价补货模型的每日滚动执行流程

补货量在凌晨确定，实际挂牌价在批发成本确定后更新。营业中记录的补架和售罄信息仅用于次日需求校准。当日预测误差、执行偏差和供应冲击分别记录。

八、稳健性、模型评价与局限

8.1 参数压力测试

保持已选补货量不变，分别对需求、成本、损耗率和加成率施加单因素扰动，并重新计算七日联合情景利润与满足率，结果见表16。

表 16 问题二策略的单因素压力测试

| 情景       | 期望周利润     | 相对变化   | 满足率   | 最差 10% 利润 |
|----------|-----------|--------|-------|-----------|
| 基准       | 10,491.61 | 0.0%   | 89.3% | 8,948.96  |
| 需求-10%   | 9,175.91  | -12.5% | 93.5% | 7,446.83  |
| 需求 +10%  | 11,336.21 | 8.1%   | 84.2% | 10,021.30 |
| 成本-10%   | 9,525.10  | -9.2%  | 88.9% | 8,154.50  |
| 成本 +10%  | 11,425.26 | 8.9%   | 89.6% | 9,718.73  |
| 损耗率-20%  | 10,252.41 | -2.3%  | 90.3% | 8,667.26  |
| 损耗率 +20% | 10,713.11 | 2.1%   | 88.3% | 9,220.77  |
| 加成率-10%  | 9,439.91  | -10.0% | 88.9% | 7,994.11  |
| 加成率 +10% | 11,530.30 | 9.9%   | 89.7% | 9,893.81  |

需求下降 10% 时周利润下降 12.5%，说明需求预测仍是主要风险源；需求上升 10% 时利润仅增加 8.1% 且满足率下降，反映固定订货量下的缺货上限。成本上升情景中的名义利润增加以价格充分传导和弹性估计有效为前提，不表示成本上涨能够改善真实经营收益。自动调价吸收了部分损耗变化，但服务率仍随损耗率变化。

8.2 约束与求解复核

问题三的约束复核结果表明：候选集合只来自指定 7 日窗口；入选数为 33，位于 27–33 之间；六个品类均至少保留一种；所有入选单品补货量不低于 2.5 kg；第一阶段

最低捕获率为 65.81%，第二阶段目标为 64.81%。七个候选规模及两阶段模型均获得满足上述条件的可行解，最终组合与约束复核结果一致。

问题二的日补货量、参考售价、情景利润与回测结果在相同数据和参数下保持一致。模型有效性由滚动回测、压力测试与约束检查共同支持，不能以论文编译或排版效果替代模型验证。

### 8.3 模型优点

- 从销售流水到最终方案保持统一口径，退货、打折、损耗与销量加权售价均实际进入计算；
- 区分观察销量与潜在需求，以 Hurdle 和右删失边界降低“销量即需求”的偏差；
- Copula 条件网络负责联合风险，价格弹性负责价格响应，两者模型角色清晰；
- 先补货、成本实现后再定价，符合凌晨采购的真实经营时序；
- 问题三采用字典序求解，先满足服务约束，再提高利润；结果显示，服务约束可能要求保留短期亏损单品；
- 结果表、图和摘要中的关键数值口径一致，并经过回测与约束复核。

### 8.4 局限与改进

- (1) 缺少真实库存与售罄时刻，右删失标记仍是代理；获得库存轨迹后应改用库存约束似然；
- (2) 缺少订单号，单品组合只能使用聚合份额和相关网络，无法识别同一购物篮内的真实互补；
- (3) 批发成本工具变量可能与供给冲击下的需求共同变化，因果弹性需结合促销控制或随机价格实验；
- (4) 附件 4 为近期静态损耗率，未反映温度、在架时长与品质等级，应升级为动态品质衰减模型；
- (5) 回测利润采用价格弹性进行反事实转换，与门店实际会计利润的口径不同；上线前应开展小规模在线试验；
- (6) 问题二采用坐标网格搜索。该方法易于解释和复现，但可能遗漏更优的联合价格点。

### 8.5 验证结果与部署条件

图20将模型证据分为四个环节。当前附件和计算结果已支持输入审计、时序回测和优化可行性检查；部署效果仍需门店实时监测验证。

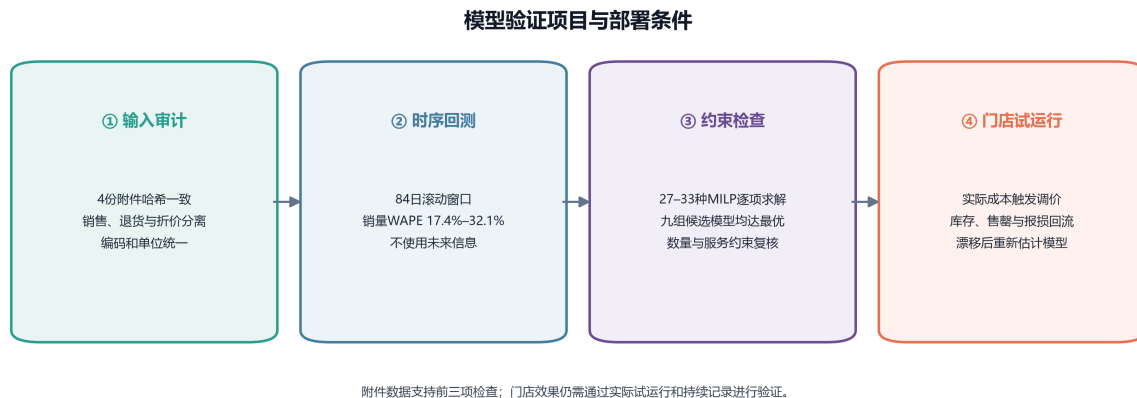


图 20 模型验证项目与部署条件

受实时库存、售罄时刻、实际报损及购物篮数据缺失的限制，本文结果仅适用于附件数据与既定假设。模型内回测利润不代表真实经营增益。该方案可作为下一轮小规模试运行的起点；试运行时应同步保存策略版本、实际售价、库存、售罄、报损和对照组结果，并据此更新需求、价格与补货参数。滚动评价持续满足服务率与风险阈值后，方可正式部署。



## 参考文献

- [1] MERSEREAU A J. Demand estimation from censored observations with inventory record inaccuracy[J/OL]. *Manufacturing & Service Operations Management*, 2015, 17(3): 335-349. DOI: 10.1287/msom.2015.0520.
- [2] FRIEDMAN J, HASTIE T, TIBSHIRANI R. Sparse inverse covariance estimation with the graphical lasso[J/OL]. *Biostatistics*, 2008, 9(3): 432-441. DOI: 10.1093/biostatistics/kxm045.
- [3] LIU H, LAFFERTY J, WASSERMAN L. The nonparanormal: Semiparametric estimation of high dimensional undirected graphs[J]. *Journal of Machine Learning Research*, 2009, 10: 2295-2328.
- [4] HYNDMAN R J, ATHANASOPOULOS G. *Forecasting: Principles and practice*[M/OL]. 3rd ed. Melbourne: OTexts, 2021. <https://otexts.com/fpp3/>.
- [5] NAHMIAS S. Perishable inventory theory: A review[J/OL]. *Operations Research*, 1982, 30(4): 680-708. DOI: 10.1287/opre.30.4.680.
- [6] PETRUZZI N C, DADA M. Pricing and the newsvendor problem: A review with extensions[J/OL]. *Operations Research*, 1999, 47(2): 183-194. DOI: 10.1287/opre.47.2.183.
- [7] SHAPIRO A, DENTCHEVA D, RUSZCZYŃSKI A. *Lectures on stochastic programming: Modeling and theory*[M/OL]. 2nd ed. Philadelphia: SIAM, 2014. DOI: 10.1137/1.9781611973433.
- [8] MCFADDEN D. Conditional logit analysis of qualitative choice behavior[M]// ZAREMBKA P. *Frontiers in Econometrics*. New York: Academic Press, 1974: 105-142.
- [9] VAN RYZIN G, MAHAJAN S. On the relationship between inventory costs and variety benefits in retail assortments[J/OL]. *Management Science*, 1999, 45(11): 1496-1509. DOI: 10.1287/mnsc.45.11.1496.
- [10] MAHAJAN S, VAN RYZIN G. Stocking retail assortments under dynamic consumer substitution[J/OL]. *Operations Research*, 2001, 49(3): 334-351. DOI: 10.1287/opre.49.3.334.11210.

## 附录 A 模型计算与执行规则

---

### 算法 1 定价与补货模型的计算流程

---

输入： 附件 1–4，预测日期 2023-07-01 至 2023-07-07

输出： 品类补货定价方案与单品组合方案

- 1: 分离正向销售、退货和打折记录，计算加权售价与损耗有效成本
  - 2: 拟合多时间尺度残差、Hurdle–Gamma、边际分布与 Copula 条件网络
  - 3: 估计工具变量岭回归价格弹性，完成 84 日滚动预测回测
  - 4: 对每个未来日联合抽样 1600 个需求—成本残差情景
  - 5: 网格搜索加成率和需求分位订货量，输出问题二方案
  - 6: 构造最近 7 日可售候选，以 MILP 和二分搜索最大化最低品类捕获率
  - 7: 在服务率下降不超过 1 个百分点时最大化利润代理
  - 8: 对入选单品进行 Hurdle–Lognormal 情景定价补货，检查全部约束
  - 9: 汇总各问结果，完成滚动回测、压力测试与约束复核
- 

#### 1.1 每日价格更新

表10展示的是预测中位成本下的参考售价。每日实际执行时，对品类  $k$  使用

$$P_{kt}^{\text{actual}} = \text{round}_{0.1} \left[ \frac{C_{kt}^{\text{actual}}}{1 - L_k} (1 + m_{kt}^*) \right]. \quad (\text{A.1})$$

实际成本超出历史 10%–90% 区间时，应重新生成情景；门店存在竞争限价时，应将价格上限作为显式约束加入模型。

#### 1.2 单品方案执行顺序

- (1) 凌晨依据图17给出的单品方案订购；若某单品临时不可供，优先选择同品类、低网络重复度且基础效用高的备选；
- (2) 实际成本到达后按单品损耗率和最优有效加成率更新售价，并取整到 0.1 元；
- (3) 营业中记录上架量、补架量、剩余库存和最后销售时刻；若提前售罄，次日提高需求分布右尾；
- (4) 闭店前根据品质和剩余量决定折价，折后销量与实际报损进入下一轮残值和损耗更新。

### 1.3 输入数据字典

表 17 附件字段、建模口径与输出去向

| 来源   | 原始字段      | 建模口径             | 主要用途      |
|------|-----------|------------------|-----------|
| 附件 1 | 单品编码、名称   | 编码按字符串读取         | 唯一连接键     |
| 附件 1 | 分类编码、名称   | 保留六个品类层级         | 层次聚合与服务约束 |
| 附件 2 | 销售日期、扫码时间 | 日期标准化, 时间转为距零点秒数 | 日历特征与售罄代理 |
| 附件 2 | 销量、销售类型   | 正向销售与退货分离        | 需求与净收入    |
| 附件 2 | 销售单价      | 按正向销量加权          | 固定篮子价格    |
| 附件 2 | 是否打折销售    | 计算打折销量占比         | 清货能力代理    |
| 附件 3 | 日期、批发价格   | 按日期一单品连接, 不任意补零  | 成本预测与工具变量 |
| 附件 4 | 品类平均损耗率   | 转换为 $[0, 1)$ 比例  | 品类有效成本    |
| 附件 4 | 单品损耗率     | 缺失时用所属品类补齐       | 单品可售量与利润  |

### 1.4 混合整数规划求解证书

表 18 问题三 MILP 求解状态证书

| 阶段            | 下界 | 上界 | 最低捕获率  | 状态      |
|---------------|----|----|--------|---------|
| 最大化最低品类捕获率    | 27 | 27 | 62.91% | Optimal |
| 最大化最低品类捕获率    | 28 | 28 | 64.39% | Optimal |
| 最大化最低品类捕获率    | 29 | 29 | 64.57% | Optimal |
| 最大化最低品类捕获率    | 30 | 30 | 64.69% | Optimal |
| 最大化最低品类捕获率    | 31 | 31 | 65.53% | Optimal |
| 最大化最低品类捕获率    | 32 | 32 | 65.67% | Optimal |
| 最大化最低品类捕获率    | 33 | 33 | 65.81% | Optimal |
| 最大化最低品类捕获率    | 27 | 33 | 65.81% | Optimal |
| 给定服务率后最大化利润代理 | 27 | 33 | 64.81% | Optimal |

注：前 8 行为二分搜索末次可行模型，最后一行为利润代理优化。

七个固定单品规模、全区间服务率优化及利润代理优化均达到表18所示的最优状态。该证书只证明给定线性化选择模型在约束范围内达到最优，不替代需求模型、价格弹性和外部选项假设的有效性检验。

## 附录 B MATLAB 主程序代码

```
1 function results = main()
2 %MAIN Solve all four questions of the 2023 C problem in MATLAB.
3 %
4 % Run from MATLAB with:
5 % cd('.../cases/2023C-vegetable-pricing/matlab');
6 % results = main;
7
8 cfg = paimon_config();
9 rng(cfg.seed, 'twister');
10 addpath(fileparts(mfilename('fullpath')));
11
12 logFile = fullfile(cfg.logDir, 'matlab-run.log');
13 if exist(logFile, 'file')
14     delete(logFile);
15 end
16 diary(logFile);
17 cleanup = onCleanup(@() diary('off'));
18
19 fprintf('Paimon 2023C MATLAB solver\n');
20 fprintf('MATLAB version: %s\n', version);
21 fprintf('Random seed: %d\n', cfg.seed);
22 fprintf('Started: %s\n', string(datetime('now')));
23
24 try
25     data = load_prepare_data(cfg);
26     q1 = solve_question1(cfg, data);
27     [forecast, q2] = solve_question2(cfg, data, q1);
28     q3 = solve_question3(cfg, data, q1, forecast, q2);
29     q4 = solve_question4(cfg);
30
31     validation = validate_results(cfg, data, q1, forecast, q2, q3, q4);
32     results = struct('config', cfg, 'q1', q1, 'forecast', forecast, ...
33         'q2', q2, 'q3', q3, 'q4', q4, 'validation', validation);
34     save(fullfile(cfg.outputDir, 'results.mat'), 'results', '-v7.3');
35     write_summary_json(cfg, results);
36
37     fprintf('Completed: %s\n', string(datetime('now')));
38     fprintf('Question 2 weekly order: %.1f kg\n', sum(q2.decisions("补货量")));
39     fprintf('Question 2 expected weekly profit: %.2f yuan\n', sum(q2.decisions("期望利润")));
40     fprintf('Question 3 selected items: %d\n', height(q3.itemPlan));
41     fprintf('Question 3 total order: %.1f kg\n', sum(q3.itemPlan("补货量")));
42     fprintf('Validation passed: %d\n', validation.passed);
43 catch ME
44     fprintf(2, 'FAILED: %s\n', getReport(ME, 'extended', 'hyperlinks', 'off'));
45     rethrow(ME);
46 end
47 end
48
49 function write_summary_json(cfg, results)
50 summary = struct();
51 summary.seed = cfg.seed;
52 salesRow = results.validation.dataAudit(results.validation.dataAudit("指标") == "销售流水数", :);
53 summary.sales_rows = str2double(salesRow("结果"));
54 summary.q2_week_order = sum(results.q2.decisions("补货量"));
55 summary.q2_week_expected_profit = sum(results.q2.decisions("期望利润"));
56 summary.q3_selected_count = height(results.q3.itemPlan);
57 summary.q3_total_order = sum(results.q3.itemPlan("补货量"));
58 summary.q3_expected_profit = sum(results.q3.itemPlan("期望利润"));
59 summary.validation_passed = results.validation.passed;
60 text = jsonencode(summary, PrettyPrint=true);
61 fid = fopen(fullfile(cfg.outputDir, 'summary.json'), 'w', 'n', 'UTF-8');
62 assert(fid > 0, 'Cannot create summary.json');
```

```

63 closer = onCleanup(@() fclose(fid));
64 fwrite(fid, text, 'char');
65 end

```

## 附录 C MATLAB 参数配置代码

```

1 function cfg = paimon_config()
2 %PAIMON_CONFIG Configuration for the 2023 C-problem MATLAB implementation.
3
4 matlabDir = fileparts(mfilename('fullpath'));
5 cfg.caseRoot = fileparts(matlabDir);
6 cfg.rawDir = fullfile(cfg.caseRoot, 'data', 'raw');
7 cfg.outputDir = fullfile(cfg.caseRoot, 'outputs', 'matlab');
8 cfg.tableDir = fullfile(cfg.outputDir, 'tables');
9 cfg.figureDir = fullfile(cfg.outputDir, 'figures');
10 cfg.logDir = fullfile(cfg.outputDir, 'logs');
11
12 cfg.seed = 20230822;
13 cfg.categories = ["花叶类"; "花菜类"; "水生根基类"; "茄类"; "辣椒类"; "食用菌"];
14 cfg.allDates = (datetime(2020, 7, 1):caldays(1):datetime(2023, 6, 30))';
15 cfg.futureDates = (datetime(2023, 7, 1):caldays(1):datetime(2023, 7, 7))';
16 cfg.backtestDays = 84;
17 cfg.scenarioCount = 1600;
18 cfg.itemScenarioCount = 2200;
19 cfg.minimumDisplay = 2.5;
20 cfg.assortmentLow = 27;
21 cfg.assortmentHigh = 33;
22 cfg.verbose = true;
23
24 folders = {cfg.outputDir, cfg.tableDir, cfg.figureDir, cfg.logDir};
25 for k = 1:numel(folders)
26     if ~isfolder(folders{k})
27         mkdir(folders{k});
28     end
29 end
30 end

```

## 附录 D 数据读取与预处理代码

```

1 function data = load_prepare_data(cfg)
2 %LOAD_PREPARE_DATA Read the four official attachments without modifying them.
3
4 fprintf('Reading official attachments...\n');
5 itemsFile = fullfile(cfg.rawDir, '附件1.xlsx');
6 salesFile = fullfile(cfg.rawDir, '附件2.xlsx');
7 wholesaleFile = fullfile(cfg.rawDir, '附件3.xlsx');
8 lossFile = fullfile(cfg.rawDir, '附件4.xlsx');
9 assert(all(isfile({itemsFile, salesFile, wholesaleFile, lossFile})), ...
10     'One or more official attachments are missing from data/raw.');
```

```

11
12 itemsRaw = readtable(itemsFile, VariableNamingRule='preserve', TextType='string');
13 items = table();
14 items.Code = normalize_codes(itemsRaw("单品编码"));
15 items.ItemName = string(itemsRaw("单品名称"));
16 items.CategoryCode = normalize_codes(itemsRaw("分类编码"));
17 items.Category = string(itemsRaw("分类名称"));
18 assert(numel(unique(items.Code)) == height(items), 'Item codes are not unique.');
```

```

19
20 salesRaw = readtable(salesFile, VariableNamingRule='preserve', TextType='string');
21 sales = table();
22 sales.Date = normalize_dates(salesRaw("销售日期"));

```

```

23 sales.Second = normalize_time_seconds(salesRaw("扫码销售时间"));
24 sales.Code = normalize_codes(salesRaw("单品编码"));
25 sales.Quantity = double(salesRaw("销量(千克)"));
26 sales.UnitPrice = double(salesRaw("销售单价(元/千克)"));
27 sales.SaleType = string(salesRaw("销售类型"));
28 sales.DiscountFlag = string(salesRaw("是否打折销售"));
29 sales.Quantity(~isfinite(sales.Quantity)) = 0;
30 sales.UnitPrice(~isfinite(sales.UnitPrice)) = 0;
31
32 isSale = sales.SaleType == "销售";
33 isReturn = sales.SaleType == "退货";
34 sales.PositiveQty = double(isSale) .* max(sales.Quantity, 0);
35 sales.ReturnQty = double(isReturn) .* abs(sales.Quantity);
36 sales.NetQty = sales.PositiveQty - sales.ReturnQty;
37 sales.PositiveRevenue = sales.PositiveQty .* sales.UnitPrice;
38 sales.NetRevenue = double(isSale) .* sales.Quantity .* sales.UnitPrice ...
39     - double(isReturn) .* abs(sales.Quantity) .* sales.UnitPrice;
40 sales.DiscountQty = double(isSale & sales.DiscountFlag == "是") .* sales.PositiveQty;
41
42 [found, itemLocation] = ismember(sales.Code, items.Code);
43 assert(all(found), 'Sales attachment contains item codes not found in attachment 1.');
```

```

44 sales.ItemName = items.ItemName(itemLocation);
45 sales.CategoryCode = items.CategoryCode(itemLocation);
46 sales.Category = items.Category(itemLocation);
47
48 fprintf('Aggregating %d sales rows to item-day level...\n', height(sales));
49 [G, itemDate, itemCode] = findgroups(sales.Date, sales.Code);
50 dailyItem = table(itemDate, itemCode, VariableNames={'Date', 'Code'});
51 dailyItem.PositiveQty = splitapply(@sum, sales.PositiveQty, G);
52 dailyItem.ReturnQty = splitapply(@sum, sales.ReturnQty, G);
53 dailyItem.NetQty = splitapply(@sum, sales.NetQty, G);
54 dailyItem.PositiveRevenue = splitapply(@sum, sales.PositiveRevenue, G);
55 dailyItem.NetRevenue = splitapply(@sum, sales.NetRevenue, G);
56 dailyItem.DiscountQty = splitapply(@sum, sales.DiscountQty, G);
57 dailyItem.LastSecond = splitapply(@max_omitnan, sales.Second, G);
58 dailyItem.TransactionCount = splitapply(@numel, sales.Code, G);
59 [found, itemLocation] = ismember(dailyItem.Code, items.Code);
60 assert(all(found), 'Daily aggregation created an unknown item code.');
```

```

61 dailyItem.ItemName = items.ItemName(itemLocation);
62 dailyItem.CategoryCode = items.CategoryCode(itemLocation);
63 dailyItem.Category = items.Category(itemLocation);
64 dailyItem.WeightedPrice = dailyItem.PositiveRevenue ./ dailyItem.PositiveQty;
65 dailyItem.WeightedPrice(dailyItem.PositiveQty <= 0) = NaN;
66 dailyItem.DiscountShare = dailyItem.DiscountQty ./ max(dailyItem.PositiveQty, eps);
67
68 wholesaleRaw = readtable(wholesaleFile, VariableNamingRule='preserve', TextType='string');
69 wholesale = table();
70 wholesale.Date = normalize_dates(wholesaleRaw("日期"));
71 wholesale.Code = normalize_codes(wholesaleRaw("单品编码"));
72 wholesale.WholesalePrice = double(wholesaleRaw("批发价格(元/千克)"));
73 [found, itemLocation] = ismember(wholesale.Code, items.Code);
74 assert(all(found), 'Wholesale attachment contains item codes not found in attachment 1.');
```

```

75 wholesale.ItemName = items.ItemName(itemLocation);
76 wholesale.Category = items.Category(itemLocation);
77
78 lossItemRaw = readtable(lossFile, Sheet='Sheet1', ...
79     VariableNamingRule='preserve', TextType='string');
80 lossItem = table();
81 lossItem.Code = normalize_codes(lossItemRaw("单品编码"));
82 lossItem.ItemName = string(lossItemRaw("单品名称"));
83 lossItem.LossRate = double(lossItemRaw("损耗率(%))") / 100;
84
85 lossSheets = sheetnames(lossFile);
86 categorySheet = lossSheets(lossSheets ~= "Sheet1");
```

```

87 assert(~isempty(categorySheet), 'Category loss-rate sheet is missing.');
```

```

88 lossCategoryRaw = readtable(lossFile, Sheet=categorySheet(1), ...
89     VariableNamingRule='preserve', TextType='string');
```

```

90 lossCategory = table();
91 lossCategory.Category = string(lossCategoryRaw.("小分类名称"));
92 lossCategory.LossRate = double(lossCategoryRaw.("平均损耗率(%)_小分类编码_不同值")) / 100;
93
94 dailyItem = outerjoin(dailyItem, wholesale(:, {'Date', 'Code', 'WholesalePrice'}), ...
95     Keys={'Date', 'Code'}, MergeKeys=true, Type='left');
```

```

96
97 [G, categoryDate, categoryName] = findgroups(dailyItem.Date, dailyItem.Category);
98 categoryAgg = table(categoryDate, categoryName, VariableNames={'Date', 'Category'});
99 categoryAgg.PositiveQty = splitapply(@sum, dailyItem.PositiveQty, G);
100 categoryAgg.ReturnQty = splitapply(@sum, dailyItem.ReturnQty, G);
101 categoryAgg.NetQty = splitapply(@sum, dailyItem.NetQty, G);
102 categoryAgg.PositiveRevenue = splitapply(@sum, dailyItem.PositiveRevenue, G);
103 categoryAgg.NetRevenue = splitapply(@sum, dailyItem.NetRevenue, G);
104 categoryAgg.DiscountQty = splitapply(@sum, dailyItem.DiscountQty, G);
105 categoryAgg.ActiveItems = splitapply(@(x) numel(unique(x)), dailyItem.Code, G);
106 categoryAgg.TransactionWeightedPrice = categoryAgg.PositiveRevenue ./ categoryAgg.PositiveQty;
107 categoryAgg.DiscountShare = categoryAgg.DiscountQty ./ max(categoryAgg.PositiveQty, eps);
108
109 [dateGrid, categoryGrid] = ndgrid(cfg.allDates, cfg.categories);
110 categoryDaily = table(dateGrid(:), categoryGrid(:), VariableNames={'Date', 'Category'});
111 categoryDaily = outerjoin(categoryDaily, categoryAgg, Keys={'Date', 'Category'}, ...
112     MergeKeys=true, Type='left');
```

```

113 zeroVariables = {'PositiveQty', 'ReturnQty', 'NetQty', 'PositiveRevenue', 'NetRevenue', ...
114     'DiscountQty', 'ActiveItems', 'DiscountShare'};
115 for k = 1:numel(zeroVariables)
116     name = zeroVariables{k};
117     categoryDaily.(name)(isnan(categoryDaily.(name))) = 0;
118 end
119
120 [fixedPrice, fixedCost] = fixed_basket_indices(cfg, dailyItem, wholesale);
121 categoryDaily = outerjoin(categoryDaily, fixedPrice, Keys={'Date', 'Category'}, ...
122     MergeKeys=true, Type='left');
```

```

123 categoryDaily = outerjoin(categoryDaily, fixedCost, Keys={'Date', 'Category'}, ...
124     MergeKeys=true, Type='left');
```

```

125 categoryDaily = outerjoin(categoryDaily, lossCategory, Keys='Category', ...
126     MergeKeys=true, Type='left');
```

```

127 categoryDaily.CategoryOrder = category_index(categoryDaily.Category, cfg.categories);
128 categoryDaily = sortrows(categoryDaily, {'Date', 'CategoryOrder'});
129
130 for k = 1:numel(cfg.categories)
131     mask = categoryDaily.Category == cfg.categories(k);
132     categoryDaily.FixedPrice(mask) = fillmissing(categoryDaily.FixedPrice(mask), ...
133         'linear', EndValues='nearest');
```

```

134     categoryDaily.FixedCost(mask) = fillmissing(categoryDaily.FixedCost(mask), ...
135         'linear', EndValues='nearest');
```

```

136 end
137 categoryDaily.EffectiveMarkup = categoryDaily.FixedPrice .* (1 - categoryDaily.LossRate) ...
138     ./ categoryDaily.FixedCost - 1;
139
140 data = struct();
141 data.items = items;
142 data.sales = sales;
143 data.dailyItem = sortrows(dailyItem, {'Date', 'Code'});
144 data.wholesale = sortrows(wholesale, {'Date', 'Code'});
145 data.lossItem = lossItem;
146 data.lossCategory = lossCategory;
147 data.categoryDaily = categoryDaily;
148
149 writetable(data.categoryDaily, fullfile(cfg.tableDir, 'prepared_category_daily.csv'), ...
150     Encoding='UTF-8');
```

```

151 fprintf('Prepared %d item-day rows and %d category-day rows.\n', ...
152         height(data.dailyItem), height(data.categoryDaily));
153 end
154
155 function codes = normalize_codes(values)
156 if isnumeric(values)
157     codes = compose('%.0f', values);
158 else
159     codes = strip(string(values));
160     numericLike = ~ismissing(codes) & ~contains(codes, lettersPattern);
161     if any(numericLike)
162         parsed = str2double(codes(numericLike));
163         ok = isfinite(parsed);
164         tmp = codes(numericLike);
165         tmp(ok) = compose('%.0f', parsed(ok));
166         codes(numericLike) = tmp;
167     end
168 end
169 codes = string(codes(:));
170 end
171
172 function dates = normalize_dates(values)
173 if isdatetime(values)
174     dates = values;
175 elseif isnumeric(values)
176     dates = datetime(values, ConvertFrom='excel');
177 else
178     dates = datetime(string(values));
179 end
180 dates = dateshift(dates(:), 'start', 'day');
181 end
182
183 function secondsValue = normalize_time_seconds(values)
184 if isduration(values)
185     secondsValue = seconds(values);
186 elseif isdatetime(values)
187     secondsValue = seconds(timeofday(values));
188 elseif isnumeric(values)
189     secondsValue = double(values) * 86400;
190 else
191     text = string(values);
192     secondsValue = nan(numel(text), 1);
193     for k = 1:numel(text)
194         try
195             d = duration(text(k), InputFormat='hh:mm:ss.SSS');
196         catch
197             try
198                 d = duration(text(k), InputFormat='hh:mm:ss');
199             catch
200                 d = seconds(NaN);
201             end
202         end
203         secondsValue(k) = seconds(d);
204     end
205 end
206 secondsValue = double(secondsValue(:));
207 end
208
209 function value = max_omitnan(x)
210 value = max(x, [], 'omitnan');
211 if isempty(value)
212     value = NaN;
213 end
214 end

```



```

215
216 function index = category_index(values, categories)
217 [found, index] = ismember(string(values), categories);
218 assert(all(found), 'Unexpected vegetable category found.');
```

```

219 end
220
221 function [priceTable, costTable] = fixed_basket_indices(cfg, dailyItem, wholesale)
222 nDates = numel(cfg.allDates);
223 priceRows = cell(numel(cfg.categories), 1);
224 costRows = cell(numel(cfg.categories), 1);
225
226 for k = 1:numel(cfg.categories)
227     category = cfg.categories(k);
228     subset = dailyItem(dailyItem.Category == category, :);
229     [G, codes] = findgroups(subset.Code);
230     totalQty = splitapply(@sum, subset.PositiveQty, G);
231     saleDays = splitapply(@(x) sum(x > 0), subset.PositiveQty, G);
232     itemStats = table(codes, totalQty, saleDays);
233     eligible = itemStats.saleDays >= 20;
234     if ~any(eligible)
235         eligible(:) = true;
236     end
237     itemStats = itemStats(eligible, :);
238     itemStats.Weight = itemStats.totalQty / sum(itemStats.totalQty);
239     itemStats = sortrows(itemStats, 'Weight', 'descend');
240     priorCumulative = [0; cumsum(itemStats.Weight(1:end-1))];
241     itemStats = itemStats(priorCumulative < 0.98, :);
242     itemStats.Weight = itemStats.Weight / sum(itemStats.Weight);
243     codes = itemStats.codes;
244     nItems = numel(codes);
245
246     [dateFound, dateIndex] = ismember(subset.Date, cfg.allDates);
247     [codeFound, codeIndex] = ismember(subset.Code, codes);
248     keep = dateFound & codeFound & isfinite(subset.WeightedPrice);
249     priceMatrix = accumarray([dateIndex(keep), codeIndex(keep)], ...
250         subset.WeightedPrice(keep), [nDates, nItems], @mean, NaN);
251     priceMatrix = fill_short_gaps(priceMatrix, 14);
252     fixedPrice = weighted_nanmean(priceMatrix, itemStats.Weight);
253
254     costSubset = wholesale(wholesale.Category == category, :);
255     [dateFound, dateIndex] = ismember(costSubset.Date, cfg.allDates);
256     [codeFound, codeIndex] = ismember(costSubset.Code, codes);
257     keep = dateFound & codeFound & isfinite(costSubset.WholesalePrice);
258     costMatrix = accumarray([dateIndex(keep), codeIndex(keep)], ...
259         costSubset.WholesalePrice(keep), [nDates, nItems], @mean, NaN);
260     costMatrix = fill_short_gaps(costMatrix, 14);
261     fixedCost = weighted_nanmean(costMatrix, itemStats.Weight);
262
263     priceRows{k} = table(cfg.allDates, repmat(category, nDates, 1), fixedPrice, ...
264         VariableNames={'Date', 'Category', 'FixedPrice'});
265     costRows{k} = table(cfg.allDates, repmat(category, nDates, 1), fixedCost, ...
266         VariableNames={'Date', 'Category', 'FixedCost'});
267 end
268 priceTable = vertcat(priceRows{:});
269 costTable = vertcat(costRows{:});
270 end
271
272 function matrix = fill_short_gaps(matrix, maxGap)
273 for j = 1:size(matrix, 2)
274     x = matrix(:, j);
275     missing = isnan(x);
276     runs = regionprops(missing, 'PixelIdxList');
277     for r = 1:numel(runs)
278         idx = runs(r).PixelIdxList;

```

```

279     if numel(idx) <= maxGap
280         left = idx(1) - 1;
281         right = idx(end) + 1;
282         if left >= 1 && ~isnan(x(left))
283             x(idx) = x(left);
284         elseif right <= numel(x) && ~isnan(x(right))
285             x(idx) = x(right);
286         end
287     end
288 end
289 matrix(:, j) = x;
290 end
291 end
292
293 function result = weighted_nanmean(matrix, weights)
294 weights = weights(:)';
295 valid = isfinite(matrix);
296 numerator = sum(fillmissing(matrix, 'constant', 0) .* weights, 2);
297 denominator = sum(valid .* weights, 2);
298 result = numerator ./ denominator;
299 result(denominator <= 0) = NaN;
300 end

```

## 附录 E 日历特征构造代码

```

1 function [X, names] = calendar_features(dates, origin)
2 %CALENDAR_FEATURES Trend, weekday, month and annual Fourier features.
3
4 dates = dates(:);
5 t = days(dates - origin);
6 X = [t / 365.25, (t / 365.25).^2];
7 names = ["trend", "trend2"];
8
9 weekdayNumber = weekday(dates); % Sunday=1, Monday=2, ..., Saturday=7.
10 for d = 3:7
11     X(:, end + 1) = double(weekdayNumber == d); %#ok<AGROW>
12     names(end + 1) = "dow_" + string(d - 2); %#ok<AGROW>
13 end
14 X(:, end + 1) = double(weekdayNumber == 1);
15 names(end + 1) = "dow_6";
16
17 monthNumber = month(dates);
18 for m = 2:12
19     X(:, end + 1) = double(monthNumber == m); %#ok<AGROW>
20     names(end + 1) = "month_" + string(m); %#ok<AGROW>
21 end
22 for h = 1:3
23     X(:, end + 1) = sin(2 * pi * h * t / 365.25); %#ok<AGROW>
24     X(:, end + 1) = cos(2 * pi * h * t / 365.25); %#ok<AGROW>
25     names(end + 1:end + 2) = ["sin_" + string(h), "cos_" + string(h)]; %#ok<AGROW>
26 end
27 end

```

## 附录 F GCV 岭回归代码

```

1 function model = ridge_gcv(X, y, lambdas)
2 %RIDGE_GCV Ridge regression with generalized-cross-validation selection.
3
4 if nargin < 3 || isempty(lambdas)
5     lambdas = logspace(-4, 3, 30);
6 end

```

```

7 X = double(X);
8 y = double(y(:));
9 valid = all(isfinite(X), 2) & isfinite(y);
10 X = X(valid, :);
11 y = y(valid);
12 assert(size(X, 1) > size(X, 2) + 5, 'Not enough valid rows for ridge regression.');
```

```

13
14 muX = mean(X, 1);
15 sigmaX = std(X, 0, 1);
16 sigmaX(sigmaX < 1e-10) = 1;
17 Z = (X - muX) ./ sigmaX;
18 muY = mean(y);
19 yc = y - muY;
20
21 [~, S, ~] = svd(Z, 'econ');
22 s2 = diag(S).^2;
23 XtX = Z' * Z;
24 Xty = Z' * yc;
25 n = numel(y);
26 scores = inf(numel(lambdas), 1);
27 betas = zeros(size(X, 2), numel(lambdas));
28 for k = 1:numel(lambdas)
29     lambda = lambdas(k);
30     beta = (XtX + lambda * eye(size(X, 2))) \ Xty;
31     resid = yc - Z * beta;
32     df = sum(s2 ./ (s2 + lambda));
33     scores(k) = n * sum(resid.^2) / max((n - df - 1)^2, 1);
34     betas(:, k) = beta;
35 end
36 [~, best] = min(scores);
37 betaStandard = betas(:, best);
38 coef = betaStandard ./ sigmaX';
39 intercept = muY - muX * coef;
40
41 model = struct();
42 model.intercept = intercept;
43 model.coef = coef;
44 model.lambda = lambdas(best);
45 model.gcv = scores(best);
46 model.predict = @(Xnew) intercept + double(Xnew) * coef;
47 end

```

## 附录 G 问题一需求特征与价格弹性求解代码

```

1 function q1 = solve_question1(cfg, data)
2 %SOLVE_QUESTION1 Distribution, censored-demand proxy, network and elasticity.
3
4 fprintf('Solving question 1...\n');
5 categoryDaily = data.categoryDaily;
6 nCategory = numel(cfg.categories);
7 nDate = numel(cfg.allDates);
8
9 summaryRows = cell(nCategory, 1);
10 for k = 1:nCategory
11     category = cfg.categories(k);
12     subset = categoryDaily(categoryDaily.Category == category, :);
13     q = subset.PositiveQty;
14     summaryRows{k} = table(category, mean(q), median(q), std(q), min(q), max(q), ...
15         mean(subset.ReturnQty), mean(subset.ActiveItems), mean(subset.DiscountShare), ...
16         std(q) / max(mean(q), eps), skewness(q, 0), subset.LossRate(1), ...
17         VariableNames={'分类名称', '日均销量', '日中位销量', '日销量标准差', '最小日销量', ...
18             '最大日销量', '日均退货量', '日均活跃单品数', '日均打折占比', '变异系数', '偏度', '损耗率'});
19 end

```

```

20 summary = vertcat(summaryRows{:});
21 writetable(summary, fullfile(cfg.tableDir, 'q1_category_distribution_summary.csv'), Encoding='UTF-8'
    );
22
23 distributionFit = fit_distributions(cfg, categoryDaily);
24 writetable(distributionFit, fullfile(cfg.tableDir, 'q1_distribution_fits.csv'), Encoding='UTF-8');
25
26 weekdayRows = cell(nCategory, 1);
27 weekdayNames = ["周日"; "周一"; "周二"; "周三"; "周四"; "周五"; "周六"];
28 for k = 1:nCategory
29     subset = categoryDaily(categoryDaily.Category == cfg.categories(k), :);
30     dayNumber = weekday(subset.Date);
31     means = accumarray(dayNumber, subset.PositiveQty, [7, 1], @mean, NaN);
32     relative = means / mean(means, 'omitnan');
33     weekdayRows{k} = table(repmat(cfg.categories(k), 7, 1), (0:6)', means, relative, ...
34         weekdayNames, VariableNames={'分类名称', '星期', '正向销量', '相对指数', '星期名称'});
35 end
36 weekdayTable = vertcat(weekdayRows{:});
37 writetable(weekdayTable, fullfile(cfg.tableDir, 'q1_weekday_indices.csv'), Encoding='UTF-8');
38
39 [quantity, active, discount, price, cost] = category_matrices(cfg, categoryDaily);
40 [residuals, fitted] = seasonal_residuals(cfg, quantity, active, discount);
41 rawCorrelation = corr(log1p(quantity), Rows='pairwise');
42 residualCorrelation = corr(residuals, Rows='pairwise');
43
44 normalScores = zeros(size(residuals));
45 for k = 1:nCategory
46     ranks = tiedrank(residuals(:, k));
47     normalScores(:, k) = norminv((ranks - 0.5) / numel(ranks));
48 end
49 [precision, glassoLambda] = graphical_lasso_select(normalScores);
50 partialCorrelation = -precision ./ sqrt(diag(precision) * diag(precision));
51 partialCorrelation(1:nCategory + 1:end) = 1;
52
53 matrix_to_table(cfg, rawCorrelation, 'q1_raw_correlation.csv');
54 matrix_to_table(cfg, residualCorrelation, 'q1_residual_correlation.csv');
55 matrix_to_table(cfg, partialCorrelation, 'q1_copula_partial_correlation.csv');
56
57 edgeCategory1 = strings(0, 1);
58 edgeCategory2 = strings(0, 1);
59 edgeValue = zeros(0, 1);
60 edgeRelation = strings(0, 1);
61 for i = 1:nCategory
62     for j = i + 1:nCategory
63         value = partialCorrelation(i, j);
64         if abs(value) >= 0.10
65             edgeCategory1(end + 1, 1) = cfg.categories(i); %#ok<AGROW>
66             edgeCategory2(end + 1, 1) = cfg.categories(j); %#ok<AGROW>
67             edgeValue(end + 1, 1) = value; %#ok<AGROW>
68             edgeRelation(end + 1, 1) = ternary(value > 0, "同向联动", "反向联动"); %#ok<AGROW>
69         end
70     end
71 end
72 edges = table(edgeCategory1, edgeCategory2, edgeValue, edgeRelation, ...
73     VariableNames={'品类1', '品类2', '条件相关', '关系'});
74 [~, order] = sort(abs(edges.("条件相关")), 'descend');
75 edges = edges(order, :);
76 writetable(edges, fullfile(cfg.tableDir, 'q1_copula_network_edges.csv'), Encoding='UTF-8');
77
78 [elasticity, elasticityDiagnostics] = estimate_elasticity(cfg, quantity, price, cost, active,
    discount);
79 elasticityTable = array2table(elasticity, VariableNames=cellstr(cfg.categories));
80 elasticityTable = addvars(elasticityTable, cfg.categories, Before=1, NewVariableNames='需求品类');
81 writetable(elasticityTable, fullfile(cfg.tableDir, 'q1_cross_price_elasticities.csv'), Encoding='UTF

```

```

-8');
82 writetable(elasticityDiagnostics, fullfile(cfg.tableDir, 'q1_elasticity_diagnostics.csv'), Encoding=
    'UTF-8');
83
84 hurdleTable = hurdle_gamma_table(data.dailyItem);
85 writetable(hurdleTable, fullfile(cfg.tableDir, 'q1_core_item_hurdle_gamma.csv'), Encoding='UTF-8');
86
87 dataAudit = table(["销售流水数"; "正向销售流水数"; "退货流水数"; "打折销售流水数"; ...
88     "单品数"; "品类数"; "批发价记录数"; "销售起始日"; "销售结束日"], ...
89     [string(height(data.sales)); string(sum(data.sales.SaleType == "销售")); ...
90     string(sum(data.sales.SaleType == "退货")); string(sum(data.sales.DiscountFlag == "是")); ...
91     string(numel(unique(data.items.Code))); string(numel(unique(data.items.Category))); ...
92     string(height(data.wholesale)); string(min(data.sales.Date), 'yyyy-MM-dd'); ...
93     string(max(data.sales.Date), 'yyyy-MM-dd')], VariableNames={'指标', '结果'});
94 writetable(dataAudit, fullfile(cfg.tableDir, 'data_quality_summary.csv'), Encoding='UTF-8');
95
96 make_q1_figures(cfg, quantity, rawCorrelation, residualCorrelation, partialCorrelation);
97
98 q1 = struct();
99 q1.summary = summary;
100 q1.distributionFit = distributionFit;
101 q1.weekday = weekdayTable;
102 q1.residuals = residuals;
103 q1.fitted = fitted;
104 q1.rawCorrelation = rawCorrelation;
105 q1.residualCorrelation = residualCorrelation;
106 q1.partialCorrelation = partialCorrelation;
107 q1.glassoLambda = glassoLambda;
108 q1.edges = edges;
109 q1.elasticity = elasticity;
110 q1.elasticityDiagnostics = elasticityDiagnostics;
111 q1.hurdle = hurdleTable;
112 q1.dataAudit = dataAudit;
113 fprintf('Question 1 complete: %d network edges, Graphical Lasso lambda %.4f.\n', ...
114     height(edges), glassoLambda);
115 end
116
117 function distributionFit = fit_distributions(cfg, categoryDaily)
118 rows = cell(numel(cfg.categories) * 3, 1);
119 cursor = 0;
120 names = ["Gamma", "Lognormal", "Weibull"];
121 matlabNames = {'Gamma', 'Lognormal', 'Weibull'};
122 for k = 1:numel(cfg.categories)
123     values = categoryDaily.PositiveQty(categoryDaily.Category == cfg.categories(k));
124     values = values(values > 0 & isfinite(values));
125     for d = 1:numel(names)
126         cursor = cursor + 1;
127         pd = fitdist(values, matlabNames{d});
128         nll = pd.NLogL;
129         parameterCount = numel(pd.ParameterValues);
130         aic = 2 * parameterCount + 2 * nll;
131         bic = log(numel(values)) * parameterCount + 2 * nll;
132         [~, pValue, ksStatistic] = kstest(values, CDF=pd);
133         rows{cursor} = table(cfg.categories(k), names(d), aic, bic, ksStatistic, pValue, ...
134             VariableNames={'分类名称', '分布', 'AIC', 'BIC', 'KS统计量', 'KS_p值'});
135     end
136 end
137 distributionFit = vertcat(rows{:});
138 distributionFit("AIC排名") = zeros(height(distributionFit), 1);
139 for k = 1:numel(cfg.categories)
140     mask = distributionFit("分类名称") == cfg.categories(k);
141     [~, order] = sort(distributionFit.AIC(mask));
142     ranks = zeros(sum(mask), 1);
143     ranks(order) = 1:sum(mask);

```

```

144     distributionFit("AIC排名")(mask) = ranks;
145 end
146 distributionFit = sortrows(distributionFit, {'分类名称', 'AIC'});
147 end
148
149 function [quantity, active, discount, price, cost] = category_matrices(cfg, categoryDaily)
150 nDate = numel(cfg.allDates);
151 nCategory = numel(cfg.categories);
152 quantity = zeros(nDate, nCategory);
153 active = zeros(nDate, nCategory);
154 discount = zeros(nDate, nCategory);
155 price = nan(nDate, nCategory);
156 cost = nan(nDate, nCategory);
157 for k = 1:nCategory
158     subset = categoryDaily(categoryDaily.Category == cfg.categories(k), :);
159     subset = sortrows(subset, 'Date');
160     quantity(:, k) = subset.PositiveQty;
161     active(:, k) = subset.ActiveItems;
162     discount(:, k) = subset.DiscountShare;
163     price(:, k) = subset.FixedPrice;
164     cost(:, k) = subset.FixedCost;
165 end
166 end
167
168 function [residuals, fitted] = seasonal_residuals(cfg, quantity, active, discount)
169 [base, ~] = calendar_features(cfg.allDates, cfg.allDates(1));
170 residuals = zeros(size(quantity));
171 fitted = zeros(size(quantity));
172 for k = 1:numel(cfg.categories)
173     X = [base, log1p(active(:, k)), discount(:, k)];
174     y = log1p(quantity(:, k));
175     model = ridge_gcv(X, y, logspace(-3, 3, 25));
176     pred = model.predict(X);
177     fitted(:, k) = pred;
178     residuals(:, k) = y - pred;
179 end
180 end
181
182 function [precision, bestLambda] = graphical_lasso_select(normalScores)
183 S = cov(normalScores, 1);
184 n = size(normalScores, 1);
185 lambdas = linspace(0.01, 0.25, 18);
186 bestScore = inf;
187 precision = inv(S + 0.05 * eye(size(S)));
188 bestLambda = NaN;
189 for lambda = lambdas
190     theta = graphical_lasso_admm(S, lambda);
191     [~, logDet] = logdet_spd(theta);
192     df = nnz(abs(triu(theta, 1)) > 1e-5) + size(theta, 1);
193     score = n * (trace(S * theta) - logDet) + log(n) * df;
194     if score < bestScore
195         bestScore = score;
196         precision = theta;
197         bestLambda = lambda;
198     end
199 end
200 end
201
202 function theta = graphical_lasso_admm(S, lambda)
203 p = size(S, 1);
204 rho = 1;
205 theta = eye(p);
206 Z = eye(p);
207 U = zeros(p);

```

```

208 for iteration = 1:500
209     A = rho * (Z - U) - S;
210     A = (A + A') / 2;
211     [Q, D] = eig(A);
212     eigenvalues = diag(D);
213     thetaNew = Q * diag((eigenvalues + sqrt(eigenvalues.^2 + 4 * rho)) / (2 * rho)) * Q';
214     thetaNew = (thetaNew + thetaNew') / 2;
215     V = thetaNew + U;
216     Znew = sign(V) .* max(abs(V) - lambda / rho, 0);
217     Znew(1:p + 1:end) = diag(V);
218     U = U + thetaNew - Znew;
219     primal = norm(thetaNew - Znew, 'fro');
220     dual = rho * norm(Znew - Z, 'fro');
221     theta = thetaNew;
222     Z = Znew;
223     if primal < 1e-7 && dual < 1e-7
224         break;
225     end
226 end
227 end
228
229 function [signValue, logDet] = logdet_spd(A)
230 R = chol((A + A') / 2);
231 logDet = 2 * sum(log(diag(R)));
232 signValue = 1;
233 end
234
235 function [elasticity, diagnostics] = estimate_elasticity(cfg, quantity, price, cost, active,
    discount)
236 [controls, ~] = calendar_features(cfg.allDates, cfg.allDates(1));
237 for k = 1:numel(cfg.categories)
238     controls = [controls, log1p(active(:, k)), discount(:, k)]; %#ok<AGROW>
239 end
240 logPrice = log(max(price, 0.05));
241 logCost = log(max(cost, 0.05));
242 valid = all(isfinite(logPrice), 2) & all(isfinite(logCost), 2) & all(isfinite(controls), 2);
243 logPrice = logPrice(valid, :);
244 logCost = logCost(valid, :);
245 controls = controls(valid, :);
246 quantity = quantity(valid, :);
247
248 firstX = [logCost, controls];
249 predictedPrice = zeros(size(logPrice));
250 firstR2 = zeros(numel(cfg.categories), 1);
251 for k = 1:numel(cfg.categories)
252     model = ridge_gcv(firstX, logPrice(:, k), logspace(-4, 3, 30));
253     predictedPrice(:, k) = model.predict(firstX);
254     firstR2(k) = 1 - sum((logPrice(:, k) - predictedPrice(:, k)).^2) ...
255         / sum((logPrice(:, k) - mean(logPrice(:, k))).^2);
256 end
257
258 secondX = [predictedPrice, controls];
259 elasticity = zeros(numel(cfg.categories));
260 ridgeParameter = zeros(numel(cfg.categories), 1);
261 rawOwn = zeros(numel(cfg.categories), 1);
262 for k = 1:numel(cfg.categories)
263     model = ridge_gcv(secondX, log1p(quantity(:, k)), logspace(-4, 3, 30));
264     elasticity(k, :) = model.coef(1:numel(cfg.categories));
265     ridgeParameter(k) = model.lambda;
266     rawOwn(k) = elasticity(k, k);
267 end
268 for k = 1:numel(cfg.categories)
269     elasticity(k, k) = min(elasticity(k, k), -0.20);
270 end

```

```

271 elasticity = min(max(elasticity, -2.5), 2.5);
272 diagnostics = table(cfg.categories, firstR2, ridgeParameter, rawOwn, -diag(elasticity), ...
273     VariableNames={'分类名称', '第一阶段R2', '岭参数', '原始自身价格系数', '决策用自身弹性'});
274 end
275
276 function hurdle = hurdle_gamma_table(dailyItem)
277 [G, code, name, category] = findgroups(dailyItem.Code, dailyItem.ItemName, dailyItem.Category);
278 totalQty = splitapply(@sum, dailyItem.PositiveQty, G);
279 saleDays = splitapply(@(x) sum(x > 0), dailyItem.PositiveQty, G);
280 rankTable = table(code, name, category, totalQty, saleDays);
281 rankTable = sortrows(rankTable, {'saleDays', 'totalQty'}, {'descend', 'descend'});
282 rankTable = rankTable(1:min(20, height(rankTable)), :);
283
284 rows = cell(height(rankTable), 1);
285 for r = 1:height(rankTable)
286     subset = dailyItem(dailyItem.Code == rankTable.code(r), :);
287     dateRange = (min(subset.Date):caldays(1):max(subset.Date));
288     [found, location] = ismember(dateRange, subset.Date);
289     quantities = zeros(numel(dateRange), 1);
290     quantities(found) = subset.PositiveQty(location(found));
291     positive = quantities(quantities > 0);
292     params = gamfit(positive);
293     positiveRows = subset(subset.PositiveQty > 0, :);
294     threshold = quantile(positiveRows.PositiveQty, 0.75);
295     suspected = sum(positiveRows.LastSecond < 18 * 3600 & positiveRows.PositiveQty >= threshold);
296     rows{r} = table(rankTable.code(r), rankTable.name(r), rankTable.category(r), ...
297         numel(dateRange), sum(quantities > 0), mean(quantities > 0), params(1), params(2), suspected
298         , ...
299         VariableNames={'单品编码', '单品名称', '分类名称', '活跃区间天数', '正销量天数', ...
300             '需求发生概率', 'Gamma形状', 'Gamma尺度', '疑似售罄日_18点'});
301 end
302 hurdle = vertcat(rows{:});
303 end
304
305 function matrix_to_table(cfg, matrix, fileName)
306 T = array2table(matrix, VariableNames=cellstr(cfg.categories));
307 T = addvars(T, cfg.categories, Before=1, NewVariableNames='分类名称');
308 writetable(T, fullfile(cfg.tableDir, fileName), Encoding='UTF-8');
309 end
310
311 function make_q1_figures(cfg, quantity, rawCorrelation, residualCorrelation, partialCorrelation)
312 f = figure(Visible='off', Color='w', Position=[100 100 1300 760]);
313 tiledlayout(2, 3, TileSpacing='compact', Padding='compact');
314 for k = 1:numel(cfg.categories)
315     nexttile;
316     plot(cfg.allDates, quantity(:, k), Color=[0.18 0.45 0.62], LineWidth=0.8);
317     title(cfg.categories(k)); ylabel('日销量/kg'); grid on;
318 end
319 sgtitle('六品类日销量时间演化');
320 exportgraphics(f, fullfile(cfg.figureDir, 'q1_category_sales.png'), Resolution=180);
321 close(f);
322
323 f = figure(Visible='off', Color='w', Position=[100 100 1400 430]);
324 tiledlayout(1, 3, TileSpacing='compact', Padding='compact');
325 matrices = {rawCorrelation, residualCorrelation, partialCorrelation};
326 titles = {'原始对数销量相关', '去季节残差相关', 'Copula条件相关'};
327 for k = 1:3
328     nexttile;
329     imagesc(matrices{k}, [-1 1]); axis image; colorbar; title(titles{k});
330     xticks(1:6); yticks(1:6); xticklabels(cfg.categories); yticklabels(cfg.categories);
331     xtickangle(30);
332 end
333 colormap(redblue_colormap(256));
334 exportgraphics(f, fullfile(cfg.figureDir, 'q1_correlation_comparison.png'), Resolution=180);

```



```

334 close(f);
335 end
336
337 function cmap = redblue_colormap(n)
338 x = linspace(0, 1, n)';
339 cmap = [x, x, ones(n, 1); ones(n, 1), flipud(x), flipud(x)];
340 cmap = interp1(linspace(0, 1, size(cmap, 1)), cmap, linspace(0, 1, n));
341 end
342
343 function value = ternary(condition, yesValue, noValue)
344 if condition
345     value = yesValue;
346 else
347     value = noValue;
348 end
349 end

```

## 附录 H 问题二定价与补货决策代码

```

1 function [forecast, q2] = solve_question2(cfg, data, q1)
2 %SOLVE_QUESTION2 Ensemble forecasts and two-stage stochastic newsvendor.
3
4 fprintf('Solving question 2...\n');
5 [quantity, cost] = category_series(cfg, data.categoryDaily);
6 nCategory = numel(cfg.categories);
7 nFuture = numel(cfg.futureDates);
8
9 quantityForecast = cell(nCategory, 1);
10 costForecast = cell(nCategory, 1);
11 demandResidual = zeros(cfg.backtestDays, nCategory);
12 costResidual = zeros(cfg.backtestDays, nCategory);
13 futureQuantity = zeros(nFuture, nCategory);
14 futureCost = zeros(nFuture, nCategory);
15 accuracyRows = cell(nCategory * 7, 1);
16 forecastRows = cell(nCategory * nFuture, 1);
17 accuracyCursor = 0;
18 forecastCursor = 0;
19
20 for k = 1:nCategory
21     quantityForecast{k} = forecast_one_series(cfg.allDates, quantity(:, k), ...
22         cfg.futureDates, "log1p", cfg.backtestDays);
23     costForecast{k} = forecast_one_series(cfg.allDates, cost(:, k), ...
24         cfg.futureDates, "log", cfg.backtestDays);
25     demandResidual(:, k) = quantityForecast{k}.residual;
26     costResidual(:, k) = costForecast{k}.residual;
27     futureQuantity(:, k) = quantityForecast{k}.future;
28     futureCost(:, k) = costForecast{k}.future;
29
30     modelNames = ["动态岭回归"; "Fourier日历回归"; "四周同星期均值"];
31     for j = 1:3
32         accuracyCursor = accuracyCursor + 1;
33         accuracyRows{accuracyCursor} = table(cfg.categories(k), "销量", modelNames(j), ...
34             quantityForecast{k}.errors(j), quantityForecast{k}.weights(j), ...
35             VariableNames={'分类名称', '对象', '模型', 'WAPE', '组合权重'});
36     end
37     accuracyCursor = accuracyCursor + 1;
38     ensembleWape = sum(abs(quantityForecast{k}.backtest.Actual ...
39         - quantityForecast{k}.backtest.Ensemble)) / sum(quantityForecast{k}.backtest.Actual);
40     accuracyRows{accuracyCursor} = table(cfg.categories(k), "销量", "误差倒数加权组合", ...
41         ensembleWape, 1, VariableNames={'分类名称', '对象', '模型', 'WAPE', '组合权重'});
42     for j = 1:3
43         accuracyCursor = accuracyCursor + 1;
44         accuracyRows{accuracyCursor} = table(cfg.categories(k), "批发成本", modelNames(j), ...

```

```

45         costForecast{k}.errors(j), costForecast{k}.weights(j), ...
46         VariableNames={'分类名称','对象','模型','WAPE','组合权重'});
47     end
48
49     demandQ = quantile(quantityForecast{k}.residual, [0.10, 0.90]);
50     costQ = quantile(costForecast{k}.residual, [0.10, 0.90]);
51     for d = 1:nFuture
52         forecastCursor = forecastCursor + 1;
53         q50 = futureQuantity(d, k);
54         c50 = futureCost(d, k);
55         forecastRows{forecastCursor} = table(cfg.futureDates(d), cfg.categories(k), ...
56             max(expm1(log1p(q50) + demandQ(1)), 0), q50, ...
57             max(expm1(log1p(q50) + demandQ(2)), 0), ...
58             c50 * exp(costQ(1)), c50, c50 * exp(costQ(2)), ...
59             VariableNames={'日期','分类名称','需求P10','需求P50','需求P90','成本P10','成本P50','成本P90'});
60     end
61 end
62
63 accuracyTable = vertcat(accuracyRows{1:accuracyCursor});
64 forecastTable = vertcat(forecastRows{:});
65 writetable(accuracyTable, fullfile(cfg.tableDir, 'q2_forecast_backtest.csv'), Encoding='UTF-8');
66 writetable(forecastTable, fullfile(cfg.tableDir, 'q2_category_forecasts.csv'), Encoding='UTF-8');
67
68 forecast = struct();
69 forecast.quantity = quantityForecast;
70 forecast.cost = costForecast;
71 forecast.futureQuantity = futureQuantity;
72 forecast.futureCost = futureCost;
73 forecast.demandResidual = demandResidual;
74 forecast.costResidual = costResidual;
75 forecast.accuracyTable = accuracyTable;
76 forecast.forecastTable = forecastTable;
77
78 [markupBounds, markupGrids, markupMedian] = markup_bounds(cfg, data.categoryDaily);
79 writetable(markupBounds, fullfile(cfg.tableDir, 'q2_markup_bounds.csv'), Encoding='UTF-8');
80
81 lossRate = zeros(nCategory, 1);
82 for k = 1:nCategory
83     lossRate(k) = data.lossCategory.LossRate(data.lossCategory.Category == cfg.categories(k));
84 end
85 elasticity = q1.elasticity;
86 nScenario = cfg.scenarioCount;
87 decisionRows = cell(nFuture * nCategory, 1);
88 scenarioRows = cell(nFuture, 1);
89 scenarioStore = cell(nFuture, 1);
90 cursor = 0;
91
92 for d = 1:nFuture
93     sampled = randi(size(demandResidual, 1), nScenario, 1);
94     demandScenario = max(expm1(log1p(futureQuantity(d, :)) + demandResidual(sampled, :)), 0);
95     costScenario = max(futureCost(d, :) .* exp(costResidual(sampled, :)), 0.05);
96     markup = markupMedian(:)';
97     order = futureQuantity(d, :) ./ (1 - lossRate);
98     referencePrice = futureCost(d, :) ./ (1 - lossRate) .* (1 + markupMedian(:)');
99
100     for iteration = 1:4
101         for k = 1:nCategory
102             bestProfit = -Inf;
103             bestMarkup = markup(k);
104             bestOrder = order(k);
105             for candidateMarkup = markupGrids{k}
106                 candidate = markup;
107                 candidate(k) = candidateMarkup;

```

```

108     prices = costScenario ./ (1 - lossRate') .* (1 + candidate);
109     ratios = min(max(prices ./ referencePrice, 0.35), 3.0);
110     logAdjustment = log(ratios) * elasticity(k, :);
111     adjustedDemand = min(max(demandScenario(:, k) .* exp(logAdjustment), 0), ...
112         futureQuantity(d, k) * 8);
113     orderCandidates = unique(quantile(adjustedDemand, linspace(0.45, 0.98, 12))) ...
114         / (1 - lossRate(k));
115     for candidateOrder = orderCandidates(:)
116         sold = min(adjustedDemand, (1 - lossRate(k)) * candidateOrder);
117         profit = prices(:, k) .* sold - costScenario(:, k) * candidateOrder;
118         expected = mean(profit);
119         if expected > bestProfit
120             bestProfit = expected;
121             bestMarkup = candidateMarkup;
122             bestOrder = candidateOrder;
123         end
124     end
125 end
126 markup(k) = bestMarkup;
127 order(k) = bestOrder;
128 end
129 end
130
131 prices = costScenario ./ (1 - lossRate') .* (1 + markup);
132 ratios = min(max(prices ./ referencePrice, 0.35), 3.0);
133 adjustedDemand = zeros(size(demandScenario));
134 for k = 1:nCategory
135     adjustedDemand(:, k) = min(max(demandScenario(:, k) .* ...
136         exp(log(ratios) * elasticity(k, :)), 0), futureQuantity(d, k) * 8);
137 end
138 sold = min(adjustedDemand, (1 - lossRate') .* order);
139 profit = prices .* sold - costScenario .* order;
140 dailyProfit = sum(profit, 2);
141 scenarioStore{d} = struct('demand', adjustedDemand, 'cost', costScenario, ...
142     'price', prices, 'sold', sold, 'profit', profit, 'order', order, 'markup', markup);
143
144 for k = 1:nCategory
145     cursor = cursor + 1;
146     sortedProfit = sort(profit(:, k));
147     tailCount = max(1, floor(nScenario / 10));
148     decisionRows{cursor} = table(cfg.futureDates(d), cfg.categories(k), ...
149         futureCost(d, k), quantile(costScenario(:, k), 0.10), ...
150         quantile(costScenario(:, k), 0.90), lossRate(k), markup(k), ...
151         round(futureCost(d, k) / (1 - lossRate(k)) * (1 + markup(k)), 1), ...
152         round(order(k), 1), mean(sold(:, k)), mean(profit(:, k)), ...
153         sum(sold(:, k)) / max(sum(adjustedDemand(:, k)), eps), ...
154         mean(adjustedDemand(:, k) > (1 - lossRate(k)) * order(k)), ...
155         mean(sortedProfit(1:tailCount)), ...
156         VariableNames={'日期','分类名称','预测批发成本P50','成本P10','成本P90','损耗率', ...
157             '最优有效加成率','参考售价','补货量','期望销量','期望利润','需求满足率', ...
158             '缺货概率','最差10_情景平均利润'});
159 end
160 sortedDaily = sort(dailyProfit);
161 tailCount = max(1, floor(nScenario / 10));
162 scenarioRows{d} = table(cfg.futureDates(d), mean(dailyProfit), ...
163     quantile(dailyProfit, 0.10), median(dailyProfit), quantile(dailyProfit, 0.90), ...
164     mean(sortedDaily(1:tailCount)), VariableNames={'日期','期望总利润','利润P10','利润P50', ...
165         '利润P90','最差10_情景平均利润'});
166 end
167
168 decisions = vertcat(decisionRows{:});
169 scenarioSummary = vertcat(scenarioRows{:});
170 writetable(decisions, fullfile(cfg.tableDir, 'q2_optimal_replenishment_pricing.csv'), Encoding='UTF
    -8');

```

```

171 writetable(scenarioSummary, fullfile(cfg.tableDir, 'q2_profit_scenarios.csv'), Encoding='UTF-8');
172
173 policyBacktest = policy_backtest(cfg, data, forecast, decisions, elasticity);
174 writetable(policyBacktest, fullfile(cfg.tableDir, 'q2_policy_backtest.csv'), Encoding='UTF-8');
175 sensitivity = parameter_sensitivity(cfg, scenarioStore, lossRate, elasticity);
176 writetable(sensitivity, fullfile(cfg.tableDir, 'q2_parameter_sensitivity.csv'), Encoding='UTF-8');
177 make_q2_figures(cfg, forecastTable, decisions, policyBacktest);
178
179 q2 = struct();
180 q2.decisions = decisions;
181 q2.scenarioSummary = scenarioSummary;
182 q2.markupBounds = markupBounds;
183 q2.markupGrids = markupGrids;
184 q2.markupMedian = markupMedian;
185 q2.scenarioStore = scenarioStore;
186 q2.policyBacktest = policyBacktest;
187 q2.sensitivity = sensitivity;
188 fprintf('Question 2 complete: weekly order %.1f kg, expected profit %.2f yuan.\n', ...
189     sum(decisions.('补货量')), sum(decisions.('期望利润')));
190 end
191
192 function [quantity, cost] = category_series(cfg, categoryDaily)
193 quantity = zeros(numel(cfg.allDates), numel(cfg.categories));
194 cost = zeros(size(quantity));
195 for k = 1:numel(cfg.categories)
196     subset = sortrows(categoryDaily(categoryDaily.Category == cfg.categories(k), :), 'Date');
197     quantity(:, k) = subset.PositiveQty;
198     cost(:, k) = fillmissing(subset.FixedCost, 'linear', EndValues='nearest');
199 end
200 end
201
202 function result = forecast_one_series(dates, rawSeries, futureDates, mode, backtestDays)
203 rawSeries = double(rawSeries(:));
204 if mode == "log"
205     rawSeries = fillmissing(rawSeries, 'linear', EndValues='nearest');
206 end
207 transformed = transform_series(rawSeries, mode);
208 origin = dates(1);
209 features = lag_features(dates, transformed, origin);
210 valid = all(isfinite(features), 2);
211 splitIndex = numel(dates) - backtestDays + 1;
212 train = valid & (1:numel(dates))' < splitIndex;
213 test = valid & (1:numel(dates))' >= splitIndex;
214
215 modelAR = ridge_gcv(features(train, :), transformed(train), logspace(-4, 3, 30));
216 predAR = inverse_transform(modelAR.predict(features(test, :)), mode);
217 [calendar, ~] = calendar_features(dates, origin);
218 modelCalendar = ridge_gcv(calendar((1:numel(dates))' < splitIndex, :), ...
219     transformed((1:numel(dates))' < splitIndex), logspace(-4, 3, 30));
220 predCalendar = inverse_transform(modelCalendar.predict(calendar(test, :)), mode);
221
222 testIndices = find(test);
223 predSameWeekday = zeros(numel(testIndices), 1);
224 for j = 1:numel(testIndices)
225     idx = testIndices(j);
226     historyIndex = idx - 7 * (1:4);
227     historyIndex = historyIndex(historyIndex >= 1);
228     predSameWeekday(j) = mean(rawSeries(historyIndex), 'omitnan');
229 end
230 actual = rawSeries(test);
231 wape = @(prediction) sum(abs(actual - prediction)) / max(sum(abs(actual)), eps);
232 errors = [wape(predAR); wape(predCalendar); wape(predSameWeekday)];
233 inverseError = 1 ./ max(errors, 0.02);
234 weights = inverseError / sum(inverseError);

```

```

235 ensemble = weights(1) * predAR + weights(2) * predCalendar + weights(3) * predSameWeekday;
236 backtest = table(dates(test), actual, predAR, predCalendar, predSameWeekday, ensemble, ...
237     VariableNames={'Date', 'Actual', 'DynamicRidge', 'FourierCalendar', 'FourWeekSameDay', 'Ensemble'});
238 residual = transform_series(actual, mode) - transform_series(ensemble, mode);
239
240 modelARAll = ridge_gcv(features(valid, :), transformed(valid), logspace(-4, 3, 30));
241 modelCalendarAll = ridge_gcv(calendar, transformed, logspace(-4, 3, 30));
242 n = numel(dates);
243 historyTransformed = [transformed; nan(numel(futureDates), 1)];
244 historyRaw = [rawSeries; nan(numel(futureDates), 1)];
245 future = zeros(numel(futureDates), 1);
246 for s = 1:numel(futureDates)
247     idx = n + s;
248     [calendarRow, ~] = calendar_features(futureDates(s), origin);
249     lagSet = [1 2 7 14 21 28];
250     lagValues = historyTransformed(idx - lagSet)';
251     row = [calendarRow, lagValues, mean(historyTransformed(idx - (1:7))), ...
252         mean(historyTransformed(idx - (1:28)))];
253     arValue = inverse_transform(modelARAll.predict(row), mode);
254     calendarValue = inverse_transform(modelCalendarAll.predict(calendarRow), mode);
255     sameIndices = idx - 7 * (1:4);
256     sameValue = mean(historyRaw(sameIndices), 'omitnan');
257     future(s) = max(weights(1) * arValue + weights(2) * calendarValue ...
258         + weights(3) * sameValue, ternary(mode == "log", 1e-4, 0));
259     historyRaw(idx) = future(s);
260     historyTransformed(idx) = transform_series(future(s), mode);
261 end
262 result = struct('future', future, 'backtest', backtest, 'residual', residual, ...
263     'errors', errors, 'weights', weights);
264 end
265
266 function features = lag_features(dates, transformed, origin)
267 [features, ~] = calendar_features(dates, origin);
268 n = numel(transformed);
269 for lag = [1 2 7 14 21 28]
270     column = nan(n, 1);
271     column(lag + 1:end) = transformed(1:end - lag);
272     features(:, end + 1) = column; %#ok<AGROW>
273 end
274 roll7 = nan(n, 1);
275 roll28 = nan(n, 1);
276 for idx = 29:n
277     roll7(idx) = mean(transformed(idx - (1:7)));
278     roll28(idx) = mean(transformed(idx - (1:28)));
279 end
280 features = [features, roll7, roll28];
281 end
282
283 function values = transform_series(values, mode)
284 if mode == "log1p"
285     values = log1p(max(double(values), 0));
286 else
287     values = log(max(double(values), 1e-4));
288 end
289 end
290
291 function values = inverse_transform(values, mode)
292 if mode == "log1p"
293     values = max(expm1(double(values)), 0);
294 else
295     values = max(exp(double(values)), 1e-4);
296 end
297 end
298

```

```

299 function [boundsTable, grids, medians] = markup_bounds(cfg, categoryDaily)
300 rows = cell(numel(cfg.categories), 1);
301 grids = cell(numel(cfg.categories), 1);
302 medians = zeros(numel(cfg.categories), 1);
303 for k = 1:numel(cfg.categories)
304     mask = categoryDaily.Category == cfg.categories(k) & categoryDaily.Date >= datetime(2022, 7, 1);
305     values = categoryDaily.EffectiveMarkup(mask);
306     values = values(isfinite(values) & values > 0.02 & values < 3);
307     q = quantile(values, [0.05 0.50 0.95]);
308     low = max(q(1), 0.08);
309     high = min(q(3), 1.80);
310     if high <= low + 0.05
311         high = low + 0.35;
312     end
313     grids{k} = linspace(low, high, 17);
314     medians{k} = min(max(q(2), low), high);
315     rows{k} = table(cfg.categories(k), q(1), q(2), q(3), low, high, ...
316         VariableNames={'分类名称', '历史P05', '历史中位数', '历史P95', '决策下界', '决策上界'});
317 end
318 boundsTable = vertcat(rows{:});
319 end
320
321 function result = policy_backtest(cfg, data, forecast, decisions, elasticity)
322 rows = cell(numel(cfg.categories), 1);
323 for k = 1:numel(cfg.categories)
324     back = forecast.quantity{k}.backtest;
325     residualCV = std(back.Actual - back.Ensemble) / max(mean(back.Actual), eps);
326     residualCV = min(max(residualCV, 0.08), 0.80);
327     categoryRows = decisions("分类名称") == cfg.categories(k);
328     modelMarkup = mean(decisions("最优有效加成率")(categoryRows));
329     categoryDaily = sortrows(data.categoryDaily(data.categoryDaily.Category == cfg.categories(k), :),
330         'Date');
331     historicalMarkup = categoryDaily.EffectiveMarkup;
332     historicalMarkup = historicalMarkup(isfinite(historicalMarkup) & historicalMarkup > 0.02 &
333         historicalMarkup < 3);
334     baselineMarkup = median(historicalMarkup);
335     eta = -elasticity(k, k);
336     modelProfit = 0; baselineProfit = 0; modelSold = 0; baselineSold = 0; demandTotal = 0;
337     for r = 1:height(back)
338         idx = find(categoryDaily.Date == back.Date(r), 1);
339         if isempty(idx), continue; end
340         actual = back.Actual(r);
341         c = categoryDaily.FixedCost(idx);
342         historicalPrice = categoryDaily.FixedPrice(idx);
343         loss = categoryDaily.LossRate(idx);
344         modelOrder = back.Ensemble(r) * (1 + 0.45 * residualCV) / (1 - loss);
345         previous = categoryDaily.PositiveQty(max(1, idx - 7):idx - 1);
346         baselineOrder = mean(previous) / (1 - loss);
347         modelPrice = c / (1 - loss) * (1 + modelMarkup);
348         baselinePrice = c / (1 - loss) * (1 + baselineMarkup);
349         modelDemand = actual * (modelPrice / max(historicalPrice, 0.1))^(eta);
350         baselineDemand = actual * (baselinePrice / max(historicalPrice, 0.1))^(eta);
351         soldModel = min(modelDemand, (1 - loss) * modelOrder);
352         soldBaseline = min(baselineDemand, (1 - loss) * baselineOrder);
353         modelProfit = modelProfit + modelPrice * soldModel - c * modelOrder;
354         baselineProfit = baselineProfit + baselinePrice * soldBaseline - c * baselineOrder;
355         modelSold = modelSold + soldModel;
356         baselineSold = baselineSold + soldBaseline;
357         demandTotal = demandTotal + actual;
358     end
359     wape = sum(abs(back.Actual - back.Ensemble)) / sum(back.Actual);
360     rows{k} = table(cfg.categories(k), height(back), wape, modelProfit, baselineProfit, ...
361         (modelProfit - baselineProfit) / max(abs(baselineProfit), eps), ...
362         modelSold / max(demandTotal, eps), baselineSold / max(demandTotal, eps), ...

```

```

361     VariableNames={'分类名称','回测天数','组合预测WAPE','本文策略模拟利润','基准策略模拟利润',
362     ...
363     '利润提升率','本文策略服务率','基准策略服务率'});
364 end
365 result = vertcat(rows{:});
366 end
367 function result = parameter_sensitivity(cfg, scenarios, baseLoss, elasticity)
368 caseNames = ["基准"; "需求-10%"; "需求+10%"; "成本-10%"; "成本+10%"; ...
369     "损耗率-20%"; "损耗率+20%"; "加成率-10%"; "加成率+10%"];
370 factors = [1 1 1 1; .9 1 1 1; 1.1 1 1 1; 1 .9 1 1; 1 1.1 1 1; ...
371     1 1 .8 1; 1 1 1.2 1; 1 1 1 .9; 1 1 1 1.1];
372 rows = cell(numel(caseNames), 1);
373 for c = 1:numel(caseNames)
374     weekProfit = zeros(cfg.scenarioCount, 1);
375     soldTotal = 0; demandTotal = 0;
376     for d = 1:numel(scenarios)
377         s = scenarios{d};
378         loss = min(max(baseLoss * factors(c, 3), 0), 0.45);
379         newCost = s.cost * factors(c, 2);
380         newMarkup = max(s.markup * factors(c, 4), 0.02);
381         newPrice = newCost ./ (1 - loss') .* (1 + newMarkup);
382         ratio = min(max(newPrice ./ s.price, 0.4), 2.5);
383         newDemand = zeros(size(s.demand));
384         for k = 1:numel(cfg.categories)
385             newDemand(:, k) = s.demand(:, k) * factors(c, 1) .* exp(log(ratio) * elasticity(k, :));
386         end
387         sold = min(newDemand, (1 - loss') .* s.order);
388         weekProfit = weekProfit + sum(newPrice .* sold - newCost .* s.order, 2);
389         soldTotal = soldTotal + sum(sold, 'all');
390         demandTotal = demandTotal + sum(newDemand, 'all');
391     end
392     sortedProfit = sort(weekProfit);
393     tailCount = max(1, floor(numel(weekProfit) / 10));
394     rows{c} = table(caseNames(c), mean(weekProfit), NaN, soldTotal / max(demandTotal, eps), ...
395         mean(sortedProfit(1:tailCount)), VariableNames={'情景','期望周利润','相对基准变化', ...
396         '需求满足率','最差10_情景平均周利润'});
397 end
398 result = vertcat(rows{:});
399 base = result("期望周利润")(result("情景") == "基准");
400 result("相对基准变化") = result("期望周利润") / base - 1;
401 end
402
403 function make_q2_figures(cfg, forecastTable, decisions, policyBacktest)
404 f = figure(Visible='off', Color='w', Position=[100 100 1300 760]);
405 tiledlayout(2, 3, TileSpacing='compact', Padding='compact');
406 for k = 1:numel(cfg.categories)
407     nexttile;
408     subset = forecastTable(forecastTable("分类名称") == cfg.categories(k), :);
409     x = subset("日期");
410     fill([x; flipud(x)], [subset("需求P10"); flipud(subset("需求P90"))], ...
411         [0.75 0.86 0.92], EdgeColor='none'); hold on;
412     plot(x, subset("需求P50"), '-o', Color=[0.15 0.42 0.62], LineWidth=1.5);
413     title(cfg.categories(k)); ylabel('需求/kg'); grid on;
414 end
415 sgtitle('2023年7月1-7日品类需求概率预测');
416 exportgraphics(f, fullfile(cfg.figureDir, 'q2_demand_forecast.png'), Resolution=180);
417 close(f);
418
419 f = figure(Visible='off', Color='w', Position=[100 100 980 520]);
420 orderMatrix = zeros(numel(cfg.futureDates), numel(cfg.categories));
421 for d = 1:numel(cfg.futureDates)
422     for k = 1:numel(cfg.categories)
423         mask = decisions("日期") == cfg.futureDates(d) & decisions("分类名称") == cfg.categories(k)

```



```

    );
    orderMatrix(d, k) = decisions.("补货量")(mask);
end
end
bar(cfg.futureDates, orderMatrix, 'stacked'); ylabel('补货量/kg'); grid on;
title('未来一周品类补货结构'); legend(cfg.categories, Location='eastoutside');
exportgraphics(f, fullfile(cfg.figureDir, 'q2_replenishment_plan.png'), Resolution=180);
close(f);
431
f = figure(Visible='off', Color='w', Position=[100 100 900 480]);
432
bar(categorical(policyBacktest.("分类名称")), ...
433     [policyBacktest.("本文策略模拟利润"), policyBacktest.("基准策略模拟利润")]);
434
ylabel('84日模拟利润/元'); grid on; title('滚动回测策略利润对比');
435
legend({'本文策略', '基准策略'}, Location='best');
436
exportgraphics(f, fullfile(cfg.figureDir, 'q2_policy_backtest.png'), Resolution=180);
437
close(f);
438
end
439
440
function value = ternary(condition, yesValue, noValue)
441
if condition, value = yesValue; else, value = noValue; end
442
end
443

```

## 附录 I 问题三单品组合优化代码

```

1 function q3 = solve_question3(cfg, data, q1, forecast, q2)
2 %SOLVE_QUESTION3 Lexicographic assortment selection and item decisions.
3
4 fprintf('Solving question 3...\n');
5 [candidates, outsideMass, baseDemand] = prepare_candidates(cfg, data, forecast, q2);
6
7 sensitivityRows = cell(cfg.assortmentHigh - cfg.assortmentLow + 1, 1);
8 solverRows = cell(numel(sensitivityRows) + 2, 1);
9 cursor = 0;
10 for count = cfg.assortmentLow:cfg.assortmentHigh
11     [x, rho, status] = maximize_minimum_capture(cfg, candidates, outsideMass, count, count);
12     cursor = cursor + 1;
13     sensitivityRows{cursor} = table(count, rho, ~isempty(x), ...
14         VariableNames={'固定单品数', '最大最低品类捕获率', '可行'});
15     solverRows{cursor} = table("最大化最低品类捕获率", count, count, rho, status, ...
16         VariableNames={'阶段', '下界', '上界', '最低捕获率', '状态'});
17 end
18
19 [xStage1, rhoStar, statusStage1] = maximize_minimum_capture(cfg, candidates, ...
20     outsideMass, cfg.assortmentLow, cfg.assortmentHigh);
21 assert(~isempty(xStage1), 'Question 3 stage-one MILP is infeasible.');
```

```

22 cursor = cursor + 1;
23 solverRows{cursor} = table("最大化最低品类捕获率", cfg.assortmentLow, ...
24     cfg.assortmentHigh, rhoStar, statusStage1, ...
25     VariableNames={'阶段', '下界', '上界', '最低捕获率', '状态'});
26
27 rhoTarget = max(rhoStar - 0.01, 0);
28 [xStage2, statusStage2] = maximize_profit_at_service(cfg, candidates, outsideMass, rhoTarget);
29 if isempty(xStage2)
30     xStage2 = xStage1;
31 end
32 cursor = cursor + 1;
33 solverRows{cursor} = table("给定服务率后最大化利润代理", cfg.assortmentLow, ...
34     cfg.assortmentHigh, rhoTarget, statusStage2, ...
35     VariableNames={'阶段', '下界', '上界', '最低捕获率', '状态'});
36
37 selected = candidates(xStage2 >= 0.5, :);
38 [itemPlan, serviceSummary] = optimize_selected_items(cfg, selected, data, q1, q2, ...
39     outsideMass, baseDemand);

```



```

40 sensitivity = vertcat(sensitivityRows{:});
41 sensitivity.("与33种服务率差") = max(sensitivity.("最大最低品类捕获率")) ...
42     - sensitivity.("最大最低品类捕获率");
43 solverLog = vertcat(solverRows{1:cursor});
44
45 writetable(candidates, fullfile(cfg.tableDir, 'q3_candidate_items.csv'), Encoding='UTF-8');
46 writetable(itemPlan, fullfile(cfg.tableDir, 'q3_selected_item_plan.csv'), Encoding='UTF-8');
47 writetable(serviceSummary, fullfile(cfg.tableDir, 'q3_category_service_summary.csv'), Encoding='UTF
    -8');
48 writetable(sensitivity, fullfile(cfg.tableDir, 'q3_assortment_count_sensitivity.csv'), Encoding='UTF
    -8');
49 writetable(solverLog, fullfile(cfg.tableDir, 'q3_solver_status.csv'), Encoding='UTF-8');
50 make_q3_figures(cfg, itemPlan, serviceSummary, sensitivity);
51
52 q3 = struct('candidates', candidates, 'itemPlan', itemPlan, 'serviceSummary', serviceSummary, ...
53     'sensitivity', sensitivity, 'solverLog', solverLog, 'rhoStar', rhoStar, ...
54     'rhoTarget', rhoTarget, 'outsideMass', outsideMass, 'baseDemand', baseDemand);
55 fprintf('Question 3 complete: selected %d items, total order %.1f kg.\n', ...
56     height(itemPlan), sum(itemPlan.("补货量")));
57 end
58
59 function [candidates, outsideMass, baseDemand] = prepare_candidates(cfg, data, forecast, q2)
60 dailyItem = data.dailyItem;
61 period = dailyItem(dailyItem.Date >= datetime(2023, 6, 24) ...
62     & dailyItem.Date <= datetime(2023, 6, 30) & dailyItem.PositiveQty > 0, :);
63 codes = unique(period.Code, 'stable');
64 recent28 = dailyItem(dailyItem.Date >= datetime(2023, 6, 3) ...
65     & dailyItem.Date <= datetime(2023, 6, 30) & ismember(dailyItem.Code, codes), :);
66
67 [G, code] = findgroups(recent28.Code);
68 total28 = splitapply(@sum, recent28.PositiveQty, G);
69 days28 = splitapply(@(x) sum(x > 0), recent28.PositiveQty, G);
70 revenue28 = splitapply(@sum, recent28.PositiveRevenue, G);
71 candidates = table(code, total28, days28, revenue28, ...
72     VariableNames={'Code', 'Sales28', 'SaleDays28', 'Revenue28'});
73 [found, location] = ismember(candidates.Code, data.items.Code);
74 assert(all(found), 'Candidate item code not found in item dictionary.');
```

```

75 candidates.ItemName = data.items.ItemName(location);
76 candidates.Category = data.items.Category(location);
77 candidates.RecentPrice = candidates.Revenue28 ./ candidates.Sales28;
78
79 [G7, code7] = findgroups(period.Code);
80 sales7 = splitapply(@sum, period.PositiveQty, G7);
81 days7 = splitapply(@(x) sum(x > 0), period.PositiveQty, G7);
82 stats7 = table(code7, sales7, days7, VariableNames={'Code', 'Sales7', 'SaleDays7'});
83 candidates = outerjoin(candidates, stats7, Keys='Code', MergeKeys=true, Type='left');
```

```

84
85 candidates.ForecastCost = nan(height(candidates), 1);
86 candidates.LossRate = nan(height(candidates), 1);
87 for i = 1:height(candidates)
88     wholesaleRows = data.wholesale(data.wholesale.Code == candidates.Code(i) ...
89         & data.wholesale.Date <= datetime(2023, 6, 30), :);
90     if ~isempty(wholesaleRows)
91         [~, last] = max(wholesaleRows.Date);
92         candidates.ForecastCost(i) = wholesaleRows.WholesalePrice(last);
93     end
94     lossRow = data.lossItem(data.lossItem.Code == candidates.Code(i), :);
95     if ~isempty(lossRow)
96         candidates.LossRate(i) = lossRow.LossRate(1);
97     end
98 end
99
100 outsideShare = estimate_outside_share(cfg, data.categoryDaily);
101 outsideMass = zeros(numel(cfg.categories), 1);

```

```

102 baseDemand = forecast.futureQuantity(1, :);
103 for k = 1:numel(cfg.categories)
104     mask = candidates.Category == cfg.categories(k);
105     fallbackCost = forecast.futureCost(1, k);
106     fallbackLoss = data.lossCategory.LossRate(data.lossCategory.Category == cfg.categories(k));
107     candidates.ForecastCost(mask & ~isfinite(candidates.ForecastCost)) = fallbackCost;
108     candidates.LossRate(mask & ~isfinite(candidates.LossRate)) = fallbackLoss;
109     candidates.RecentPrice(mask & ~isfinite(candidates.RecentPrice)) = ...
110         candidates.ForecastCost(mask & ~isfinite(candidates.RecentPrice)) ...
111         ./ (1 - candidates.LossRate(mask & ~isfinite(candidates.RecentPrice))) * 1.5;
112     prior = max(sum(candidates.Sales28(mask)) * 0.5 / max(sum(mask), 1), 0.2);
113     candidates.PriorQty(mask) = prior;
114     candidates.UtilityRaw(mask) = candidates.Sales28(mask) + prior;
115     candidates.BaseUtility(mask) = candidates.UtilityRaw(mask) / sum(candidates.UtilityRaw(mask));
116     outsideMass(k) = outsideShare(k) / (1 - outsideShare(k)) * sum(candidates.BaseUtility(mask));
117 end
118
119 candidates.ReferenceMarkup = zeros(height(candidates), 1);
120 candidates.ReferencePrice = zeros(height(candidates), 1);
121 candidates.HistoricalMarkup = zeros(height(candidates), 1);
122 candidates.CandidateDemand = zeros(height(candidates), 1);
123 candidates.MinimumDisplayProfit = zeros(height(candidates), 1);
124 for k = 1:numel(cfg.categories)
125     mask = candidates.Category == cfg.categories(k);
126     markup = min(max(q2.markupMedian(k), 0.08), 1.8);
127     candidates.ReferenceMarkup(mask) = markup;
128     candidates.ReferencePrice(mask) = candidates.ForecastCost(mask) ./ (1 - candidates.LossRate(mask)) * (1 + markup);
129     candidates.HistoricalMarkup(mask) = candidates.RecentPrice(mask) .* (1 - candidates.LossRate(mask)) ...
130         ./ candidates.ForecastCost(mask) - 1;
131     candidates.CandidateDemand(mask) = baseDemand(k) * candidates.BaseUtility(mask) ...
132         / (outsideMass(k) + sum(candidates.BaseUtility(mask)));
133     candidates.MinimumDisplayProfit(mask) = candidates.ReferencePrice(mask) ...
134         .* min(candidates.CandidateDemand(mask), cfg.minimumDisplay * (1 - candidates.LossRate(mask))) ...
135         - candidates.ForecastCost(mask) * cfg.minimumDisplay;
136 end
137
138 candidates.Redundancy = item_redundancy(candidates, dailyItem);
139 candidates.ProfitProxy = candidates.MinimumDisplayProfit ...
140     + candidates.ReferencePrice .* candidates.CandidateDemand ...
141     - candidates.ForecastCost .* candidates.CandidateDemand ./ (1 - candidates.LossRate);
142 candidates.ProfitProxy = candidates.ProfitProxy ...
143     - 0.08 * abs(candidates.ProfitProxy) .* candidates.Redundancy;
144 end
145
146 function outsideShare = estimate_outside_share(cfg, categoryDaily)
147 outsideShare = zeros(numel(cfg.categories), 1);
148 for k = 1:numel(cfg.categories)
149     subset = sortrows(categoryDaily(categoryDaily.Category == cfg.categories(k), :), 'Date');
150     [X, ~] = calendar_features(subset.Date, subset.Date(1));
151     X = [X, log1p(subset.ActiveItems)];
152     model = ridge_gcv(X, log1p(subset.PositiveQty), logspace(-4, 3, 30));
153     outsideShare(k) = min(max(model.coef(end), 0.06), 0.30);
154 end
155 end
156
157 function redundancy = item_redundancy(candidates, dailyItem)
158 dates = (datetime(2023, 1, 2):caldays(1):datetime(2023, 6, 30))';
159 nItem = height(candidates);
160 subset = dailyItem(dailyItem.Date >= dates(1) & dailyItem.Date <= dates(end) ...
161     & ismember(dailyItem.Code, candidates.Code), :);
162 [dateFound, dateIndex] = ismember(subset.Date, dates);

```

```

163 [codeFound, codeIndex] = ismember(subset.Code, candidates.Code);
164 keep = dateFound & codeFound;
165 matrix = accumarray([dateIndex(keep), codeIndex(keep)], subset.PositiveQty(keep), ...
166     [numel(dates), nItem], @sum, 0);
167 correlation = corr(log1p(matrix), Rows='pairwise');
168 redundancy = zeros(nItem, 1);
169 for i = 1:nItem
170     same = find(candidates.Category == candidates.Category(i) & (1:nItem)' ~= i);
171     values = correlation(i, same);
172     values = sort(values(isfinite(values) & values > 0), 'descend');
173     redundancy(i) = mean(values(1:min(3, numel(values))), 'omitnan');
174     if ~isfinite(redundancy(i)), redundancy(i) = 0; end
175 end
176 end
177
178 function [x, rho, status] = maximize_minimum_capture(cfg, candidates, outsideMass, countLow,
    countHigh)
179 low = 0; high = 0.999; x = [];
180 status = "Infeasible";
181 options = optimoptions('intlinprog', Display='off', MaxTime=30, RelativeGapTolerance=1e-8);
182 for iteration = 1:28
183     mid = (low + high) / 2;
184     [A, b] = assortment_constraints(cfg, candidates, outsideMass, countLow, countHigh, mid);
185     [candidateX, ~, exitflag] = intlinprog(zeros(height(candidates), 1), ...
186         1:height(candidates), A, b, [], [], zeros(height(candidates), 1), ...
187         ones(height(candidates), 1), options);
188     if exitflag > 0
189         low = mid;
190         x = candidateX;
191         status = "Optimal";
192     else
193         high = mid;
194     end
195 end
196 rho = low;
197 end
198
199 function [x, status] = maximize_profit_at_service(cfg, candidates, outsideMass, target)
200 [A, b] = assortment_constraints(cfg, candidates, outsideMass, ...
201     cfg.assortmentLow, cfg.assortmentHigh, target);
202 options = optimoptions('intlinprog', Display='off', MaxTime=60, RelativeGapTolerance=1e-8);
203 [x, ~, exitflag] = intlinprog(-candidates.ProfitProxy, 1:height(candidates), ...
204     A, b, [], [], zeros(height(candidates), 1), ones(height(candidates), 1), options);
205 if exitflag > 0
206     status = "Optimal";
207 else
208     x = [];
209     status = "Infeasible";
210 end
211 end
212
213 function [A, b] = assortment_constraints(cfg, candidates, outsideMass, countLow, countHigh, target)
214 n = height(candidates);
215 A = [ones(1, n); -ones(1, n)];
216 b = [countHigh; -countLow];
217 multiplier = target / max(1 - target, eps);
218 for k = 1:numel(cfg.categories)
219     row = zeros(1, n);
220     mask = candidates.Category == cfg.categories(k);
221     row(mask) = -candidates.BaseUtility(mask);
222     A(end + 1, :) = row; %#ok<AGROW>
223     b(end + 1, 1) = -multiplier * outsideMass(k); %#ok<AGROW>
224 end
225 end

```

```

226
227 function [itemPlan, serviceSummary] = optimize_selected_items(cfg, selected, data, q1, q2,
    outsideMass, baseDemand)
228 eta = -diag(q1.elasticity);
229 selected.OptimalMarkup = selected.ReferenceMarkup;
230 selected.Price = selected.ReferencePrice;
231 simulations = cell(height(selected), 1);
232
233 for iteration = 1:3
234     selected.PriceUtility = zeros(height(selected), 1);
235     selected.MeanDemand = zeros(height(selected), 1);
236     for k = 1:numel(cfg.categories)
237         mask = selected.Category == cfg.categories(k);
238         selected.PriceUtility(mask) = selected.BaseUtility(mask) .* ...
239             min(max(selected.Price(mask) ./ selected.RecentPrice(mask), 0.4), 2.5).^(-eta(k));
240         denominator = sum(selected.PriceUtility(mask)) + outsideMass(k);
241         selected.MeanDemand(mask) = baseDemand(k) * selected.PriceUtility(mask) / denominator;
242     end
243
244     for i = 1:height(selected)
245         k = find(cfg.categories == selected.Category(i));
246         historyDates = (datetime(2023, 4, 2):caldays(1):datetime(2023, 6, 30))';
247         historyRows = data.dailyItem(data.dailyItem.Code == selected.Code(i) ...
248             & data.dailyItem.Date >= historyDates(1) & data.dailyItem.Date <= historyDates(end), :);
249         history = zeros(numel(historyDates), 1);
250         [found, location] = ismember(historyDates, historyRows.Date);
251         history(found) = historyRows.PositiveQty(location(found));
252         positive = history(history > 0);
253         positiveProbability = min(max(mean(history > 0) + 0.20, 0.50), 0.98);
254         if numel(positive) >= 4
255             cv = std(positive) / max(mean(positive), eps);
256         else
257             cv = 0.65;
258         end
259         cv = min(max(cv, 0.25), 1.30);
260         sigma = sqrt(log(1 + cv^2));
261         conditionalMean = selected.MeanDemand(i) / positiveProbability;
262         mu = log(max(conditionalMean, 1e-4)) - 0.5 * sigma^2;
263         demandBase = double(rand(cfg.itemScenarioCount, 1) < positiveProbability) ...
264             .* exp(mu + sigma * randn(cfg.itemScenarioCount, 1));
265         costScenario = selected.ForecastCost(i) * exp(-0.5 * 0.08^2 ...
266             + 0.08 * randn(cfg.itemScenarioCount, 1));
267         boundRow = q2.markupBounds(q2.markupBounds("分类名称") == selected.Category(i), :);
268         low = boundRow("决策下界"); high = boundRow("决策上界");
269         historicalMarkup = max(selected.HistoricalMarkup(i), 0.05);
270         itemLow = max(0.05, min(low, 0.85 * historicalMarkup));
271         itemHigh = min(high, max(itemLow + 0.05, 1.20 * historicalMarkup));
272         markupCandidates = linspace(itemLow, itemHigh, 15);
273         bestExpected = -Inf;
274         best = struct();
275         for markup = markupCandidates
276             priceScenario = costScenario / (1 - selected.LossRate(i)) * (1 + markup);
277             demand = demandBase .* min(max(priceScenario / max(selected.RecentPrice(i), 0.1), 0.35),
278                 3).^(-eta(k));
279             orderCandidates = max(cfg.minimumDisplay, ...
280                 unique(quantile(demand, linspace(0.50, 0.98, 11))) / (1 - selected.LossRate(i)));
281             for order = orderCandidates(:)'
282                 sold = min(demand, (1 - selected.LossRate(i)) * order);
283                 profit = priceScenario .* sold - costScenario * order;
284                 expected = mean(profit);
285                 if expected > bestExpected
286                     bestExpected = expected;
287                     best = struct('markup', markup, 'order', order, 'price', priceScenario, ...
288                         'demand', demand, 'sold', sold, 'profit', profit);

```

```

288         end
289     end
290 end
291 selected.OptimalMarkup(i) = best.markup;
292 selected.Price(i) = round(selected.ForecastCost(i) / (1 - selected.LossRate(i)) ...
293     * (1 + best.markup), 1);
294 simulations{i} = best;
295 end
296 end
297
298 rows = cell(height(selected), 1);
299 for i = 1:height(selected)
300     s = simulations{i};
301     rows{i} = table(selected.Code(i), selected.ItemName(i), selected.Category(i), ...
302         selected.ForecastCost(i), selected.LossRate(i), s.markup, selected.Price(i), ...
303         round(s.order, 1), mean(s.demand), mean(s.sold), mean(s.profit), ...
304         sum(s.sold) / max(sum(s.demand), eps), ...
305         mean(s.demand > (1 - selected.LossRate(i)) * s.order), ...
306         selected.Redundancy(i), selected.HistoricalMarkup(i), ...
307         VariableNames={'单品编码','单品名称','分类名称','预测成本','损耗率','最优有效加成率', ...
308             '建议售价','补货量','期望需求','期望销量','期望利润','需求满足率','缺货概率', ...
309             '网络重复度','历史有效加成率'});
310 end
311 itemPlan = vertcat(rows{:});
312 itemPlan = sortrows(itemPlan, {'分类名称','期望利润'}, {'ascend','descend'});
313
314 serviceRows = cell(numel(cfg.categories), 1);
315 for k = 1:numel(cfg.categories)
316     mask = itemPlan("分类名称") == cfg.categories(k);
317     serviceRows{k} = table(cfg.categories(k), sum(mask), baseDemand(k), ...
318         sum(itemPlan("期望销量")(mask)), sum(itemPlan("期望销量")(mask)) / max(baseDemand(k), eps), ...
319         sum(itemPlan("补货量")(mask)), sum(itemPlan("期望利润")(mask)), ...
320         VariableNames={'分类名称','选择单品数','品类基准需求','预计销量','综合需求满足率','补货量',
321             '期望利润'});
322 end
323 serviceSummary = vertcat(serviceRows{:});
324 end
325
326 function make_q3_figures(cfg, itemPlan, serviceSummary, sensitivity)
327 f = figure(Visible='off', Color='w', Position=[100 100 900 470]);
328 plot(sensitivity("固定单品数"), sensitivity("最大最低品类捕获率"), '-o', ...
329     LineWidth=1.8, MarkerFaceColor=[0.25 0.45 0.65]);
330 xlabel('固定单品数'); ylabel('最大最低品类捕获率'); grid on;
331 title('单品数量敏感性');
332 exportgraphics(f, fullfile(cfg.figureDir, 'q3_count_sensitivity.png'), Resolution=180);
333 close(f);
334
335 f = figure(Visible='off', Color='w', Position=[100 100 980 520]);
336 yyaxis left;
337 bar(categorical(serviceSummary("分类名称")), serviceSummary("补货量")); ylabel('补货量/kg');
338 yyaxis right;
339 plot(categorical(serviceSummary("分类名称")), serviceSummary("综合需求满足率"), ...
340     '-o', LineWidth=1.8); ylabel('需求满足率'); grid on;
341 title(sprintf('单品组合: %d个单品', height(itemPlan)));
342 exportgraphics(f, fullfile(cfg.figureDir, 'q3_assortment_service.png'), Resolution=180);
343 close(f);
344 end

```

## 附录 J 问题四数据采集优先级代码

```

1 function q4 = solve_question4(cfg)
2 %SOLVE_QUESTION4 Rank additional data by expected information value/cost.

```

```

3
4 dataName = [
5     "实时库存、订货量、到货量、售罄时刻"
6     "实际报损量、报损原因、打折时刻与折后销量"
7     "凌晨供应商报价、可供数量、等级、产地、履约率"
8     "货架面积、陈列位置、面数及补架记录"
9     "匿名订单号或购物篮编号"
10    "促销、优惠券、满减与广告曝光"
11    "客流量、会员结构与客群画像"
12    "竞争门店价格与促销"
13    "温度、降雨、湿度、节假日和社区活动"
14    "冷链温度、在架时长与质量图像"];
15 effect = [
16    "区分零需求、未上架与缺货；将销量右删失修正为潜在需求"
17    "动态估计损耗率与残值，优化日内折价和订购量"
18    "刻画成本与供应联合不确定性，校准两阶段定价"
19    "识别陈列效应并补充空间容量约束"
20    "可靠识别共同购买、互补和替代，校准Nested Logit"
21    "解除价格与促销混淆，提高价格弹性因果解释"
22    "将销量变化拆分为客流和购买率变化"
23    "估计渠道替代和竞争价格效应"
24    "改善短期需求概率预测并解释异常波动"
25    "建立品质衰减曲线和动态折价模型"];
26 question = ["问题1-3"; "问题2-3"; "问题2-3"; "问题1、3"; "问题1、3"; ...
27    "问题1-2"; "问题1-2"; "问题1-2"; "问题1-2"; "问题2-3"];
28 priority = ["最高"; "最高"; "最高"; "高"; "高"; "高"; "中高"; "中高"; "中"; "中高"];
29 benefit = [5; 5; 5; 4; 4; 4; 3; 3; 3; 4];
30 cost = [2; 3; 4; 3; 3; 3; 3; 2; 4];
31 q4 = table(dataName, effect, question, priority, benefit, cost, benefit ./ cost, ...
32    VariableNames={'拟采集数据','决策作用','对应问题','优先级','预期信息收益评分','采集成本评分','
33    信息价值成本比'});
34 q4 = sortrows(q4, {'信息价值成本比','预期信息收益评分'}, {'descend','descend'});
35 writetable(q4, fullfile(cfg.tableDir, 'q4_information_value_priorities.csv'), Encoding='UTF-8');
36 fprintf('Question 4 complete.\n');
end

```

## 附录 K 求解结果与约束检查代码

```

1 function validation = validate_results(cfg, data, q1, forecast, q2, q3, q4)
2 %VALIDATE_RESULTS Constraint, dimension, boundary and traceability checks.
3
4 checks = strings(0, 1);
5 passed = false(0, 1);
6 detail = strings(0, 1);
7 addCheck("附件读取", height(data.sales) == 878503 && height(data.items) == 251, ...
8     sprintf('sales=%d, items=%d', height(data.sales), height(data.items)));
9 addCheck("六品类完整", numel(unique(data.categoryDaily.Category)) == 6, ...
10    sprintf('categories=%d', numel(unique(data.categoryDaily.Category))));
11 addCheck("问题一网络有限", height(q1.edges) >= 1 && height(q1.edges) <= 15, ...
12    sprintf('edges=%d', height(q1.edges)));
13 addCheck("自身价格弹性为负", all(diag(q1.elasticity) <= -0.20), ...
14    sprintf('range=[%.3f, %.3f]', min(-diag(q1.elasticity)), max(-diag(q1.elasticity))));
15 addCheck("预测日期完整", size(forecast.futureQuantity, 1) == 7, ...
16    sprintf('days=%d', size(forecast.futureQuantity, 1)));
17 addCheck("问题二决策完整", height(q2.decisions) == 42, ...
18    sprintf('rows=%d', height(q2.decisions)));
19 addCheck("问题二补货非负", all(q2.decisions("补货量") >= 0), ...
20    sprintf('minimum=%.3f', min(q2.decisions("补货量"))));
21 selectedCount = height(q3.itemPlan);
22 addCheck("问题三单品数约束", selectedCount >= cfg.assortmentLow && selectedCount <= cfg.
    assortmentHigh, ...
23    sprintf('selected=%d', selectedCount));
24 addCheck("问题三最低陈列量", all(q3.itemPlan("补货量") >= cfg.minimumDisplay), ...

```



```

25     sprintf('minimum=%.3f', min(q3.itemPlan("补货量"))));
26 addCheck("问题三六品类覆盖", numel(unique(q3.itemPlan("分类名称"))) == 6, ...
27     sprintf('categories=%d', numel(unique(q3.itemPlan("分类名称")))));
28 addCheck("问题三求解状态", all(q3.solverLog("状态") == "Optimal"), ...
29     strjoin(unique(q3.solverLog("状态")), ','));
30 addCheck("问题四信息价值表", height(q4) == 10 && all(q4("信息价值成本比") > 0), ...
31     sprintf('rows=%d', height(q4)));
32
33 validationTable = table(checks, passed, detail, VariableNames={'检查项','通过','说明'});
34 writetable(validationTable, fullfile(cfg.tableDir, 'validation_checks.csv'), Encoding='UTF-8');
35 validation = struct('passed', all(passed), 'checks', validationTable, 'dataAudit', q1.dataAudit);
36 assert(validation.passed, 'One or more MATLAB validation checks failed.');
```

```

37
38 function addCheck(name, condition, message)
39     checks(end + 1, 1) = name;
40     passed(end + 1, 1) = logical(condition);
41     detail(end + 1, 1) = string(message);
42 end
43 end

```

## 附录 L 论文图片生成程序

运行本程序可依次重建模型结果图、评价与决策图以及逐问建模思路图，并将核对通过的二十一张图片同步为论文用图。

```

1 from __future__ import annotations
2
3 import shutil
4 from pathlib import Path
5 import sys
6
7 def find_project_root() -> Path:
8     for candidate in Path(__file__).resolve().parents:
9         if (candidate / "data" / "processed").is_dir() and (candidate / "outputs" / "tables").is_dir():
10             return candidate
11     raise FileNotFoundError("Cannot locate the project data and output tables.")
12
13
14 ROOT = find_project_root()
15 VENDOR = ROOT / "analysis" / "vendor"
16 if VENDOR.exists():
17     sys.path.insert(0, str(VENDOR))
18
19 import export_modeling_diagrams
20 import export_paper_visuals
21 import figure_core
22
23
24 OUTPUT_FIGURES = ROOT / "outputs" / "figures"
25 PAPER_FIGURES = ROOT / "paper" / "figures" if (ROOT / "paper" / "main.tex").is_file() else ROOT / "figures"
26
27 EXPECTED_FIGURES = [
28     "fig_overall_workflow.png",
29     "fig_q1_modeling_route.png",
30     "fig_q1_sales_distribution.png",
31     "fig_q1_weekday_heatmap.png",
32     "fig_q1_hurdle_gamma_summary.png",
33     "fig_q1_correlation_comparison.png",
34     "fig_q1_copula_network.png",
35     "fig_q2_modeling_route.png",
36     "fig_q2_forecast_model_comparison.png",

```

```

37     "fig_q2_demand_forecast.png",
38     "fig_q2_replenishment_plan.png",
39     "fig_q2_full_policy_matrix.png",
40     "fig_q2_elasticity_backtest.png",
41     "fig_q3_modeling_route.png",
42     "fig_q3_count_sensitivity.png",
43     "fig_q3_assortment_service.png",
44     "fig_q3_selected_item_plan.png",
45     "fig_q4_modeling_route.png",
46     "fig_daily_operation_timeline.png",
47     "fig_model_evidence_chain.png",
48     "fig_four_question_summary.png",
49 ]
50
51
52 def main() -> None:
53     OUTPUT_FIGURES.mkdir(parents=True, exist_ok=True)
54     PAPER_FIGURES.mkdir(parents=True, exist_ok=True)
55
56     figure_core.main()
57     export_paper_visuals.main()
58     export_modeling_diagrams.main()
59
60     missing = [name for name in EXPECTED_FIGURES if not (OUTPUT_FIGURES / name).is_file()]
61     if missing:
62         raise FileNotFoundError(f"Missing generated figures: {missing}")
63
64     for name in EXPECTED_FIGURES:
65         shutil.copy2(OUTPUT_FIGURES / name, PAPER_FIGURES / name)
66
67     paper_missing = [name for name in EXPECTED_FIGURES if not (PAPER_FIGURES / name).is_file()]
68     if paper_missing:
69         raise FileNotFoundError(f"Missing paper figures after synchronization: {paper_missing}")
70
71     print(f"Generated and synchronized {len(EXPECTED_FIGURES)} paper figures.")
72
73
74 if __name__ == "__main__":
75     main()

```

## 附录 M 基础分析与模型结果图绘制代码

```

1 from __future__ import annotations
2
3 from pathlib import Path
4 import sys
5
6
7 def find_project_root() -> Path:
8     for candidate in Path(__file__).resolve().parents:
9         if (candidate / "data" / "processed").is_dir() and (candidate / "outputs" / "tables").is_dir(
10             ):
11             return candidate
12     raise FileNotFoundError("Cannot locate the project data and output tables.")
13
14 ROOT = find_project_root()
15 VENDOR = ROOT / "analysis" / "vendor"
16 if VENDOR.exists():
17     sys.path.insert(0, str(VENDOR))
18
19 import matplotlib.pyplot as plt
20 import networkx as nx

```



```

21 import numpy as np
22 import pandas as pd
23 import seaborn as sns
24 from matplotlib import font_manager
25
26
27 CACHE = ROOT / "data" / "processed"
28 TABLES = ROOT / "outputs" / "tables"
29 FIGURES = ROOT / "outputs" / "figures"
30 FIGURES.mkdir(parents=True, exist_ok=True)
31
32 CATEGORIES = ["花叶类", "花菜类", "水生根茎类", "茄类", "辣椒类", "食用菌"]
33 CAT_COLORS = {
34     "花叶类": "#2A9D8F",
35     "花菜类": "#7CB518",
36     "水生根茎类": "#457B9D",
37     "茄类": "#6A4C93",
38     "辣椒类": "#E63946",
39     "食用菌": "#8D6E63",
40 }
41 PLOT_FONT = "Microsoft YaHei"
42
43
44 def configure_plotting() -> None:
45     global PLOT_FONT
46     candidates = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial Unicode MS"]
47     available = {font.name for font in font_manager.fontManager.ttflist}
48     PLOT_FONT = next((name for name in candidates if name in available), "DejaVu Sans")
49     plt.rcParams.update(
50         {
51             "font.sans-serif": [PLOT_FONT, "DejaVu Sans"],
52             "axes.unicode_minus": False,
53             "figure.dpi": 150,
54             "savefig.dpi": 220,
55             "axes.titleweight": "bold",
56             "axes.edgecolor": "#6B7280",
57             "grid.color": "#D1D5DB",
58             "grid.alpha": 0.45,
59         }
60     )
61     sns.set_theme(style="whitegrid", font=PLOT_FONT)
62
63
64 def read_table(name: str, *, index: str | None = None) -> pd.DataFrame:
65     frame = pd.read_csv(TABLES / f"{name}.csv")
66     if index is not None:
67         frame = frame.set_index(index)
68     return frame
69
70
71 def plot_question_one() -> None:
72     prepared = pd.read_pickle(CACHE / "prepared.pkl")
73     category_daily = prepared["category_daily"].copy()
74     category_daily["销售日期"] = pd.to_datetime(category_daily["销售日期"])
75     weekday = read_table("q1_weekday_indices")
76     raw = read_table("q1_raw_correlation", index="分类名称").loc[CATEGORIES, CATEGORIES]
77     residual_corr = read_table("q1_residual_correlation", index="分类名称").loc[CATEGORIES,
78                                     CATEGORIES]
79     partial_corr = read_table("q1_copula_partial_correlation", index="分类名称").loc[CATEGORIES,
80                                     CATEGORIES]
81     edge_table = read_table("q1_copula_network_edges")
82
83     fig, axes = plt.subplots(2, 1, figsize=(11, 8.2), constrained_layout=True)
84     recent = category_daily[category_daily["销售日期"] >= "2022-07-01"]

```

```

83 for category in CATEGORIES:
84     subset = recent[recent["分类名称"].eq(category)]
85     smoothed = subset.set_index("销售日期")["正向销量"].rolling(14, center=True).mean()
86     axes[0].plot(smoothed.index, smoothed.values, label=category, lw=1.5, color=CAT_COLORS[
category])
87 axes[0].set_title("六类蔬菜日销量的 14 日移动平均 (2022-07 至 2023-06)")
88 axes[0].set_ylabel("销量 (千克/日)")
89 axes[0].legend(ncol=3, frameon=False)
90 sns.boxplot(
91     data=category_daily,
92     x="分类名称",
93     y="正向销量",
94     order=CATEGORIES,
95     hue="分类名称",
96     palette=CAT_COLORS,
97     legend=False,
98     ax=axes[1],
99     showfliers=False,
100 )
101 axes[1].set_title("品类日销量分布 (隐藏极端离群点以便比较)")
102 axes[1].set_xlabel("")
103 axes[1].set_ylabel("销量 (千克/日)")
104 fig.savefig(FIGURES / "fig_q1_sales_distribution.png", bbox_inches="tight")
105 plt.close(fig)
106
107 heat = weekday.pivot(index="分类名称", columns="星期名称", values="相对指数").reindex(
108     index=CATEGORIES, columns=["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
109 )
110 fig, ax = plt.subplots(figsize=(10, 4.4), constrained_layout=True)
111 sns.heatmap(heat, annot=True, fmt=".2f", cmap="YlGnBu", center=1, ax=ax, cbar_kws={"label": "相
对日均销量"})
112 ax.set_title("周内销量效应热力图 (1.00 表示该品类周平均)")
113 ax.set_xlabel("")
114 ax.set_ylabel("")
115 fig.savefig(FIGURES / "fig_q1_weekday_heatmap.png", bbox_inches="tight")
116 plt.close(fig)
117
118 fig, axes = plt.subplots(1, 3, figsize=(15, 4.6), constrained_layout=True)
119 for ax, matrix, title in zip(
120     axes,
121     [raw, residual_corr, partial_corr],
122     ["原始 log 销量相关", "去趋势季节残差相关", "Copula 条件相关"],
123 ):
124     sns.heatmap(
125         matrix,
126         vmin=-1,
127         vmax=1,
128         center=0,
129         cmap="RdBu_r",
130         annot=True,
131         fmt=".2f",
132         square=True,
133         cbar=False,
134         ax=ax,
135     )
136     ax.set_title(title)
137     ax.set_xlabel("")
138     ax.set_ylabel("")
139     ax.tick_params(axis="x", rotation=45)
140     ax.tick_params(axis="y", rotation=0)
141 fig.savefig(FIGURES / "fig_q1_correlation_comparison.png", bbox_inches="tight")
142 plt.close(fig)
143
144 graph = nx.Graph()

```

```

145 graph.add_nodes_from(CATEGORIES)
146 for row in edge_table.itertuples(index=False):
147     graph.add_edge(row.品类1, row.品类2, weight=abs(row.条件相关), sign=np.sign(row.条件相关))
148 positions = nx.circular_layout(graph)
149 fig, ax = plt.subplots(figsize=(8, 6), constrained_layout=True)
150 nx.draw_networkx_nodes(
151     graph,
152     positions,
153     node_color=[CAT_COLORS[name] for name in graph.nodes],
154     node_size=1800,
155     edgecolors="white",
156     linewidths=1.5,
157     ax=ax,
158 )
159 nx.draw_networkx_labels(
160     graph,
161     positions,
162     font_size=11,
163     font_weight="bold",
164     font_family=PLOT_FONT,
165     ax=ax,
166 )
167 for left, right, attrs in graph.edges(data=True):
168     nx.draw_networkx_edges(
169         graph,
170         positions,
171         edgelist=[(left, right)],
172         width=1 + 7 * attrs["weight"],
173         edge_color="#D1495B" if attrs["sign"] < 0 else "#277DA1",
174         alpha=0.75,
175         ax=ax,
176     )
177     midpoint = (positions[left] + positions[right]) / 2
178     ax.text(midpoint[0], midpoint[1], f"{partial_corr.loc[left, right]:.2f}", fontsize=9, ha="
179 center", va="center")
180 ax.set_title("非正态 Copula-Graphical Lasso 条件需求网络\n蓝色为同向, 红色为反向, 线宽表示强度")
181 ax.axis("off")
182 fig.savefig(FIGURES / "fig_q1_copula_network.png", bbox_inches="tight")
183 plt.close(fig)
184
185 def plot_question_two() -> None:
186     forecast = read_table("q2_category_forecasts")
187     forecast["日期"] = pd.to_datetime(forecast["日期"])
188     decisions = read_table("q2_optimal_replenishment_pricing")
189     decisions["日期"] = pd.to_datetime(decisions["日期"])
190     policy_backtest = read_table("q2_policy_backtest")
191     elasticity = read_table("q1_cross_price_elasticities", index="需求品类").loc[CATEGORIES,
192 CATEGORIES]
193
194 fig, axes = plt.subplots(2, 3, figsize=(14, 8.2), constrained_layout=True)
195 for ax, category in zip(axes.flat, CATEGORIES):
196     subset = forecast[forecast["分类名称"].eq(category)]
197     x = subset["日期"].to_numpy()
198     ax.fill_between(x, subset["需求P10"].to_numpy(), subset["需求P90"].to_numpy(), alpha=0.22,
199 color=CAT_COLORS[category])
200     ax.plot(x, subset["需求P50"].to_numpy(), marker="o", color=CAT_COLORS[category], lw=2)
201     ax.set_title(category)
202     ax.set_ylabel("需求 (千克)")
203     ax.tick_params(axis="x", rotation=30)
204     fig.suptitle("2023 年 7 月 1-7 日品类需求概率预测 (P10-P90)", fontsize=15, fontweight="bold")
205     fig.savefig(FIGURES / "fig_q2_demand_forecast.png", bbox_inches="tight")
206     plt.close(fig)

```

```

205 fig, ax = plt.subplots(figsize=(10.5, 4.8), constrained_layout=True)
206 pivot = decisions.pivot(index="日期", columns="分类名称", values="补货量")[CATEGORIES]
207 bottom = np.zeros(len(pivot))
208 for category in CATEGORIES:
209     ax.bar(pivot.index, pivot[category], bottom=bottom, label=category, color=CAT_COLORS[
210         category])
211     bottom += pivot[category].to_numpy()
212 ax.set_title("未来一周最优补货结构")
213 ax.set_ylabel("补货量 (千克)")
214 ax.legend(ncol=3, frameon=False)
215 ax.tick_params(axis="x", rotation=25)
216 fig.savefig(FIGURES / "fig_q2_replenishment_plan.png", bbox_inches="tight")
217 plt.close(fig)
218
219 fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.5), constrained_layout=True)
220 sns.heatmap(
221     elasticity,
222     annot=True,
223     fmt=".2f",
224     cmap="RdBu_r",
225     center=0,
226     ax=axes[0],
227     cbar_kws={"label": "需求对价格的弹性"},
228 )
229 axes[0].set_title("工具变量-岭回归价格弹性矩阵")
230 axes[0].set_xlabel("价格品类")
231 axes[0].set_ylabel("需求品类")
232 plot_data = policy_backtest.set_index("分类名称")[["本文策略模拟利润", "基准策略模拟利润"]]
233 plot_data.plot(kind="bar", ax=axes[1], color=["#277DA1", "#A8A29E"])
234 axes[1].set_title("84 日滚动回测的策略利润")
235 axes[1].set_xlabel("")
236 axes[1].set_ylabel("模拟利润 (元)")
237 axes[1].tick_params(axis="x", rotation=30)
238 axes[1].legend(frameon=False)
239 fig.savefig(FIGURES / "fig_q2_elasticity_backtest.png", bbox_inches="tight")
240 plt.close(fig)
241
242
243 def plot_question_three() -> None:
244     item_plan = read_table("q3_selected_item_plan")
245     service = read_table("q3_category_service_summary")
246     sensitivity = read_table("q3_assortment_count_sensitivity")
247
248     fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.8), constrained_layout=True)
249     counts = item_plan["分类名称"].value_counts().reindex(CATEGORIES).fillna(0)
250     axes[0].bar(counts.index, counts.values, color=[CAT_COLORS[name] for name in counts.index])
251     axes[0].set_title("7 月 1 日入选单品的品类结构")
252     axes[0].set_ylabel("单品数")
253     axes[0].tick_params(axis="x", rotation=30)
254     axes[0].bar_label(axes[0].containers[0])
255     axes[1].bar(
256         service["分类名称"],
257         service["综合需求满足率"],
258         color=[CAT_COLORS[name] for name in service["分类名称"]],
259     )
260     axes[1].axhline(1.0, color="#374151", lw=1, ls="--")
261     axes[1].set_title("单品计划的品类需求满足率")
262     axes[1].set_ylabel("满足率")
263     axes[1].tick_params(axis="x", rotation=30)
264     fig.savefig(FIGURES / "fig_q3_assortment_service.png", bbox_inches="tight")
265     plt.close(fig)
266
267     fig, ax = plt.subplots(figsize=(8.5, 4.2), constrained_layout=True)

```

```

268     ax.plot(
269         sensitivity["固定单品数"],
270         sensitivity["最大最低品类捕获率"],
271         marker="o",
272         lw=2,
273         color="#277DA1",
274     )
275     ax.set_title("单品数量对最差品类需求捕获率的影响")
276     ax.set_xlabel("可售单品数")
277     ax.set_ylabel("最大最低品类捕获率")
278     ax.set_xticks(range(27, 34))
279     fig.savefig(FIGURES / "fig_q3_count_sensitivity.png", bbox_inches="tight")
280     plt.close(fig)
281
282
283 def plot_overall_workflow() -> None:
284     fig, ax = plt.subplots(figsize=(12, 3.8), constrained_layout=True)
285     ax.axis("off")
286     stages = [
287         (0.08, "多源数据清洗\n退货/打折/长编码"),
288         (0.25, "潜在需求修正\nHurdle-Gamma/删失"),
289         (0.42, "需求网络\nCopula + 价格弹性"),
290         (0.59, "概率预测\n组合预测 + 联合情景"),
291         (0.76, "两阶段决策\n成本响应定价 + 报童"),
292         (0.93, "单品组合\n选择模型 + 字典序"),
293     ]
294     for index, (x, label) in enumerate(stages):
295         ax.add_patch(
296             plt.Rectangle(
297                 (x - 0.07, 0.38),
298                 0.14,
299                 0.28,
300                 facecolor="#E8F1F5" if index % 2 == 0 else "#F3EDE2",
301                 edgecolor="#355070",
302                 linewidth=1.5,
303                 transform=ax.transAxes,
304             )
305         )
306         ax.text(x, 0.52, label, ha="center", va="center", fontsize=10.5, transform=ax.transAxes)
307         if index < len(stages) - 1:
308             ax.annotate(
309                 "",
310                 xy=(stages[index + 1][0] - 0.075, 0.52),
311                 xytext=(x + 0.075, 0.52),
312                 xycoords=ax.transAxes,
313                 arrowprops=dict(arrowstyle="->", color="#355070", lw=1.8),
314             )
315         ax.text(0.5, 0.82, "四个问题的计算顺序", ha="center", va="center", fontsize=15, fontweight="
316             bold", color="#1F3A5F", transform=ax.transAxes)
317         ax.text(0.5, 0.18, "问题1估计需求特征 → 问题2计算品类方案 → 问题3计算单品方案 → 问题4确定新增
318             数据", ha="center", va="center", fontsize=10.5, color="#4B5563", transform=ax.transAxes)
319         fig.savefig(FIGURES / "fig_overall_workflow.png", bbox_inches="tight")
320         plt.close(fig)
321
322 def main() -> None:
323     configure_plotting()
324     plot_question_one()
325     plot_question_two()
326     plot_question_three()
327     plot_overall_workflow()
328     print("Created 10 model and result figures from verified tables and processed data.")
329

```

```

330 if __name__ == "__main__":
331     main()

```

## 附录 N 预测、策略与验证图绘制代码

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5  import sys
6
7
8  def find_project_root() -> Path:
9      for candidate in Path(__file__).resolve().parents:
10         if (candidate / "outputs" / "tables").is_dir():
11             return candidate
12         raise FileNotFoundError("Cannot locate the project output tables.")
13
14
15 ROOT = find_project_root()
16 VENDOR = ROOT / "analysis" / "vendor"
17 if VENDOR.exists():
18     sys.path.insert(0, str(VENDOR))
19
20 import matplotlib.pyplot as plt
21 import numpy as np
22 import pandas as pd
23 import seaborn as sns
24 from matplotlib import font_manager
25 from matplotlib.colors import LinearSegmentedColormap
26 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
27
28
29 TABLES = ROOT / "outputs" / "tables"
30 FIGURES = ROOT / "outputs" / "figures"
31 FIGURES.mkdir(parents=True, exist_ok=True)
32
33 CATEGORIES = ["花叶类", "花菜类", "水生根茎类", "茄类", "辣椒类", "食用菌"]
34 COLORS = {
35     "花叶类": "#2A9D8F",
36     "花菜类": "#7CB518",
37     "水生根茎类": "#457B9D",
38     "茄类": "#6A4C93",
39     "辣椒类": "#E63946",
40     "食用菌": "#8D6E63",
41 }
42
43
44 def configure_plotting() -> None:
45     candidates = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial Unicode MS"]
46     available = {font.name for font in font_manager.fontManager.ttflist}
47     chosen = next((name for name in candidates if name in available), "DejaVu Sans")
48     plt.rcParams.update(
49         {
50             "font.sans-serif": [chosen, "DejaVu Sans"],
51             "axes.unicode_minus": False,
52             "figure.dpi": 150,
53             "savefig.dpi": 240,
54             "axes.edgecolor": "#6B7280",
55             "grid.color": "#D1D5DB",
56             "grid.alpha": 0.35,
57         }
58     )

```

```

59     sns.set_theme(style="whitegrid", font=chosen)
60
61
62 def save(fig: plt.Figure, name: str) -> None:
63     fig.savefig(FIGURES / name, bbox_inches="tight", facecolor="white")
64     plt.close(fig)
65
66
67 def plot_hurdle_summary() -> None:
68     data = pd.read_csv(TABLES / "q1_core_item_hurdle_gamma.csv")
69     data["正需求均值"] = data["Gamma形状"] * data["Gamma尺度"]
70     data = data.sort_values("需求发生概率", ascending=True).reset_index(drop=True)
71     y = np.arange(len(data))
72
73     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12.2, 7.1), sharey=True)
74     bar_colors = [COLORS.get(category, "#64748B") for category in data["分类名称"]]
75     ax1.barh(y, data["需求发生概率"] * 100, color=bar_colors, alpha=0.92)
76     ax1.set_yticks(y, data["单品名称"])
77     ax1.set_xlim(50, 102)
78     ax1.set_xlabel("需求发生概率 (%)")
79     ax1.set_title("需求发生概率", fontweight="bold")
80     for yi, value in zip(y, data["需求发生概率"] * 100):
81         ax1.text(value + 0.6, yi, f"{value:.1f}%", va="center", fontsize=8.2)
82
83     sizes = 42 + 8 * np.sqrt(data["疑似售罄日_18点"].clip(lower=0))
84     ax2.scatter(
85         data["正需求均值"],
86         y,
87         s=sizes,
88         c=bar_colors,
89         edgecolor="white",
90         linewidth=0.8,
91         alpha=0.94,
92     )
93     ax2.set_xlabel("Gamma 正需求均值 (kg)")
94     ax2.set_title("正需求规模与疑似售罄诊断", fontweight="bold")
95     for x, yi, soldout in zip(data["正需求均值"], y, data["疑似售罄日_18点"]):
96         ax2.text(x + 0.6, yi, f"{int(soldout)}日", va="center", fontsize=8.0)
97     ax2.tick_params(axis="y", labelleft=False)
98
99     handles = [plt.Line2D([0], [0], color=COLORS[c], lw=6, label=c) for c in CATEGORIES]
100    fig.legend(handles=handles, loc="lower center", ncol=6, frameon=False, bbox_to_anchor=(0.5, 0.045))
101    fig.text(0.5, 0.018, "圆点大小同时反映疑似售罄日数；完整参数保存在 q1_core_item_hurdle_gamma.csv", ha="center", fontsize=9)
102    fig.tight_layout(rect=(0, 0.10, 1, 1))
103    save(fig, "fig_q1_hurdle_gamma_summary.png")
104
105
106 def _forecast_panel(ax: plt.Axes, data: pd.DataFrame, target: str) -> None:
107     subset = data[data["对象"] == target].copy()
108     models = ["动态岭回归", "Fourier日历回归", "四周同星期均值"]
109     if target == "销量":
110         models.append("误差倒数加权组合")
111     matrix = subset.pivot(index="分类名称", columns="模型", values="WAPE").reindex(index=CATEGORIES, columns=models)
112     weights = subset.pivot(index="分类名称", columns="模型", values="组合权重").reindex(index=CATEGORIES, columns=models)
113     annotation = matrix.copy().astype(object)
114     for row in CATEGORIES:
115         for model in models:
116             value = matrix.loc[row, model]
117             weight = weights.loc[row, model]
118             if pd.isna(value):

```

```

119         annotation.loc[row, model] = ""
120     elif model == "误差倒数加权组合":
121         annotation.loc[row, model] = f"{value:.1%}"
122     else:
123         annotation.loc[row, model] = f"{value:.1%}\nw={weight:.3f}"
124 sns.heatmap(
125     matrix * 100,
126     annot=annotation,
127     fmt="",
128     cmap="YlGnBu_r",
129     linewidths=0.7,
130     linecolor="white",
131     cbar_kws={"label": "WAPE (%) ", "shrink": 0.80},
132     ax=ax,
133 )
134 ax.set_title(f"{target}候选模型：误差与组合权重", fontweight="bold")
135 ax.set_xlabel("")
136 ax.set_ylabel("")
137 ax.tick_params(axis="x", rotation=18)
138 ax.tick_params(axis="y", rotation=0)
139
140
141 def plot_forecast_comparison() -> None:
142     data = pd.read_csv(TABLES / "q2_forecast_backtest.csv")
143     fig, axes = plt.subplots(1, 2, figsize=(14.2, 5.4))
144     _forecast_panel(axes[0], data, "销量")
145     _forecast_panel(axes[1], data, "批发成本")
146     fig.text(0.5, 0.012, "单元格首行为 WAPE；括号外的 w 为误差倒数归一化权重，组合结果只报告 WAPE",
147            ha="center", fontsize=9.2)
147     fig.tight_layout(rect=(0, 0.055, 1, 1), w_pad=1.8)
148     save(fig, "fig_q2_forecast_model_comparison.png")
149
150
151 def _policy_heatmap(ax: plt.Axes, matrix: pd.DataFrame, title: str, fmt: str, cmap: str, cbar_label: str) -> None:
152     sns.heatmap(
153         matrix,
154         annot=True,
155         fmt=fmt,
156         cmap=cmap,
157         linewidths=0.65,
158         linecolor="white",
159         cbar_kws={"label": cbar_label, "shrink": 0.76},
160         ax=ax,
161     )
162     ax.set_title(title, fontweight="bold")
163     ax.set_xlabel("日期")
164     ax.set_ylabel("")
165     ax.tick_params(axis="x", rotation=0)
166     ax.tick_params(axis="y", rotation=0)
167
168
169 def plot_q2_policy_matrix() -> None:
170     data = pd.read_csv(TABLES / "q2_optimal_replenishment_pricing.csv")
171     data["日期"] = pd.to_datetime(data["日期"]).dt.strftime("%07-%d")
172     dates = list(dict.fromkeys(data["日期"]))
173     metrics = [
174         ("补货量", "补货量 (kg)", ".1f", "YlGnBu", "kg"),
175         ("参考售价", "参考售价 (元/kg)", ".1f", "YlOrRd", "元/kg"),
176         ("需求满足率", "需求满足率", ".1%", "BuGn", "比例"),
177         ("期望利润", "情景期望利润 (元)", ".0f", "PuBu", "元"),
178     ]
179     fig, axes = plt.subplots(2, 2, figsize=(13.4, 8.6))
180     for ax, (column, title, fmt, cmap, label) in zip(axes.flat, metrics):

```



```

181     matrix = data.pivot(index="分类名称", columns="日期", values=column).reindex(index=
    CATEGORIES, columns=dates)
182     _policy_heatmap(ax, matrix, title, fmt, cmap, label)
183     fig.text(0.5, 0.012, "四幅矩阵共同覆盖 7 日 × 6 品类的 42 条决策；实际售价需在到货成本揭示后按
    成本响应规则更新。", ha="center", fontsize=9.2)
184     fig.tight_layout(rect=(0, 0.035, 1, 1), h_pad=1.8, w_pad=1.2)
185     save(fig, "fig_q2_full_policy_matrix.png")
186
187
188 def plot_q3_item_plan() -> None:
189     data = pd.read_csv(TABLES / "q3_selected_item_plan.csv")
190     fig, axes = plt.subplots(3, 2, figsize=(12.8, 11.0), sharex=True)
191     axes = axes.flat
192     max_order = float(data["补货量"].max()) * 1.40
193     for ax, category in zip(axes, CATEGORIES):
194         subset = data[data["分类名称"] == category].sort_values("补货量", ascending=True).
            reset_index(drop=True)
195         y = np.arange(len(subset))
196         bar_colors = np.where(subset["期望利润"] >= 0, COLORS[category], "#D9776A")
197         ax.barh(y, subset["补货量"], color=bar_colors, alpha=0.92)
198         ax.set_yticks(y, subset["单品名称"])
199         ax.set_xlim(0, max_order)
200         ax.set_title(f"{category} ({len(subset)}种)", fontweight="bold", color=COLORS[category])
201         ax.set_xlabel("补货量 (kg)")
202         for yi, (order, price, profit) in enumerate(zip(subset["补货量"], subset["建议售价"], subset
            ["期望利润"])):
203             ax.text(order + 0.35, yi, f"价{price:.1f} / 利{profit:+.1f}", va="center", fontsize=7.8)
204             positive = plt.Line2D([0], [0], color="#2A9D8F", lw=7, label="期望利润非负")
205             negative = plt.Line2D([0], [0], color="#D9776A", lw=7, label="期望利润为负")
206             fig.legend(handles=[positive, negative], loc="lower center", ncol=2, frameon=False,
                bbox_to_anchor=(0.5, 0.005))
207             fig.text(0.5, 0.035, "条长为补货量；条末标注“建议售价 / 期望利润”。负利润单品由服务率约束保
                留。", ha="center", fontsize=9.2)
208             fig.tight_layout(rect=(0, 0.065, 1, 1), h_pad=1.45, w_pad=2.2)
209             save(fig, "fig_q3_selected_item_plan.png")
210
211
212 def plot_evidence_chain() -> None:
213     fig, ax = plt.subplots(figsize=(13.6, 5.8))
214     ax.set_xlim(0, 1)
215     ax.set_ylim(0, 1)
216     ax.axis("off")
217     cards = [
218         (
219             0.025,
220             "㉑ 输入审计",
221             "4份附件哈希一致\n销售、退货与折价分离\n编码和单位统一",
222             "#E8F5F2",
223             "#2A9D8F",
224         ),
225         (
226             0.275,
227             "㉒ 时序回溯",
228             "84日滚动窗口\n销量WAPE 17.4% - 32.1%\n不使用未来信息",
229             "#EDF4F8",
230             "#457B9D",
231         ),
232         (
233             0.525,
234             "㉓ 约束检查",
235             "27 - 33种MILP逐项求解\n九组候选模型均达最优\n数量与服务约束复核",
236             "#F2EEF8",
237             "#6A4C93",
238         ),

```

```

239     (
240         0.775,
241         "ⓐ 门店试运行",
242         "实际成本触发调价\n库存、售罄与报损回流\n漂移后重新估计模型",
243         "#FFF1EE",
244         "#E76F51",
245     ),
246 ]
247 for x, title, body, fill, edge in cards:
248     card = FancyBboxPatch(
249         (x, 0.24),
250         0.20,
251         0.56,
252         boxstyle="round,pad=0.015,rounding_size=0.025",
253         facecolor=fill,
254         edgecolor=edge,
255         linewidth=2.1,
256     )
257     ax.add_patch(card)
258     ax.text(x + 0.10, 0.69, title, ha="center", va="center", fontsize=13.2, fontweight="bold",
259           color=edge)
260     ax.text(x + 0.10, 0.48, body, ha="center", va="center", fontsize=10.2, linespacing=1.65,
261           color="#1F2937")
262 for x in (0.235, 0.485, 0.735):
263     arrow = FancyArrowPatch((x, 0.52), (x + 0.028, 0.52), arrowstyle="-|>", mutation_scale=18,
264                             color="#64748B", linewidth=1.8)
265     ax.add_patch(arrow)
266 ax.text(0.50, 0.90, "模型验证项目与部署条件", ha="center", va="center", fontsize=17, fontweight=
267       "bold", color="#111827")
268 ax.text(
269     0.50,
270     0.12,
271     "附件数据支持前三项检查；门店效果仍需通过实际试运行和持续记录进行验证。",
272     ha="center",
273     va="center",
274     fontsize=10.3,
275     color="#374151",
276 )
277 fig.tight_layout()
278 save(fig, "fig_model_evidence_chain.png")
279
280
281 def plot_operation_timeline() -> None:
282     fig, ax = plt.subplots(figsize=(13.6, 5.8))
283     ax.set_xlim(0, 1)
284     ax.set_ylim(0, 1)
285     ax.axis("off")
286     stages = [
287         (0.03, "闭店后", "销量、库存、报损\n更新潜在需求", "#E8F5F2", "#2A9D8F"),
288         (0.225, "凌晨订货", "供应报价、可供量\n生成成本情景", "#EDF4F8", "#457B9D"),
289         (0.42, "到货后", "实际成本揭示\n按规则更新售价", "#F2EEF8", "#6A4C93"),
290         (0.615, "营业中", "补架、售罄、折价\n记录服务状态", "#FFF1EE", "#E76F51"),
291         (0.81, "次日复核", "误差、服务率、利润\n触发滚动重估", "#F4F1EE", "#8D6E63"),
292     ]
293 for x, title, body, fill, edge in stages:
294     card = FancyBboxPatch(
295         (x, 0.31),
296         0.16,
297         0.42,
298         boxstyle="round,pad=0.015,rounding_size=0.025",
299         facecolor=fill,
300         edgecolor=edge,
301         linewidth=2.0,
302     )

```

```

299     ax.add_patch(card)
300     ax.text(x + 0.08, 0.62, title, ha="center", va="center", fontsize=12.8, fontweight="bold",
301           color=edge)
302     ax.text(x + 0.08, 0.46, body, ha="center", va="center", fontsize=9.8, linespacing=1.55,
303           color="#1F2937")
304     for x in (0.195, 0.390, 0.585, 0.780):
305         ax.add_patch(FancyArrowPatch((x, 0.52), (x + 0.025, 0.52), arrowstyle="-|>", mutation_scale
306                                     =17, color="#64748B", linewidth=1.7))
307     ax.add_patch(
308         FancyArrowPatch(
309             (0.89, 0.28),
310             (0.11, 0.28),
311             connectionstyle="arc3,rad=-0.22",
312             arrowstyle="-|>",
313             mutation_scale=17,
314             color="#64748B",
315             linewidth=1.7,
316         )
317     )
318     ax.text(0.5, 0.88, "门店每日数据更新与决策顺序", ha="center", va="center", fontsize=17,
319           fontweight="bold", color="#111827")
320     ax.text(0.5, 0.10, "各项记录分别用于更新需求、成本、约束或验证指标。", ha="center", va="center",
321           fontsize=10.2, color="#374151")
322     fig.tight_layout()
323     save(fig, "fig_daily_operation_timeline.png")
324
325 def plot_conclusion_summary() -> None:
326     summary = json.loads((ROOT / "outputs" / "model_summary.json").read_text(encoding="utf-8"))
327     edge = summary["q1"]["network_edges"][0]
328     elasticities = [row["决策用自身弹性"] for row in summary["q1"]["elasticity_details"]]
329     ensemble_wape = [
330         row["WAPE"]
331         for row in summary["q2"]["forecast_accuracy"]
332         if row["对象"] == "销量" and row["模型"] == "误差倒数加权组合"
333     ]
334     services = [row["综合需求满足率"] for row in summary["q3"]["service"]]
335     top_data = [row["拟采集数据"] for row in summary["q4"]["priorities"][:2]]
336
337     cards = [
338         (
339             "问题一 | 分布与关联",
340             f"最强条件边: {edge['品类1']}—{edge['品类2']} ({edge['条件相关']:.3f}) \n"
341             f"自身价格弹性: {min(elasticities):.3f} - {max(elasticities):.3f} \n"
342             "结论边界: 条件相关不等于因果互补",
343             "#E8F5F2",
344             "#2A9D8F",
345         ),
346         (
347             "问题二 | 定价与补货",
348             f"组合销量WAPE: {min(ensemble_wape):.1%} - {max(ensemble_wape):.1%} \n"
349             f"一周补货: {summary['q2']['week_order_qty']:.1f} kg \n"
350             f"情景期望利润: {summary['q2']['week_expected_profit']:.2f} 元",
351             "#EDF4F8",
352             "#457B9D",
353         ),
354         (
355             "问题三 | 单品组合",
356             f"{summary['q3']['candidate_count']}个候选中选择{summary['q3']['selected_count']}个 \n"
357             f"总补货: {summary['q3']['total_order']:.1f} kg; 期望利润: {summary['q3'][''
358             expected_profit']:.2f} 元 \n"
359             f"预计销量满足率: {min(services):.1%} - {max(services):.1%}",
360             "#F2EEF8",
361             "#6A4C93",
362         )
363     ]

```

```

357     ),
358     (
359         "问题四 | 新增数据",
360         f"首要: {top_data[0]}\n",
361         f"其次: {top_data[1]}\n",
362         "作用: 修正潜在需求并动态估计损耗",
363         "#FFF1EE",
364         "#E76F51",
365     ),
366 ]
367
368 fig, ax = plt.subplots(figsize=(13.2, 7.2))
369 ax.set_xlim(0, 1)
370 ax.set_ylim(0, 1)
371 ax.axis("off")
372 positions = [(0.04, 0.54), (0.52, 0.54), (0.04, 0.08), (0.52, 0.08)]
373 for (title, body, fill, edge_color), (x, y) in zip(cards, positions):
374     card = FancyBboxPatch(
375         (x, y),
376         0.44,
377         0.34,
378         boxstyle="round,pad=0.018,rounding_size=0.025",
379         facecolor=fill,
380         edgecolor=edge_color,
381         linewidth=2.1,
382     )
383     ax.add_patch(card)
384     ax.text(x + 0.22, y + 0.265, title, ha="center", va="center", fontsize=13.2, fontweight="
bold", color=edge_color)
385     ax.text(x + 0.22, y + 0.135, body, ha="center", va="center", fontsize=10.2, linespacing
=1.55, color="#1F2937")
386 ax.text(0.50, 0.955, "四问核心结论总览", ha="center", va="center", fontsize=17, fontweight="
bold", color="#111827")
387 fig.tight_layout()
388 save(fig, "fig_four_question_summary.png")
389
390
391 def main() -> None:
392     configure_plotting()
393     plot_hurdle_summary()
394     plot_forecast_comparison()
395     plot_q2_policy_matrix()
396     plot_q3_item_plan()
397     plot_evidence_chain()
398     plot_operation_timeline()
399     plot_conclusion_summary()
400     for name in (
401         "fig_q1_hurdle_gamma_summary.png",
402         "fig_q2_forecast_model_comparison.png",
403         "fig_q2_full_policy_matrix.png",
404         "fig_q3_selected_item_plan.png",
405         "fig_model_evidence_chain.png",
406         "fig_daily_operation_timeline.png",
407         "fig_four_question_summary.png",
408     ):
409         print(FIGURES / name)
410
411 if __name__ == "__main__":
412     main()
413

```

## 附录 O 逐问建模思路图绘制代码

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import matplotlib.pyplot as plt
6 from matplotlib import font_manager
7 from matplotlib.patches import Ellipse, FancyArrowPatch, FancyBboxPatch, Rectangle
8
9
10 def find_project_root() -> Path:
11     for candidate in Path(__file__).resolve().parents:
12         if (candidate / "outputs" / "tables").is_dir():
13             return candidate
14     raise FileNotFoundError("Cannot locate the project output tables.")
15
16
17 ROOT = find_project_root()
18 FIGURES = ROOT / "outputs" / "figures"
19 PAPER_FIGURES = ROOT / "paper" / "figures" if (ROOT / "paper" / "main.tex").is_file() else ROOT / "
    figures"
20
21 INK = "#364152"
22 MUTED = "#667085"
23 BLUE = "#E7F0F7"
24 ORANGE = "#F8E9D7"
25 GRAY = "#EEF0F2"
26 GREEN = "#E6F2EC"
27 WHITE = "#FFFFFF"
28
29
30 def configure_plotting() -> None:
31     candidates = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial Unicode MS"]
32     available = {font.name for font in font_manager.fontManager.ttflist}
33     chosen = next((name for name in candidates if name in available), "DejaVu Sans")
34     plt.rcParams.update(
35         {
36             "font.sans-serif": [chosen, "DejaVu Sans"],
37             "axes.unicode_minus": False,
38             "figure.dpi": 160,
39             "savefig.dpi": 260,
40         }
41     )
42
43
44 def node(ax, xy, text, *, width=1.62, height=0.76, face=GRAY, ellipse=False, fontsize=9.2):
45     x, y = xy
46     if ellipse:
47         patch = Ellipse((x, y), width, height, facecolor=face, edgecolor=INK, linewidth=1.25)
48     else:
49         patch = FancyBboxPatch(
50             (x - width / 2, y - height / 2),
51             width,
52             height,
53             boxstyle="round,pad=0.035,rounding_size=0.035",
54             facecolor=face,
55             edgecolor=INK,
56             linewidth=1.15,
57         )
58     ax.add_patch(patch)
59     ax.text(x, y, text, ha="center", va="center", fontsize=fontsize, color="#252B37", linespacing
        =1.22)
60
61
62 def arrow(ax, start, end, *, label=None, rad=0.0, color=INK, label_offset=(0.0, 0.13)):

```

```

63     patch = FancyArrowPatch(
64         start,
65         end,
66         arrowstyle="-|>",
67         mutation_scale=12,
68         linewidth=1.2,
69         color=color,
70         connectionstyle=f"arc3,rad={rad}",
71         shrinkA=4,
72         shrinkB=4,
73     )
74     ax.add_patch(patch)
75     if label:
76         ax.text(
77             (start[0] + end[0]) / 2 + label_offset[0],
78             (start[1] + end[1]) / 2 + label_offset[1],
79             label,
80             ha="center",
81             va="center",
82             fontsize=7.8,
83             color=MUTED,
84         )
85
86
87 def group(ax, x0, y0, width, height, label):
88     ax.add_patch(
89         Rectangle(
90             (x0, y0),
91             width,
92             height,
93             fill=False,
94             edgecolor="#586575",
95             linewidth=1.15,
96             linestyle=(0, (5, 4)),
97         )
98     )
99     ax.text(x0 + width / 2, y0 - 0.20, label, ha="center", va="top", fontsize=9.1, fontweight="bold",
100            , color=INK)
101
102
103 def canvas(title):
104     fig, ax = plt.subplots(figsize=(14.8, 4.45))
105     ax.set_xlim(0, 14.8)
106     ax.set_ylim(0, 4.45)
107     ax.axis("off")
108     ax.text(7.4, 4.20, title, ha="center", va="center", fontsize=12.2, fontweight="bold", color="#263B56")
109     return fig, ax
110
111
112 def save(fig, filename):
113     FIGURES.mkdir(parents=True, exist_ok=True)
114     PAPER_FIGURES.mkdir(parents=True, exist_ok=True)
115     for directory in (FIGURES, PAPER_FIGURES):
116         fig.savefig(directory / filename, bbox_inches="tight", facecolor=WHITE, pad_inches=0.05)
117     plt.close(fig)
118
119
120 def question_one():
121     fig, ax = canvas("问题一：销量分布、品类关联与价格响应")
122     group(ax, 0.25, 0.64, 3.15, 2.95, "数据检查与潜在需求估计")
123     group(ax, 3.65, 0.64, 6.95, 2.95, "销量分布与条件关联")
124     group(ax, 10.85, 0.64, 3.65, 2.95, "价格响应与结果检查")

```

```

125 specs = [
126     (1.00, 1.32, "销售流水\n退货与折扣", BLUE, False),
127     (2.55, 1.38, "Hurdle--Gamma\n疑似售罄删失", ORANGE, False),
128     (4.25, 1.38, "多尺度分解\n趋势·周内·年度", GRAY, False),
129     (5.90, 1.28, "边际分布\nAIC/BIC/KS", BLUE, False),
130     (7.55, 1.34, "经验秩变换\nCopula 残差", GRAY, False),
131     (9.10, 1.26, "稀疏条件\n需求网络", ORANGE, False),
132     (11.55, 1.32, "工具变量\n岭弹性", ORANGE, True),
133     (13.55, 1.40, "联合情景参数\n弹性矩阵", GREEN, False),
134 ]
135 for x, width, text, face, is_ellipse in specs:
136     node(ax, (x, 2.48), text, face=face, width=width, ellipse=is_ellipse)
137 for left, right in zip(specs, specs[1:]):
138     arrow(ax, (left[0] + left[1] / 2 + 0.02, 2.48), (right[0] - right[1] / 2 - 0.02, 2.48))
139 arrow(ax, (13.55, 2.06), (4.25, 1.55), label="核对价格系数符号、残差结构和疑似售罄标记", rad
    =0.06, color="#7A4E2D", label_offset=(0, -0.12))
140 save(fig, "fig_q1_modeling_route.png")
141
142
143 def question_two():
144     fig, ax = canvas("问题二：预测误差驱动的两阶段定价与补货")
145     group(ax, 0.25, 0.64, 4.55, 2.95, "组合预测模块")
146     group(ax, 5.05, 0.64, 5.45, 2.95, "联合情景与随机决策模块")
147     group(ax, 10.75, 0.64, 3.75, 2.95, "检验与执行模块")
148
149     node(ax, (1.08, 2.75), "动态岭回归", face=GRAY, width=1.34, height=0.55)
150     node(ax, (1.08, 1.98), "Fourier 日历", face=GRAY, width=1.34, height=0.55)
151     node(ax, (1.08, 1.21), "四周同星期", face=GRAY, width=1.34, height=0.55)
152     node(ax, (2.85, 2.48), "84 日滚动回溯\n误差倒数加权", face=ORANGE, width=1.36)
153     node(ax, (4.12, 2.48), "需求与成本\n组合预测", face=BLUE, width=1.05)
154     route = [
155         (2.85, 1.36),
156         (4.12, 1.05),
157         (5.75, 1.35),
158         (7.40, 1.25),
159         (9.20, 1.40),
160         (11.55, 1.35),
161         (13.50, 1.42),
162     ]
163     node(ax, (5.75, 2.48), "同日残差\n联合重抽样", face=GRAY, width=1.35)
164     node(ax, (7.40, 2.48), "弹性矩阵\n修正需求", face=BLUE, width=1.25)
165     node(ax, (9.20, 2.48), "两阶段随机\n报童模型", face=ORANGE, width=1.40, ellipse=True)
166     node(ax, (11.55, 2.48), "服务率·缺货率\n尾部利润", face=GRAY, width=1.35)
167     node(ax, (13.50, 2.48), "七日补货量\n成本响应售价", face=GREEN, width=1.42)
168
169     for y in (2.75, 1.98, 1.21):
170         arrow(ax, (1.76, y), (2.15, 2.48))
171     for left, right in zip(route, route[1:]):
172         arrow(ax, (left[0] + left[1] / 2 + 0.02, 2.48), (right[0] - right[1] / 2 - 0.02, 2.48))
173     arrow(ax, (13.50, 2.04), (2.85, 1.47), label="次日使用新误差更新模型权重和联合情景", rad=0.07,
        color="#7A4E2D", label_offset=(0, -0.13))
174     save(fig, "fig_q2_modeling_route.png")
175
176
177 def question_three():
178     fig, ax = canvas("问题三：品类总量约束下的字典序单品组合")
179     group(ax, 0.25, 0.64, 4.45, 2.95, "候选构造与层次需求")
180     group(ax, 4.95, 0.64, 5.80, 2.95, "两阶段组合优化")
181     group(ax, 11.00, 0.64, 3.50, 2.95, "结果与约束检查")
182
183     specs = [
184         (1.00, 1.28, "近一周可售\n单品候选", BLUE, False),
185         (2.60, 1.28, "品类需求总量\n选择型 MNL", ORANGE, False),
186         (4.05, 1.18, "网络重复度\n单品利润代理", GRAY, False),

```

```

187     (5.80, 1.50, "第一阶段 MILP\n最大化最弱捕获率", ORANGE, False),
188     (7.60, 1.18, "允许下降\n1 个百分点", BLUE, False),
189     (9.35, 1.45, "第二阶段\n最大化情景利润", ORANGE, True),
190     (11.60, 1.35, "数量 · 陈列量\n品类覆盖检查", GRAY, False),
191     (13.55, 1.40, "33 个单品\n定价补货方案", GREEN, False),
192 ]
193 for x, width, text, face, is_ellipse in specs:
194     node(ax, (x, 2.48), text, face=face, width=width, ellipse=is_ellipse)
195 for left, right in zip(specs, specs[1:]):
196     arrow(ax, (left[0] + left[1] / 2 + 0.02, 2.48), (right[0] - right[1] / 2 - 0.02, 2.48))
197 arrow(ax, (13.55, 2.04), (5.80, 1.48), label="约束不满足时返回组合层重新计算", rad=0.07, color=
198 "#7A4E2D", label_offset=(0, -0.13))
199 save(fig, "fig_q3_modeling_route.png")
200
201 def question_four():
202     fig, ax = canvas("问题四：新增数据的用途与评价")
203     group(ax, 0.25, 0.64, 4.15, 2.95, "确定采集优先级")
204     group(ax, 4.65, 0.64, 5.70, 2.95, "更新模型参数和约束")
205     group(ax, 10.60, 0.64, 3.90, 2.95, "试点与日常评价")
206
207     specs = [
208         (1.00, 1.30, "库存截断\n损耗 · 供应缺口", BLUE, False),
209         (2.65, 1.35, "EVSI/采集成本\n专家预筛选", ORANGE, False),
210         (4.00, 1.00, "采集清单\n与字段口径", GRAY, False),
211         (5.70, 1.32, "库存与售罄\n修正潜在需求", BLUE, False),
212         (7.40, 1.35, "报价与可供量\n更新成本约束", GRAY, False),
213         (9.25, 1.45, "购物篮 · 陈列\n更新网络与容量", BLUE, False),
214         (11.55, 1.30, "A/B 试点\n滚动回测", ORANGE, True),
215         (13.55, 1.40, "每日重优化\n误差归因", GREEN, False),
216     ]
217 for x, width, text, face, is_ellipse in specs:
218     node(ax, (x, 2.48), text, face=face, width=width, ellipse=is_ellipse)
219 for left, right in zip(specs, specs[1:]):
220     arrow(ax, (left[0] + left[1] / 2 + 0.02, 2.48), (right[0] - right[1] / 2 - 0.02, 2.48))
221 arrow(ax, (13.55, 2.04), (2.65, 1.47), label="试点后按实际利润和服务率调整采集优先级", rad
222 =0.07, color="#7A4E2D", label_offset=(0, -0.13))
223 save(fig, "fig_q4_modeling_route.png")
224
225 def main() -> None:
226     configure_plotting()
227     question_one()
228     question_two()
229     question_three()
230     question_four()
231     print("Created four C228-style per-question modeling diagrams.")
232
233
234 if __name__ == "__main__":
235     main()

```